

Lecture 7 — Software Testing

Lecture id: L07 **Source decks:** Software Testing (~83p) **Labs:** TestLab1 (~1p) **Process phase(s):** Verification **Citation key:** (Software Testing p.X) ; readings [Raj13] (Rajlich, *Software Engineering: The Current Practice*), [MC09] (Martin & Coplien, *Clean Code*), [GHJV94] (Gang of Four, *Design Patterns*), [Fowler99] (Fowler, *Refactoring*), [Martin] (Martin, *Clean Code / Clean Architecture*). **Grounding note:** Every claim below is anchored to a specific page of the Software Testing deck via (Software Testing p.X) and to TestLab1 where relevant. The deck is slide-based, so several pages are diagram-only (e.g. p.15–17 Assertions visuals, p.24–28 Fragile-Test sensitivities, p.36–38 Traditional-vs-TDD diagrams, p.61–68 Mockito problem/solution visuals); for those pages the surrounding textual slides and the standard testing canon (Dijkstra, Beck, Fowler, Mockito docs) are used to expand the diagram into prose, and any such expansion is flagged. Reading-key citations [Raj13] etc. mark where the course textbook and the named secondary readings reinforce a slide claim; they are pointers to the controlled reading list, not verbatim page cites. Where the deck only shows a label, this guide says so rather than inventing detail.

Overview

Lecture 7 is the **Verification** phase of the SB5-MAI software-change process. In Rajlich's change mini-cycle (Initiation → Concept Location → Impact Analysis → Prefactoring → Actualization → Postfactoring → Conclusion → **Verification**), Verification is where you *prove that the change you made does what it should and broke nothing it shouldn't* [Raj13]. Testing is the engine of Verification: it is the activity that "make[s] software fail" on purpose so that defects surface in the lab instead of in production (Software Testing p.1).

The deck opens with the **theoretical limits of testing**. Because of Turing's halting problem, it is *theoretically impossible to build a perfect test suite* (Software Testing p.5); the best a programmer can do is design tests that "come close to be adequate" (Software Testing p.5). This is the engineering reality captured by Dijkstra's maxim, quoted verbatim on the slides: **"Testing can demonstrate the presence of bugs, but not their absence."** (Software Testing p.6). Residual bugs can always hide in untested paths (Software Testing p.6). Verification therefore is not about *certainty*; it is about *systematic risk reduction*.

From that foundation the deck builds out the full testing toolbox a maintainer needs:

1. **A vocabulary of testing kinds** — unit, integration, system, differential, stress, random — organised along the white-box / black-box axis (Software Testing p.8).
2. **A diagnostic discipline** — when a test fails, *where* is the fault? In the software under test (SUT), the acceptability test, the specification, or the platform (OS/compiler/libs/hardware)? (Software Testing p.9). The Mars Climate Orbiter (metric vs. English units) is the cautionary tale (Software Testing p.9).
3. **The mechanics of making code testable** — clean code, refactoring, no hidden threads/globals, fault injection, and above all **assertions** (Software Testing p.14, p.18–21).
4. **The fragile-test problem** — interface, behavior, data, and context sensitivity, plus the special difficulty of testing GUIs, time-dependent code, and code with external dependencies (Software Testing p.24–32).
5. **Test-Driven Development (TDD)** — red/green/refactor, worked end-to-end on a "Borrow Book" use case (Software Testing p.34–49).

6. **JUnit** — the xUnit framework, lifecycle annotations, assertions, suites, exception tests (Software Testing p.50–56, p.76).
7. **Test doubles via Mockito** — mocks, stubs, spies, `when/thenReturn`, `verify`, and the limitations of mocking (Software Testing p.57–75).
8. **Acceptance & regression testing** — user-defined tests, manual vs. automated, and why automation enables regression testing (Software Testing p.77–81).
9. **The tester's mindset** — "Developer wants the code to succeed; Tester wants the code to fail" (Software Testing p.83).

For the JHotDraw case study (the course's running maintenance subject) and for the lab/portfolio, the actionable core is: write **JUnit 4** unit tests for the *domain-logic* methods of the feature you changed, cover best-case and **boundary** cases, replace external dependencies with **mocks/stubs**, and use **Java assertions** to encode invariants (TestLab1 p.1). That body of tests is the evidence you present at the Verification step.

Learning Objectives

After this lecture you should be able to:

1. **Explain why testing is incomplete in principle** — connect Turing's halting problem to the impossibility of a perfect test suite, and state Dijkstra's "presence, not absence" principle (Software Testing p.4–6).
 2. **Classify a test** along two axes: by level (unit / integration / system) and by knowledge (white-box / black-box), and place differential, stress, and random testing in that map (Software Testing p.8).
 3. **Diagnose a failing test** — walk the decision tree to decide whether the bug is in the SUT, the acceptability test, the specification, or the platform (Software Testing p.9).
 4. **Make software testable** — apply clean code, remove hidden threads/global-variable swaps/pointer soup, add module unit tests, support fault injection, and write assertions (Software Testing p.14).
 5. **Use assertions correctly** — write executable invariant checks, enable/disable them, and obey the three rules: not for error handling, no side effects, no silly assertions (Software Testing p.17–21).
 6. **Recognise and avoid the fragile-test problem** — name the four sensitivities (interface, behavior, data, context) and the strategy "test under the UI" (Software Testing p.24–30).
 7. **Design tests with black-box techniques** — equivalence partitioning of input/output space and boundary-value analysis — and reason about white-box coverage (Software Testing p.11–12, p.23).
 8. **Practise TDD** — drive a feature with red/green/refactor, generate test ideas, and use refactoring as the mandatory third step (Software Testing p.34–49).
 9. **Write JUnit tests** — use `@Before/@After/@BeforeClass/@AfterClass`, the assertion family, `@Test(expected=...)`, and test suites (Software Testing p.51–56).
 10. **Apply test doubles** — distinguish mock / stub / spy, configure them with Mockito (`mock()`, `@Mock`, `when().thenReturn()`, `verify()`, `@Spy`, `@InjectMocks`), and know Mockito's limitations (Software Testing p.57–75).
 11. **Define acceptance and regression tests** and argue for automation over manual testing (Software Testing p.77–81).
 12. **Adopt the dual mindset** of developer-who-wants-success and tester-who-wants-failure (Software Testing p.83).
-

Key Concepts

This is the bulk of the guide. Each concept is self-contained and cited.

Why testing is incomplete (the theoretical floor)

What it is. This is the foundational result that *no test suite can ever be perfect* — testing has a hard ceiling baked into the mathematics of computation, not just a practical one. A **Turing machine** is "a simple mathematical model of computation [that] manipulates symbols on a tape based on rules, yet can implement any computer algorithm" (Software Testing p.3) — in other words, the simplest possible model that is still powerful enough to express *any* computation a real computer can perform. All mainstream languages — JAVA, C#, C++, C — are Turing-complete (Software Testing p.4), meaning each can express anything a Turing machine can and therefore inherits Turing's **halting problem**: there is no general algorithm that, given an arbitrary program and input, decides whether that program eventually halts or loops forever. Because you cannot even decide *halting* in general, you certainly cannot decide *correctness* in general, so the deck draws the consequence explicitly: there is a **"theoretical reason for testing incompleteness"** and it is **"theoretically impossible to create a perfect test suite"** (Software Testing p.5). Testing techniques "cannot guarantee a complete correctness of software"; only "well designed tests come close to be adequate" (Software Testing p.5).

What it's used for / why it matters. This result is what tells you what testing *can and cannot deliver*, so you set the right expectation. The practical statement is Dijkstra's principle, on the slide verbatim: **"Testing can demonstrate the presence of bugs, but not their absence."** (Software Testing p.6). A test that fails proves a bug exists (presence); a test that passes proves only that *this one input* behaved, never that *all* inputs will (absence). The corollary: "Residual bugs can still hide in the code, undetected by tests, as no test suite guarantees an error-free program." (Software Testing p.6) — there are infinitely many input paths and your finite suite touches only some of them. The benefit of internalising this is twofold: it stops you over-claiming ("all green, therefore correct"), and it *motivates* the disciplined design techniques (equivalence partitioning, boundary analysis, coverage) that squeeze the most assurance out of a finite test budget.

When & how it applies. Whenever you finish a green test run and are tempted to call the change "proven," recall that you have shown *evidence of working features*, never a *proof of correctness*. Concretely: a sort routine that passes 1,000 random arrays is not proven correct — an adversarial input (e.g. all-equal elements, or `Integer.MIN_VALUE`) may still break it. This humility is exactly what justifies the whole Verification phase [Raj13]: Verification is systematic *risk reduction*, not certification of perfection.

The basic test model (input → SUT → output → oracle)

What it is. This is the universal anatomy that *every* test shares, regardless of level or framework. Every test instantiates the same machine: **test inputs** go into the **Software Under Test (SUT)**, which produces **test outputs**, which are then judged **"Outputs OK?"** by a test **oracle** (Software Testing p.7). The SUT is whatever you are exercising (a method, a module, a whole system); the oracle is the decision procedure that compares the actual output against what *should* have happened and emits pass/fail.

What it's used for / why it matters. The model exists to make explicit the part testers most often get wrong: the **oracle**. A test is only as trustworthy as its oracle — if the oracle encodes the wrong expectation, a "passing" test is a lie and a "failing" test sends you hunting a non-bug (this is the "bug in the acceptance test" branch of the p.9 triage). Choosing a *correct* and *cheap* oracle is therefore half the art of testing: you want a judge that is right but does not cost as much to build as the SUT itself.

When & how it applies. The oracle takes different concrete forms depending on what you know. For a typical unit test the oracle is a hard-coded expected value, e.g. `assertEquals(expected, actual)` — cheap and exact when you can compute the right answer by hand. When you *cannot* easily state the right answer, you reach for the other oracle forms the deck shows later: a **reference implementation** the output must match (differential testing — "is the new version's output `=?` the old version's"), or an **invariant** that must always hold regardless of the specific value (`checkRep()` , e.g. "a queue's size is always between 0 and max"). Example: testing a new fast `sqrt` , you may not know `sqrt(2)` to the last bit, but you *do* know `(result * result)` must be within epsilon of 2 — that property is a perfectly good oracle.

Testing levels: unit, integration, system

What they are. The deck arranges the testing kinds in one diagram (Software Testing p.8). The three **levels** form a *scope hierarchy* — they differ in how much of the system a single test exercises at once, from one method up to the whole running product. Each level catches a different *class* of fault, which is why you want all three rather than just one.

- **Unit testing** — tests the smallest testable unit in isolation. *What it is:* a fast, focused check on one piece of code with everything around it held still. TestLab1 defines it precisely: "individual units of source code ... are tested to determine whether they are fit for use ... In object-oriented programming, a unit is often an entire interface, such as a class, but could be an individual method." (TestLab1 p.1). Unit tests are "typically automated tests written and run by software developers" (TestLab1 p.1). *What it's for:* it catches *logic* faults inside a single component — a wrong condition, an off-by-one, a mishandled edge — and it pinpoints them precisely because nothing else is in the way; the deck names units as the right tool for "complex behaviour of classes" (Software Testing p.40). *When & how:* the moment execution would *leave* the method under test (a call to a database, a clock, a network service) you have a *dependency* that destroys isolation, so you replace it with a mock/stub, keeping the test a single code-path through a single method (TestLab1 p.1). Example: testing `User.borrowBook` you stub out the `OverdueService` so the test exercises only the borrowing logic, not the network.
- **Integration testing** — tests that already-unit-tested components work *together* across their interfaces (Software Testing p.8). *What it is:* the next layer up, where two or more units are wired together and tested as a pair/group with their real seams in place. *What it's for:* it catches *interaction* faults that unit tests structurally cannot see — mismatched assumptions at an interface, wrong call order, data-format disagreements between modules. The fragile-test sensitivities (next section) are largely integration concerns: an interface change can break the seam between two modules even when each module's own unit tests still pass. *When & how:* run it after the units are green, on the boundaries that matter most (e.g. does the persistence layer correctly hand a `Book` to the domain layer). The Mars Orbiter units-mismatch (p.9) is the archetypal integration/spec fault — each side was internally consistent, the bug lived in the seam.
- **System testing** — tests the whole assembled system end-to-end against its specification (Software Testing p.8). *What it is:* the broadest level, exercising the real, fully integrated product the way a user would. *What it's for:* it catches faults that only emerge from the *whole* — configuration, end-to-end workflows, requirements not met despite every part working — and validates the system against what the spec actually promised. *When & how:* run last, against acceptance criteria / use cases; e.g. "a user logs in, searches, borrows a book, and the loan appears on their account." TDD applies at all three levels: "All kind of tests: unit, component, and system tests" (Software Testing p.35), so the red/green/refactor discipline is not unit-only.

Differential, stress, and random testing

What they are. The same p.8 diagram adds three kinds that cut *across* the levels — they are not a fourth scope tier but *strategies* you can apply at unit, integration, or system scope. Each one solves the oracle problem or the input-selection problem in a different way.

- **Differential testing** — run the SUT and a *reference* (e.g. an older version, an alternative implementation, or a parallel of `=?`) on the same inputs and assert the outputs match (the slide marks it "differential testing ... `=?`", Software Testing p.8). *What it's for:* it solves the *oracle* problem when you cannot easily state the right answer by hand — the other implementation is the oracle, so you only need agreement, not absolute truth. It catches divergences a hand-written test might miss. *When & how, in maintenance:* this is especially powerful here because the *previous release is a free oracle for the new one* — run the old and new versions on the same workload and any mismatch flags either a regression or an intended behaviour change to review (a regression-flavoured idea, see Regression below). Example: refactoring a tax calculator, feed last year's 50,000 real records through old and new code and require identical output.
- **Stress testing** — push the system past expected load/volume/rate to find where it breaks (Software Testing p.8). *What it's for:* it is *not* about correctness on a single input; it targets *non-functional* failure modes — memory leaks, deadlocks, throughput collapse, resource exhaustion — that appear only under sustained or extreme conditions and never show up in a normal unit test. *When & how:* apply at system level before release; e.g. simulate 10× the expected concurrent users, or feed a 100× larger file, and watch where latency, memory, or correctness degrades.
- **Random testing** (fuzzing-adjacent) — generate random/ `?` inputs to explore the input space cheaply and surface inputs the author never considered (Software Testing p.8). *What it's for:* humans test the cases they *thought of*; random testing finds the ones they *didn't* — malformed inputs, weird Unicode, extreme numbers — that trigger crashes or unhandled exceptions. *When & how:* point a generator at a parser/decoder and run thousands of random byte sequences, asserting only that it never crashes or violates an invariant; pair it with `checkRep`-style oracles since you cannot pre-compute the "right" answer for random input.

White-box vs. black-box testing

What it is. The horizontal axis of the p.8 map is **white box vs. black box** (Software Testing p.8) — the two ways of *knowing what to test*, distinguished by *how much of the implementation the tester is allowed to look at*. They are complementary perspectives on the same SUT, not rivals: a thorough suite uses both.

- **Black-box testing** — the tester sees only the **specification / interface**, not the code. *What it is:* you treat the SUT as an opaque box and derive tests purely from the contract — given this input domain, the output must fall in this range. *What it's for:* it validates *what the software is supposed to do* independently of how it does it, so it survives internal rewrites (a refactor that preserves behaviour leaves black-box tests green) and it catches *missing* functionality that white-box testing cannot, because you cannot cover a code path that was never written. The deck's specification slide shows exactly this view:
`sqrt: R+ -> R+, sqrt: R -> C, sqrt: F+ -> F+, sqrt: F -> F+ U exception` map domain to range (Software Testing p.23). *When & how:* design from the spec using equivalence partitioning and boundary-value analysis (next two concepts) — e.g. for `sqrt` you derive test classes (positive, zero, negative, NaN) straight off the domain→range table without ever reading the algorithm.
- **White-box (glass-box) testing** — the tester sees the **source code** and designs tests to exercise its internal structure: every statement, every branch, every path. *What it is:* you open the box and aim tests at the *implementation's* routes. *What it's for:* it catches faults tied to *how* the code is written — an unreachable branch, an unhandled internal case, a special path the spec never mentioned — and it lets you

measure thoroughness via coverage. *When & how:* the "Testing the Queue" slides (Software Testing p.13) are white-box: the tester *knows* there is a `tail` wrap-around (`if (tail == max) tail = 0;`, Software Testing p.11) and writes `testEnqueueTail()` specifically to drive `getTail()` back to 0 (Software Testing p.13) — a test you would never think to write from the spec alone, because "wrap-around" is an implementation detail. White-box knowledge is precisely what lets you target **coverage criteria** (below).

Equivalence partitioning (black-box design technique)

What it is. A black-box test-design technique that shrinks an infinite (or huge) input space to a handful of tests by grouping inputs the program *should treat identically*. Shown on the "Equivalent Tests" slide (Software Testing p.12): partition the **input space** into classes of inputs that the program should handle the same way, then test exactly *one representative per class* — testing more members of the same class is redundant because they all exercise the same logic. The slide draws an *input space* and an *output space* and asks which inputs are "equivalent": `q.enqueue(7); x=q.dequeue(); if(x==7) success` is in the same equivalence class as the same code with a `sleep(100)` inserted, because the queue's contract is time-independent — the delay cannot change the result, so both are one class (Software Testing p.12). It also flags the **output-space** partition "Large Integers?" — values near the edge of the representable range form their own class because they may overflow (Software Testing p.12).

What it's used for / why it matters. It is the tool that turns the impossibility result of p.5 into a *practical, finite* plan: you cannot test every input, but you *can* test one representative of every behaviourally distinct group, and that gives *adequate, justifiable* coverage of an infinite domain with a small, defensible set of tests. It also prevents the common waste of writing twenty tests that all hit the same code path while leaving a whole class untested.

When & how it's applied. *Worked case — a `grade(score)` function* with rules: `<0` invalid, `0–59` "F", `60–100` "pass", `>100` invalid. The classes are: {negative}, {0–59}, {60–100}, {>100}. You write *one* test each — say `grade(-5)`, `grade(30)`, `grade(75)`, `grade(150)` — and that is sufficient *partition* coverage; adding `grade(31)` and `grade(32)` buys nothing because they sit in the same class as `grade(30)`. (You then add boundary-value tests on top, next concept, because the *edges* of these classes are where defects cluster.) For the queue, the classes include "queue has room" vs "queue is full", which is why one enqueue-when-full test stands in for all full-queue enqueues.

Boundary-value analysis (black-box design technique)

What it is. The companion to equivalence partitioning: a black-box technique that targets the *edges* of each equivalence class — the exact values where one class turns into the next — rather than the comfortable middle. Where partitioning picks a representative *inside* each class, boundary analysis picks the values *at the seams*.

What it's used for / why it matters. Defects cluster at the edges. Off-by-one errors (`<` vs `<=`), overflow at the maximum, empty-collection handling, and the first/last iteration of a loop all live exactly at boundaries — so testing the middle of a class often passes while the edge silently fails. Boundary analysis is the cheapest way to catch the single most common family of bugs. TestLab1 makes it a *required* step: "Write JUnit tests for identified boundary cases" (TestLab1 p.1), so it is both an exam point and a lab deliverable.

When & how it's applied. The Fixed Size Queue is the canonical boundary playground (Software Testing p.11): a queue of `max=2` — enqueue into an empty queue (boundary: `size==0`), enqueue until full (boundary: `size==max`, where `enqueue` must return `false`), dequeue from empty (boundary: returns `null`), and the **wrap-around boundary** where `tail / head` hit `max` and reset to 0 (`if (tail == max) tail = 0;`, Software Testing p.11). The worked run `q=Queue(2); enqueue(6); enqueue(7); enqueue(8); dequeue() ×3` is expected to yield `true, true, false, 6, 7, null` (Software Testing p.11) — every value in that expected

tuple is a *boundary* outcome (full-queue reject, empty-queue null, the wrap point), which is why the deck chose it. **Rule of thumb:** for every numeric/size boundary B, test **B-1, B, and B+1**. Worked example for the `grade` function above: at the 60 boundary test `grade(59) → "F"`, `grade(60) → "pass"`, `grade(61) → "pass"`; at the 100 boundary test `grade(100)`, `grade(101)`. Those five tests find the `<-vs-<=` mistakes that the partition representatives `grade(30)` / `grade(75)` would never reveal.

The Fixed Size Queue — the deck's running SUT in full (p.11)

What it is. The deck's primary code example — a bounded FIFO ring buffer — appears in full on one slide (Software Testing p.11) and is then reused throughout the lecture: it is the subject of the equivalence-class discussion (p.12), the hand-rolled tests (p.13), the `checkRep` assertions (p.19), and the JUnit `EnqueueTest` / `DequeueTest` / `QueueTestSuite` (p.52, p.54). Knowing this class line-by-line therefore pays off across half the deck. The slide states the contract first: a **Fixed Size Queue** supports `enqueue`, `dequeue`, and guarantees **FIFO order** — its own worked one-liner is `enqueue(7); enqueue(8); dequeue(); → 7 then 8` (the slide spells it `enqueue(8)`; Software Testing p.11). Two small state diagrams accompany the code: one at `size=0` with `head` and `tail` pointing at the same slot, and one at `size=1` holding the value `7` with `head` and `tail` separated (Software Testing p.11).

The complete source, as shown on the slide (Software Testing p.11):

```
public class Queue {
    public Queue(int sizeMax) {
        assert sizeMax > 0;
        this.max = sizeMax;
        this.head = 0;
        this.tail = 0;
        this.size = 0;
        this.data = new int[sizeMax];
    }
    public boolean empty() {
        return size == 0;
    }
    public boolean full() {
        return size == max;
    }
    public boolean enqueue(int x) {
        if (size == max)
            return false;
        data[tail] = x;
        size += 1;
        tail += 1;
        if (tail == max)
            tail = 0;
        return true;
    }
    public Integer dequeue() {
        if (size == 0)
            return null;
        int x = data[head];
        size -= 1;
        head += 1;
        if (head == max) {
            head = 0;
        }
        return x;
    }
}
```

Member-by-member anatomy (why each line is exam-relevant):

- **Constructor** `Queue(int sizeMax)` — note the very first executable line is `assert sizeMax > 0;` (Software Testing p.11): a *precondition assertion* in the p.18 sense. A size of 0 or negative can only come from a *buggy caller* (no legitimate use case asks for a zero-capacity queue), so per Rule 1 it is an assertion, not an exception. The constructor then zeroes `head`, `tail`, `size` and allocates the backing array `data = new int[sizeMax]`.
- **`empty()` / `full()`** — pure boolean queries over `size` (`size == 0`, `size == max`); they are the observable state the tests assert on (e.g. `assertTrue(q.empty())` in p.52).
- **`enqueue(int x)` returns `boolean`** — the *full-queue* case is signalled by **returning `false`** without modifying anything (`if (size == max) return false;`). Otherwise it writes `data[tail] = x`, increments `size` and `tail`, and applies the **wrap-around rule** `if (tail == max) tail = 0;` — the ring-buffer mechanic that the white-box test `testEnqueueTail()` (p.13) and the JUnit `testEnqueue` (p.52) deliberately target.
- **`dequeue()` returns `Integer (boxed)`, not `int`** — the *empty-queue* case is signalled by **returning `null`** (`if (size == 0) return null;`), which is only possible because the return type is the boxed `Integer`. Otherwise it reads `data[head]`, decrements `size`, increments `head`, and applies the mirrored wrap-around `if (head == max) { head = 0; }`. *Exam point:* the two methods use **different error-signalling conventions** (boolean `false` vs `null`) — be able to say which method returns what on its boundary.

The worked run, traced call by call. The slide runs six calls against a queue of capacity 2 (Software Testing p.11):

```
q = Queue(2)
r1 = q.enqueue(6);
r2 = q.enqueue(7);
r3 = q.enqueue(8);
r4 = q.dequeue();
r5 = q.dequeue();
r6 = q.dequeue();
```

Call	Return s	size after	head after	tail after	data after	Why
<code>new Queue(2)</code>	—	0	0	0	<code>[_, _]</code>	<code>assert 2 > 0</code> passes; all counters zeroed
<code>enqueue(6)</code>	<code>true</code>	1	0	1	<code>[6, _]</code>	room available; write at tail 0
<code>enqueue(7)</code>	<code>true</code>	2	0	0	<code>[6, 7]</code>	write at tail 1; <code>tail</code> hits <code>max=2</code> → wraps to 0
<code>enqueue(8)</code>	<code>false</code>	2	0	0	<code>[6, 7]</code>	<code>size == max</code> → reject, no state change
<code>dequeue()</code>	6	1	1	0	<code>[6, 7]</code>	FIFO: oldest element (head 0) leaves first
<code>dequeue()</code>	7	0	0	0	<code>[6, 7]</code>	read at head 1; <code>head</code> hits <code>max</code> → wraps to 0
<code>dequeue()</code>	<code>null</code>	0	0	0	<code>[6, 7]</code>	<code>size == 0</code> → empty-queue signal

The slide's expected output line — `true, true, false, 6, 7, null` (Software Testing p.11) — is exactly the `r1..r6` column above. Notice that every interesting value in the tuple is a **boundary outcome** (full-queue reject, the two FIFO values across a wrap, empty-queue `null`), which is why this guide treats the run as the canonical boundary-value demonstration. Also note the queue's *physical array never erases values* —

dequeue only moves `head`; stale data (6 , 7) remains in `data[]` , invisible through the API. That is precisely the kind of internal detail only **white-box** tests and the `checkRep` invariant (p.19) can reason about.

Hand-rolled test drivers before JUnit — `testEquality` and `testEnqueueTail` (p.13)

What they are. Before introducing JUnit, the deck shows two tests for the Queue written *by hand* as plain `static boolean` methods that print to `out` and return a verdict (Software Testing p.13). They demonstrate that automated testing needs no framework — and simultaneously motivate why a framework is worth having, by exhibiting the boilerplate. Both appear verbatim on the slide:

```
public static boolean testEquality() {
    IQueue q = new Queue(2);
    boolean res = q.empty();
    if (!res) { out.println("test1 NOT OK"); return false; }
    res = q.enqueue(10);
    if (!res) { out.println("test1 NOT OK"); return false; }
    res = q.enqueue(11);
    if (!res) { out.println("test1 NOT OK"); return false; }
    int x = q.dequeue();
    if (x != 10) { out.println("test1 NOT OK"); return false; }
    x = q.dequeue();
    if (x != 11) { out.println("test1 NOT OK"); return false; }
    res = q.empty();
    if (!res) { out.println("test1 NOT OK"); return false; }
    out.println("test1 OK");
    return true;
}

public static boolean testEnqueueTail() {
    Queue q = new Queue(2);
    boolean res = q.empty();
    if (!res) { out.println("test2 NOT OK"); return false; }
    res = q.enqueue(1);
    if (!res) { out.println("test2 NOT OK"); return false; }
    res = q.enqueue(2);
    if (!res) { out.println("test2 NOT OK"); return false; }
    q.enqueue(3);
    if (q.getTail() != 0) { out.println("test2 NOT OK"); return false; }
    out.println("test2 OK");
    return true;
}
```

(Software Testing p.13; the slide's method name really is spelled `testEquality`.)

What each test checks — and the black/white-box contrast between them.

- **`testEquality` ("test1") is a black-box FIFO-contract test.** It is written against the `IQueue` interface (`IQueue q = new Queue(2);`) — the test only uses contract operations: starts empty, `enqueue(10)` and `enqueue(11)` both succeed, `dequeue()` yields 10 then 11 *in insertion order*, and the queue is empty again at the end (Software Testing p.13). Nothing in it depends on `head` / `tail` mechanics; any correct FIFO implementation passes it.
- **`testEnqueueTail` ("test2") is a white-box implementation test.** It is written against the **concrete** `Queue` class and ends with `if (q.getTail() != 0) ...` — asserting on the *internal* `tail` cursor through the `getTail()` accessor (Software Testing p.13). The scenario (capacity 2; enqueue 1, 2, then a third) deliberately drives the wrap-around path `if (tail == max) tail = 0;` from p.11. A tester who had never seen the source would have no reason to write this test; it exists *because* the tester knows the ring-buffer

implementation (Software Testing p.13). This pairing is the deck's concrete illustration of the white-box / black-box axis from p.8.

Why the deck shows them before JUnit. Every check needs four lines of ceremony (`if (!res) { out.println("...NOT OK"); return false; }`), the pass/fail protocol is manual (`println` + boolean return), there is no shared fixture, and somebody must remember to *call* these methods and read the console. Compare the JUnit version of the very same wrap-around scenario on p.52: `assertTrue(q.getTail() == 0);` — one line, automatic discovery via `@Test`, automatic reporting, fresh fixture via `@Before` (Software Testing p.52). The hand-rolled slide is the "before" picture that makes the framework's value self-evident: JUnit does not change *what* you test, it removes the boilerplate around *how*.

White-box coverage criteria (statement / branch / path)

What it is. Coverage criteria are *metrics that quantify how thoroughly your tests exercise the code's internal structure* — they answer "have I tested enough?" with a number instead of a hunch. They only make sense in white-box terms because you need to see the code to know which statements, branches, and paths exist. The deck establishes the *purpose* (drive every internal route) through the Queue examples; the standard hierarchy a maintenance exam expects you to define, in increasing strength, is:

- **Statement (line) coverage** — *what it requires*: every executable statement is run by at least one test. *What it's for / catches*: it finds completely *untested code* — lines no test ever reaches, e.g. an error handler that is never triggered. *Why it's weak*: it is the *weakest* criterion because reaching a line is not the same as testing its decision; a test that enters an `if` only when the condition is true executes the body's statements but never checks what happens when the condition is false. (Grounded in the deck's emphasis that you "only write production code to make a failing test pass," which yields high statement coverage as a by-product, Software Testing p.49; coverage-criterion definitions expanded from the standard canon [Raj13] .)
- **Branch (decision) coverage** — *what it requires*: every branch of every decision is taken at least once — *both* the true and the false outcome of each `if` / loop condition. *What it's for / catches*: it catches faults hiding on the *untaken* side of a decision (a missing `else`, a wrong default), which statement coverage misses. *Stronger than statement coverage* because forcing both outcomes implies running the statements on each side. *Worked*: in the Queue, branch coverage forces a test where `enqueue` returns `false` (queue full) *and* one where it returns `true` (Software Testing p.11) — exactly the `true, true, false ...` expected tuple (p.11), so the deck's boundary run is also a branch-coverage demonstration.
- **Path coverage** — *what it requires*: every *combination* of branches (every independent end-to-end execution path through the method) is exercised. *What it's for / catches*: it catches faults that only appear from a *specific sequence* of decisions — e.g. branch A-true then B-false interacting badly — which branch coverage (each decision independently) can miss. *Why it's strongest yet usually infeasible*: the number of paths multiplies with each decision and *explodes* for loops (a loop can run 0, 1, 2, ... n times, each a distinct path), which is precisely the practical face of the p.5 impossibility result; in practice you approximate it with a set of representative paths.

The white-box `testEnqueueTail()` (Software Testing p.13) is a path-targeted test: it deliberately drives the `tail == max → tail = 0` path to assert `getTail() == 0`. **Exam framing**: the relationship is **statement $\subset$ branch $\subset$ path** in strength (full path coverage implies full branch coverage implies full statement coverage, not the reverse); higher coverage costs more tests; and crucially, even **100% coverage still does not prove correctness** — you can run every path with inputs that happen to pass while a different input on the same path fails (back to p.6).

Diagnosing a failing test: where is the fault?

What it is. A *triage discipline* — a decision tree — for locating the true source of a red test before you touch anything. The key insight is that a failing test does **not** automatically mean "the code is wrong"; the fault could be anywhere in the chain that produced the verdict. The deck's decision tree (Software Testing p.9) walks four candidate fault locations in order:

1. **Bug in the SUT?** — the software under test is wrong. (Most common; this is the one you hope for because it's yours to fix and the fix is local.)
2. **Bug in the acceptability/acceptance test?** — the test/oracle itself is wrong (wrong expected value, wrong setup, brittle fixture). A wrong test is *worse than no test* because it either hides real bugs or wastes time chasing phantoms.
3. **Bug in the specification?** — the requirement was ambiguous or wrong, so both code and test faithfully implement a *flawed contract*. Here nothing is technically "broken" yet the result is wrong; the fix is upstream, in the requirement.
4. **Bug in the OS, compilers, libs, hardware?** — the platform. Rare but real (a compiler optimisation bug, a libc difference, a floating-point quirk across CPUs).

What it's used for / why it matters. It prevents the most damaging mistake in maintenance: "fixing" correct code because the *test* or *spec* was actually at fault, which injects a real bug while chasing a fake one. Walking the tree in order (cheap-and-likely first, exotic last) is the efficient way to spend diagnostic effort.

When & how it applies. The **Mars Climate Orbiter** is the slide's worked failure: one team used **metric (m/s)** and another **English (ft/s)** units, a *specification/interface* mismatch that no unit test caught because each side was internally consistent — each module passed its own tests, the bug lived in the *agreement between them* (Software Testing p.9). The practical takeaway: when a test goes red, *triage the fault location before "fixing" anything*, and be especially alert to integration-level mismatches that live in the spec/interface (location 3) rather than in any one piece of code.

Specifications, domain and range

What it is. A specification is the *contract* of a piece of code, written as a mapping from each input **domain** (the set of legal inputs) to the expected output **range** (the set of permitted results), independent of any implementation (Software Testing p.23). It is the formal answer to "what is this supposed to do?" and it is the raw material black-box testing is built from. The `sqrt` family on p.23 is the model:

- `sqrt: R+ -> R+` (non-negative reals → non-negative reals),
- `sqrt: R -> C` (all reals → complex, since $\sqrt{-1}$ is imaginary),
- `sqrt: F+ -> F+` (non-negative *floats* → floats),
- `sqrt: F -> F+ U exception` (floats including negatives → float **or an exception**) (Software Testing p.23).

The slide also shows concrete behaviours: input `9` → `3` or `3` (a positive root), input `-1` → `i`, `NaN`, `exception` depending on the type system (Software Testing p.23).

What it's used for / why it matters. The spec is what *tells you which tests must exist* — it defines the equivalence classes (positive, zero, negative, NaN, infinity, overflow) so you do not have to guess. Without a spec you cannot write a correct oracle (you have nothing to compare against) and you cannot judge whether a behaviour is a bug or a feature. The four `sqrt` signatures also teach a crucial maintenance lesson: the *same operation has different contracts* depending on the type domain — mathematical reals (`R`) admit complex

roots, while machine floats (`F`) must instead raise an exception or return `NaN`, because `F` cannot represent `i`.

When & how it's applied. You read the domain→range table and turn each distinct region into a test: a positive input (`9 → 3`), the zero edge, a negative input (`-1 → exception or NaN` per the type system), plus the *float-specific* boundary classes the table implies — overflow, `NaN`, `-0.0` — which behave nothing like their real-number cousins. This is exactly how black-box tests are generated "straight off" the contract, and it is why the deck pairs the spec slide with equivalence partitioning and boundary analysis: the spec hands you the partitions, those techniques hand you the representative and edge values.

Making software testable (Design for Testability)

What it is. *Design for testability* (DfT) is the set of code-design properties that make a system *easy to test* — testability is a quality you build *in*, not bolt *on*. The premise is blunt: you cannot test code that is built to resist testing (hidden state, tangled dependencies, non-determinism), so the SUT must be shaped to admit tests. The deck's "Creating Testable Software" checklist (Software Testing p.14):

- **Clean Code and Refactor** — *what/why*: readable, small, well-separated code is testable code, because a method with one clear responsibility has a small, checkable behaviour; clutter and god-classes have too many interacting states to pin down [Martin] [Fowler99].
- **"Describe what it does and how it interacts"** — *what/why*: explicit, documented interfaces give the oracle something precise to assert against and make dependencies visible so they can be substituted (Software Testing p.14).
- **No extra threads** — *what/why*: concurrency makes tests *non-deterministic* (the same test passes or fails depending on timing), which destroys the repeatability a test needs to be trustworthy (Software Testing p.14).
- **No swap of global variables** — *what/why*: hidden global state makes tests *order-dependent and non-repeatable* — test B passes only if test A ran first and left the right globals — so each test no longer means anything in isolation (Software Testing p.14).
- **No pointer soup** — *what/why*: tangled aliasing (many references mutating shared objects) defeats *isolation*, because you can no longer reason about, or hold still, the part you are testing (Software Testing p.14).
- **Module unit tests** — *what/why*: design modules with clear seams so each can be unit-tested *alone*, without standing up the whole system (Software Testing p.14).
- **Support fault injection** — *what/why*: build seams where you can deliberately inject failures, so you can reach the *unhappy paths* (I/O errors, timeouts) that never trigger on their own (see Fault Injection) (Software Testing p.14).
- **Assertions, Assertions, Assertions!!!** — the slide triples the word for emphasis (Software Testing p.14): *what/why*: embedded invariant checks turn the code into its own continuous tester (see Assertions).

When & how it applies. You apply DfT *while writing or refactoring*, not after — e.g. instead of a method that reads `System.currentTimeMillis()` directly (untestable, context-sensitive), you inject a `Clock` interface so a test can pin "now". This is a direct bridge to the change process: the **Prefactoring** step (clean the code *before* you change it) and **Postfactoring** (clean it *after*) exist partly to keep code testable, and a passing test suite is in turn what makes those refactorings *safe* [Raj13] [Fowler99] — a virtuous loop where testable code enables refactoring and refactoring keeps code testable.

Assertions

What it is. An **assertion** is an "Executable check for a property that must be true (invariant)" (Software Testing p.18) — a line of code embedded *inside* the production code that states a condition the program believes must always hold at that point, and aborts (in Java, throws `AssertionError`) if it ever does not. It is the program checking *itself* at runtime, distinct from a unit test which checks the program *from outside*.

What it's used for / why it matters. Assertions catch *programmer* faults — broken assumptions, impossible states — at the *exact moment and place* they first occur, instead of letting corrupted state propagate until it surfaces as a baffling failure far away. They turn "this should never happen" from a comment into an enforced, self-documenting check, which is why they make code "self-checking" and "fail early, closer to the bug" (see below). Example from the deck:

```
double sqrt(arg){
    //...compute result...
    assert result > 0;    // invariant: a real square root is positive
    return result;
}
```

(Software Testing p.18)

The three rules of assertions (Software Testing p.18) — *what each rule is and why it exists*:

- **Rule 1 — Assertions are not for error handling.** *What:* assertions check *programmer* mistakes ("this should never happen"), not *expected* runtime errors such as bad user input, missing files, or network failures. *Why:* the two have opposite recovery semantics — bad input is a normal, anticipated condition the program must *handle and continue* (an exception), whereas a violated invariant means the program's own logic is broken and it is *unsafe to continue* (an assertion). *How to tell them apart:* if a well-behaved caller could legitimately trigger it, it is an exception; if only a bug could trigger it, it is an assertion. TestLab1 echoes this exactly: "Assertions should be used to check something that should never happen" and "an assertion should stop the program from running, but an exception should let the program continue running." (TestLab1 p.1).
- **Rule 2 — No side effects.** *What:* the asserted expression must not change program state — `assert foo()==0;` where `foo()` mutates a global variable is forbidden. *Why:* because assertions can be *disabled* (they are off by default in Java), an assertion with a side effect would make the program behave *differently* with assertions on versus off — a heisenbug where turning on the safety net changes the outcome (Software Testing p.18).
- **Rule 3 — No silly assertions.** *What:* do not assert things that are tautologically or trivially true, like `assert 1+1==2;`. *Why:* such checks can never fail, so they add visual and runtime noise without buying any safety, diluting the signal value of the real assertions around them (Software Testing p.18).

Why assertions? (the benefits). They (1) "Make code self-checking, leading to effective testing" — the code validates itself on *every* run, so even tests that did not explicitly check a property still benefit when an embedded assertion trips; (2) "Make code fail early, closer to the bug" — the failure surfaces at the *source* of the bad state, not three layers downstream where it is far harder to diagnose; and (3) "Document assumptions, preconditions, postconditions and invariants" — an `assert` is *executable documentation* that, unlike a comment, cannot silently drift out of date because it is checked (Software Testing p.20). They are not academic: **GCC ships ~9,000 assertions, LLVM ~13,000 — about one assertion per 110 lines of code** (Software Testing p.20), evidence that mature, production compilers lean on them heavily.

Enabling / disabling — what it is and when to choose which. In Java assertions are **off by default** and must be enabled at runtime (the deck has dedicated "Enable Assertions" / "Disable Assertions?" slides, Software Testing p.16–17; in practice `java -ea / -enableassertions`). *Why off by default:* so shipped code pays no runtime cost unless you opt in. *The trade-off:* disabling them makes code "run faster" and "keeps running" past a violation (no `AssertionError`), but you lose the safety net — a corrupted state continues silently instead of halting. The deck pushes back on the reflexive "disable in production", arguing "even in production code, [it] may be better to fail early" (Software Testing p.17), because a clean crash near the bug is often safer than limping on with corrupt data. **When to use which:** keep assertions **on** in "running software that can be recovered by failing early" (a crash-and-restart is acceptable), and **disable** them only in a "mission critical stage when it is better to continue than recover" (Software Testing p.21) — e.g. a flight-control loop where halting is more dangerous than proceeding on possibly-bad state.

Representation invariant (`checkRep`). *What it is:* a single method that bundles all the assertions describing a class's *representation invariant* — the conditions a well-formed instance must always satisfy about its internal fields. *What it's for:* called from tests (or after every mutator), it converts an ordinary unit test into a *property check*: instead of asserting one expected value, it verifies the object is *internally consistent* after every operation, catching corruption that a value-only assertion would miss. The Queue's invariant checker is the model assertion-cluster (Software Testing p.19):

```
public void checkRep() {
    assert this.getSize() >= 0 && this.getSize() <= this.getMax();
    if (this.getTail() > this.getHead())
        assert (this.getTail() - this.getHead()) == this.getSize();
    if (this.getTail() < this.getHead())
        assert (this.getHead() - this.getTail()) == (this.getMax() - this.getSize());
    if (this.getTail() == this.getHead())
        assert (this.getSize() == 0) || (this.getSize() == this.getMax());
}
```

This encodes the FIFO ring-buffer invariant and is invoked from tests (the JUnit `setUp()` wraps the queue in a `CheckRepWrapper`, Software Testing p.52) so that *every* operation is validated against the invariant — turning a unit test into a property check. TestLab1 requires exactly this: "Use JAVA Assertions to test invariants." (TestLab1 p.1).

The fragile-test problem (test brittleness)

What it is. A **fragile test** is one that breaks for reasons *unrelated to a real defect* — the code is still correct, but an incidental change makes the test go red. *Why it matters:* fragile tests make the suite "cry wolf", so the team starts ignoring red builds and eventually distrusts the whole suite, which destroys testing's entire value (the deck devotes a section to it, Software Testing p.24). The cure is to identify *what* the test is over-coupled to and decouple it. The deck enumerates four **sensitivities** — the four ways a test can be over-coupled to the SUT (Software Testing p.25–28; these pages are titled diagrams, expanded here from the section heading and standard xUnit-patterns vocabulary):

- **Interface sensitivity** (Software Testing p.25) — *what it is:* the test is coupled to the *shape* of the API (method signatures, parameter order, names) rather than to behaviour. *Why it bites:* rename a method or reorder parameters and the test breaks even though behaviour is unchanged; in a refactor-heavy maintenance setting this generates constant false alarms. *Mitigation:* test through *stable* interfaces and avoid asserting on incidental signature details — this is the Mars-Orbiter failure class (a mismatch at the interface, not in behaviour).

- **Behavior sensitivity** (Software Testing p.26) — *what it is*: the test is coupled to *behaviour that legitimately changed*. *Nuance*: here a break can be *correct* (the behaviour really did change, and the test rightly flags it for review) — but if the test over-specifies *incidental* behaviour (exact log text, an internal call order that does not matter), the break is still noise. *Mitigation*: assert only the behaviour the spec actually promises, not implementation accidents.
- **Data sensitivity** (Software Testing p.27) — *what it is*: the test is coupled to specific external data/state — a particular live database row, a fixed file on disk, a record someone else might change. *Why it bites*: change or remove that fixture and the test breaks for reasons that have nothing to do with the code. *Mitigation*: own and control your test data — set it up fresh in the test and tear it down after (TestLab1's "Testdata" sections, e.g. the user CPR "1234651234", do exactly this — the test *creates* the data it needs, TestLab1/Software Testing p.42).
- **Context sensitivity** (Software Testing p.28) — *what it is*: the test is coupled to the *environment* — wall-clock time, time zone, locale, file system layout, network availability, or the order other tests ran in. *Why it bites*: the same test passes on your machine at noon and fails on CI at midnight, or in another locale. *Mitigation*: inject the environment (a fake clock, a fixed locale) so the test controls it. The **time-dependent** and **fault-injection** slides (p.31–32) are concrete context-sensitivity examples and their cures.

The cross-cutting cure. The deck's overarching remedy is "**Testing under the UI**" (Software Testing p.29): pull the logic out from behind the GUI and test it directly, converting a brittle **Manual Test** into a stable **Automatic Test** (Software Testing p.29) — because the UI layer is where most interface/data/context coupling lives, moving the assertions below it removes the fragility at its source.

Testing through and under the GUI

What the problem is. GUIs are intrinsically hard to test because their "inputs" are clicks, events, and swipes (not parameter values you can pass) and their "outputs" are *application states* — a window repainted, a list updated — not return values you can `assertEquals` on (Software Testing p.30). That mismatch makes GUI tests slow, context-sensitive, and brittle. The deck gives two complementary options, ordered by preference:

- **Test under the UI** — *what it is*: call the domain/application layer *directly*, bypassing the widgets entirely, so the bulk of logic is covered by fast, deterministic automatic tests (Software Testing p.29). *Why prefer it*: the business rules — where most bugs live — become ordinary unit-testable code with real return values and oracles; you sidestep the click/state problem altogether. *How*: structure the app so a button handler does nothing but call a plain method (`controller.borrowBook(...)`) that you can test in JUnit without any window open.
- **Record and play using scripts** — *what it is*: capture a real GUI session (the sequence of clicks/inputs) and replay it later as a test; the deck names this for GUI-level testing (Software Testing p.30) and lists **Sikuli Script** as a tool, which matches *screenshots* on screen to drive the UI (Software Testing p.56). *What it's for*: verifying the *thin* presentation layer end-to-end — wiring, layout, that clicking the button really invokes the handler. *Caveat / when*: it is more fragile (context- and data-sensitive — a moved widget or changed pixel breaks it) and slower, so it sits *above* the under-the-UI layer and should be used sparingly, only for the few flows that genuinely need real-UI verification.

This maps onto **Clean Architecture** [Martin]: keep business rules independent of the UI so the rules are unit-testable, and verify the thin UI layer separately and sparingly — the same logic-below, UI-thin split the deck's two options describe.

Fault injection

What it is. Fault injection is the technique of *deliberately causing a dependency to fail on demand* so you can test the error-handling code that normally never runs. **What it's for:** the *unhappy paths* — disk-full, network-drop, file-not-found, timeout — are the paths most likely to be buggy precisely *because* they almost never execute in normal runs, so they rarely get exercised by accident; fault injection lets you reach them deterministically and confirm the program degrades gracefully instead of crashing or corrupting data. The deck's example wraps the real `open()` with `my-open()` (Software Testing p.31):

```
file = open("/tmp/foo", "w")    // real
file = my-open("/tmp/foo", "w") // injected wrapper:
                                // "succeed 100 times, then fail 1% of calls"
```

(Software Testing p.31)

When & how it applies. By controlling *when* and *how often* a dependency fails ("succeed 100 times, then fail 1% of calls"), you make a rare failure *reproducible* and can assert the program retries, rolls back, or reports cleanly. Supporting fault injection is on the testability checklist (Software Testing p.14), because the code must expose a seam (like the `my-open` wrapper) for the failure to be injected. Fault injection is closely related to test doubles: in JUnit/Mockito terms, a stub configured to *throw* on the Nth call —

`when(svc.read()).thenThrow(new IOException())` — is fault injection, which is why the two topics sit next to each other in the deck.

Time-dependent problems

What it is. A *time-dependent problem* is any code whose behaviour depends on the current time — timeouts, schedulers, expiry/overdue checks, "is the book overdue?" (Software Testing p.32). **Why it's a testing hazard:** such code is *context-sensitive* (a fragile-test sensitivity) — it reads a global, ever-changing input (the wall clock) that the test cannot control, so the same test gives different results at different moments and cannot be made repeatable. **The fix — what & how:** the standard mitigation, consistent with the deck's "test under the UI / inject the dependency" theme, is to **inject the clock** — wrap time behind an interface (e.g. a `Clock` with `now()`) and pass it in, so production uses the real clock while tests substitute a controllable fake/stub that returns whatever "now" the scenario needs. **Worked scenario:** to test "a book borrowed on day 1 is overdue after 14 days," set the fake clock to day 1, borrow, advance it to day 16, and assert `isOverdue()` is true — no waiting, fully deterministic. The deck's Mockito section later demonstrates exactly this style of dependency replacement (e.g. "Overdue book", Software Testing p.68).

Automatic tests as code (JUnit by example)

What it is. This concept makes concrete that an *automatic test is just ordinary code* — a method that sets up the SUT, exercises it, and asserts an oracle — runnable by a machine with no human in the loop (Software Testing p.33). **Why that matters:** because a test is code, it can be re-run instantly and for free, which is exactly what makes regression testing (run the whole suite on every change) possible; a manual test cannot scale that way. **The pattern to learn (Arrange-Act-Assert):** assert a *precondition*, perform the *action*, assert the *postcondition*. The deck's successful and failing login illustrate it:

```
@Test
public void testLoginAdmin() {
    LibraryApp libApp = new LibraryApp();
    assertFalse(libApp.adminLoggedIn());
    boolean login = libApp.adminLogin("adminadmin");
    assertTrue(login);
}
```

```

    assertTrue(libApp.adminLoggedIn());
}

@Test
public void testWrongPassword() {
    LibraryApp libApp = new LibraryApp();
    assertFalse(libApp.adminLoggedIn());
    boolean login = libApp.adminLogin("admin");
    assertFalse(login);
    assertFalse(libApp.adminLoggedIn());
}

```

(Software Testing p.33). Note the pattern: assert a **precondition** (not logged in), perform the action, assert the **postcondition** (logged in / still not logged in). This is Arrange-Act-Assert.

Test-Driven Development (TDD)

What it is. TDD is a development *discipline* that inverts the usual order: you write the **test before the implementation** (Software Testing p.35). The test states an expectation ("Tests = expectations on software"), and only then do you write code to satisfy it. It applies to "All kind of tests: unit, component, and system tests" (Software Testing p.35), not just unit. The deck contrasts three stances: **Traditional Testing** (code first, test after — p.36), a **transition state** "Moving to TDD" (p.37), and **Real TDD** (test strictly first — p.38).

What it's used for / why it matters. Writing the test first delivers three benefits you cannot easily get by testing afterwards: (1) it *guarantees* the code is testable, because the test exists before the code and shaped its design; (2) it *designs the API from the caller's viewpoint* — you feel the awkwardness of a bad interface immediately, since you are its first user; and (3) it yields high coverage *by construction*, because no production line gets written without a failing test demanding it. It also keeps you honest about scope — you only build what a test requires.

The TDD cycle — red / green / refactor (Software Testing p.39):

1. **red** — create a *failing* test (it fails because the feature doesn't exist yet). *Purpose:* prove the test can fail, and capture the requirement precisely before writing code.
2. **green** — write *only enough* production code to make the test pass — no more. *Purpose:* satisfy exactly this requirement and nothing speculative.
3. **refactor** — clean up the code (and tests) without changing behaviour. *Purpose:* pay down the mess the "quickest pass" may have introduced, safely, because the green test now guards behaviour.

When & how it applies. Then "Repeat for functionality, bug, ..." until "no more ideas for tests" (Software Testing p.39). The **discipline rules** (Software Testing p.39) govern how you stay in the loop: "One test at a time" (never write three failing tests at once — you lose focus); "Implement only as much code so that the test does not fail" (resist gold-plating); and "If the method looks incomplete, add more failing tests that force you to implement more code." This last rule is *how TDD grows an implementation*: you cannot write code you don't have a test for, so to justify more code you must first write the failing test that demands it — the test suite literally pulls the implementation into existence, requirement by requirement. *Concrete cycle:* to add "borrow a book," you write `testBorrowBook()` (red, no method yet), add `borrowedBooks.add(book)` (green), then refactor — exactly the Borrow-Book walkthrough below.

Where do test ideas come from? (Software Testing p.40) — *because TDD needs a steady supply of failing tests, the deck catalogues their sources, each mapped to the kind of test it yields:*

- **Use-case scenarios (missing functions) → Acceptance tests** — *what/why*: a use case describes a user-visible feature, so it becomes an acceptance test that proves the feature is present and behaves as the user expects.
- **Possibility for defects (missing code) → Defect tests** — *what/why*: whenever "you want to write more code than is necessary to pass the test," that extra code is unjustified until a test demands it, so you write a *defect test* to pin down the edge case (null input, overflow) that justifies it (Software Testing p.40). This is how TDD prevents speculative, untested code.
- **Complex behaviour of classes → Unit tests** — *what/why*: intricate class logic is where subtle bugs hide, so it earns focused unit tests targeting each tricky path.
- **Code experiments** — *what/why*: "How does the system behave, if ..." — exploratory tests you write to *learn* an unfamiliar API or legacy component's actual behaviour, doubling as documentation (Software Testing p.40).
- And the meta-rule: "Make a list of new test ideas." — *why*: during implementation you constantly notice new cases; jot them down rather than chase them now, preserving the "one test at a time" discipline (Software Testing p.40).

Traditional Testing → Moving to TDD → Real TDD (the three-diagram progression, p.36–38)

What the three slides are. Between the TDD definition (p.35) and the red/green/refactor cycle (p.39), the deck devotes three consecutive diagram slides to *where the test-writing sits in time relative to the implementation*: "Traditional Testing" (Software Testing p.36), "Traditional Testing / Moving to TDD" (Software Testing p.37), and "Traditional Testing / Real TDD" (Software Testing p.38). The slides are timeline diagrams with no body text beyond their titles (flagged in the grounding note above), so the prose here expands the titles using the deck's own definition of TDD on the neighbouring slides.

- **Stage 1 — Traditional Testing (p.36)**: code first, test after. The implementation is written to completion and tests (if any) are bolted on at the end. The consequences are the ones the rest of the deck argues against: testability is accidental (the code was not shaped by a test, so it may have hidden dependencies, p.14), coverage is whatever the after-the-fact tests happen to reach, and the tests arrive too late to influence the design (contrast with the design benefit on p.49).
- **Stage 2 — Moving to TDD (p.37)**: a transition state in which tests move *earlier* — written alongside or immediately after each piece of implementation rather than at the very end. The same slide still carries the "Traditional Testing" label for contrast (Software Testing p.37), showing a team partway through adopting the discipline: feedback comes sooner, but the test does not yet *precede* and therefore does not yet *drive* the code.
- **Stage 3 — Real TDD (p.38)**: the test comes strictly **before** the implementation — "Test before the implementation," with tests acting as "expectations on software" (Software Testing p.35, visualised on p.38). Only at this stage do the full p.49 advantages materialise: coverage *by construction* (no production line exists without a failing test that demanded it) and API design from the caller's viewpoint.

Why the progression matters for the exam. It frames TDD adoption as a *spectrum*, not a binary: the distinguishing variable is *when the test is written relative to the code* (after everything → alongside → strictly before). If asked "what distinguishes real TDD from simply having tests?", the answer is stage 3's ordering constraint plus the p.39 discipline rules (one test at a time; implement only enough to pass) — having a large suite written after the fact is stage 1 with good coverage, not TDD (Software Testing p.36–39).

TDD worked example — "Borrow Book" (the deck's flagship)

What this example is for. It is the deck's end-to-end demonstration that the red/green/refactor cycle and the "test ideas" sources above are not abstract — it grows a real feature one failing test at a time, showing *how*

each rule plays out: start from a use case, write the failing main-scenario test, implement the minimum, add a failing test for the alternative path, implement again, then add defect/boundary tests. Follow it as the template for your own JHotDraw feature in the lab. The deck runs a full TDD loop on a library "borrow book" use case (Software Testing p.41–47):

1. The use case (Software Testing p.41): *Name*: borrow book. *Actor*: user. *Main scenario*: the user borrows a book. *Alternative scenario*: the user already has 10 books borrowed → the system shows an error.

2. Test for the main scenario — first define **test data** (a user CPR "1234651234", a book signature "Som001") and the **test case** steps (retrieve user, retrieve book, borrow, assert the book is now in the user's borrowed list) (Software Testing p.42). Then the failing JUnit test (Software Testing p.43):

```
@Test
public void testBorrowBook() throws Exception {
    String cprNumber = "1234651234";
    User user = libApp.userByCprNumber(cprNumber);
    assertEquals(cprNumber, user.getCprNumber());

    String signature = "Som001";
    Book book = libApp.bookBySignature(signature);
    assertEquals(signature, book.getSignature());

    List<Book> borrowedBooks = user.getBorrowedBooks();
    assertFalse(borrowedBooks.contains(book)); // precondition

    user.borrowBook(book); // act

    borrowedBooks = user.getBorrowedBooks();
    assertEquals(1, borrowedBooks.size()); // postcondition
    assertTrue(borrowedBooks.contains(book));
}
```

3. Implement just enough to go green (Software Testing p.44):

```
public void borrowBook(Book book) {
    borrowedBooks.add(book);
}
```

4. Test the alternative scenario — test data adds 10 books "book1"..."book10"; the test borrows all 10, then tries to borrow "Som001" and asserts a `TooManyBooksException` is thrown (Software Testing p.45). **5. Implement the alternative** (Software Testing p.46):

```
public void borrowBook(Book book) throws TooManyBooksException {
    if (borrowedBooks.size() >= 10) {
        throw new TooManyBooksException();
    }
    borrowedBooks.add(book);
}
```

6. More test cases (defect/boundary tests) — "What happens if book==null in borrowBook?" Retrieve the user, call `borrowBook(null)`, and check the borrowed count did not change (Software Testing p.47). This is a boundary/defect test in the p.40 sense.

The example demonstrates the whole cycle: red (write `testBorrowBook`), green (`borrowedBooks.add`), then a new red for the alternative path forcing more code, then defect tests for the edges.

Refactoring as the third TDD step

What it is. Refactoring is "the third step in TDD" (Software Testing p.48): "Restructure the system *without changing its functionality*" — you alter the code's internal design while keeping its observable behaviour byte-for-byte identical. **What it's for / why it matters:** the "green" step optimises for *passing the test fast*, which often leaves duplication and mess; refactoring is the step that pays that down, with the goal to "improve the design ... e.g. remove code duplication (**DRY principle** — Don't Repeat Yourself)" (Software Testing p.48). It keeps the codebase healthy so the *next* cycle is easy rather than fighting accumulated debt. **Two non-negotiables the deck stresses:** (1) it is a **necessary step**, not optional — skipping it lets entropy build; and (2) it **requires a good test suite**, because the passing tests are the *only* proof that your restructuring preserved behaviour — without them you are just editing and hoping (Software Testing p.48). **When & how it applies:** the deck notes "later in the course more about refactoring mechanics", linking forward to the Refactoring / Prefactoring / Postfactoring lectures [Fowler99] [Raj13]. In change-process terms the relationship is exact: *green tests are the licence to refactor*; without them, the **Prefactoring** (clean before changing) and **Postfactoring** (clean after) steps of the change mini-cycle are unsafe, because you could silently break behaviour and never know.

TDD advantages

What this concept is. The deck's summary of *why* you would adopt the red/green/refactor discipline, grouped into two benefit families (Software Testing p.49):

- **Test benefits** — *what & why*: "Good code coverage: Only write production code to make a failing test pass," so high coverage is achieved *by construction*, not as an afterthought — every line exists because some test demanded it, which means almost every line is exercised by at least one test (Software Testing p.49). The practical payoff is a strong regression suite that comes free with the implementation.
- **Design benefits** — *what & why*: "Helps design the system: defines usage of the system before the system is implemented" — because you write the *call* before the *implementation*, you experience the API as its first client and naturally shape a clean, usable interface; and you get a **testable system** as a by-product, since code written to satisfy a test is, by definition, testable (Software Testing p.49). *When this pays off*: most in maintenance, where a well-tested, well-designed module is the one you can change later without fear.

JUnit (the xUnit framework)

What it is. "JUnit is a framework for writing tests" in Java — it provides the scaffolding (a test runner, lifecycle hooks, and an assertion library) so you write *just* the test logic and the framework handles discovering, running, and reporting tests. It was written by **Erich Gamma** (a *Design Patterns* [GHJV94] author) and **Kent Beck** (eXtreme Programming / TDD), supports "Unit-, component-, and acceptance tests," and is the Java member of the **xUnit** family (<http://www.junit.org>) (Software Testing p.51). **Why it matters:** it standardises and automates testing so a whole suite runs with one command and one green/red verdict — the precondition for regression testing and CI. (The same authorship as JHotDraw and *Design Patterns* is why the course's case study is so mock-friendly: interface-based, pattern-driven code.)

Lifecycle annotations (JUnit 4; Software Testing p.52) — *what each is, and crucially how often it runs*:

- `@BeforeClass` / `@AfterClass` — *what*: `static` methods that run **once** before/after *all* tests in the class. *What for / when*: expensive shared setup you do *not* want to repeat per test — open a database connection, start a server, load a big file — done once and reused.
- `@Before` / `@After` — *what*: run before/after **each** test method, giving every test a *fresh* fixture. *What for / why*: test isolation — each test starts from a clean, identical state so tests cannot contaminate each other

(the antidote to the order-dependence/global-state fragility from p.14). On p.52 `setUp()` builds `q = new CheckRepWrapper(new Queue(2));` so every test starts with a clean, invariant-checked queue, and `tearDown()` empties it.

- `@Test` — *what*: marks a method as a test the runner should execute (and report pass/fail for).

Exam-critical distinction: `@BeforeClass` runs **once per class**, `@Before` runs **once per test method** — using the wrong one is a common source of either slow suites (heavy setup repeated needlessly) or leaked state (heavy setup shared when it should be fresh).

```
public class EnqueueTest {
    private IQueue q;
    @BeforeClass public static void setUpClass() {}
    @AfterClass public static void tearDownClass() {}
    @Before public void setUp() { q = new CheckRepWrapper(new Queue(2)); }
    @After public void tearDown() { q.empty(); }

    @Test public void testEnqueue() {
        boolean res = q.empty(); assertTrue(res);
        res = q.enqueue(1); assertTrue(res);
        res = q.enqueue(2); assertTrue(res);
        q.enqueue(3);
        assertTrue(q.getTail() == 0); // wrap-around boundary
    }
}
```

(Software Testing p.52)

JUnit assertions (`import static org.junit.Assert.*;`) (Software Testing p.53) — *what they are*: the framework's oracle library, methods that compare actual to expected and fail the test (with a message) if they disagree.

- General: `assertTrue(bexp)`, `assertTrue(msg, bexp)` — the catch-all; asserts any boolean condition.
- Specialised for readability: `assertFalse(bexp)`, `fail()` (force a failure, e.g. on an unreachable line), `assertEquals(exp, act)`, `assertNull(obj)`, `assertNotNull(obj)`, ... (Software Testing p.53). *Why prefer the specialised forms*: their failure messages are clearer — `assertEquals` reports "expected 3 but was 4", whereas `assertTrue(x==y)` can only say "expected true but was false", telling you nothing about *which* values disagreed. Choosing the most specific assertion makes a red test self-explanatory.

Test suites — *what they are*: a way to group several test classes so they run together as one unit, with `@RunWith(Suite.class)` and `@Suite.SuiteClasses({...})` (Software Testing p.54). *What for*: run all the tests for a subsystem (or the whole regression suite) with a single command and a single combined verdict, and share any common `@BeforeClass` setup across them.

```
@RunWith(Suite.class)
@Suite.SuiteClasses({EnqueueTest.class, DequeueTest.class})
public class QueueTestSuite { /* optional @BeforeClass etc. */ }
```

Testing exceptions — *what it is*: verifying that code *throws* the right exception when it should (a postcondition that an error path was taken). The deck shows two idioms (Software Testing p.55):

```
// (a) try/fail/catch - lets you inspect the exception
@Test public void testMThrowsException() {
    try { m(); fail(); } // fail() if no exception thrown
    catch (MyException e) { /* assert e's values */ }
}
```

```
// (b) annotation form - concise
@Test(expected = MyException.class)
public void testMThrowsException() { ... }
```

When to use which: use (a) the **try/fail/catch** idiom when you must *inspect the exception's contents* (its message, an error code, a wrapped cause) — the `catch` block gives you the object to assert on, and the bare `fail()` after `m()` guarantees the test fails if *no* exception was thrown (otherwise a non-throwing bug would pass silently). Use (b) the **annotation form** when *merely the type* matters and you do not need the exception object — it is more concise but cannot assert on contents.

Related xUnit tools the deck lists (Software Testing p.56) — *the ecosystem and what each is for*: **xUnit** (the framework *family* itself), **CppUnit** (the C++ port — same ideas, different language), **JUnit** (Java), **Mockito** (test-double / mocking framework, used *with* JUnit), **Sikuli Script** (image-based GUI testing — matches on-screen pictures to drive the UI), **DbUnit** (manages database fixtures — puts the DB into a known state before each test and restores it after, taming data sensitivity).

Test doubles — mocks, stubs, spies

What a test double is. A *test double* is a stand-in object that replaces a real dependency during a test — the term is borrowed from film "stunt doubles". **Why you need them:** when a unit's execution would *leave* the method under test (a call to a database, clock, network, or another class), that dependency makes the test slow, non-deterministic, and no longer isolated to *your* code. Replacing it with a double keeps the test fast, repeatable, and focused on the SUT alone (TestLab1 p.1). **The core distinction to master** is *state verification* (check the returned value / final state — use a stub) versus *behaviour verification* (check that a particular interaction happened — use a mock). The deck (via Mockito) defines three kinds:

- **Stub** — *what it is*: "objects holding **predefined data** to provide responses during tests. They resemble real objects with minimal methods, used when real data responses are undesirable. Stubs are considered the **lightest and most static** test doubles." (Software Testing p.60) — a stub simply answers queries with canned values and you do *not* check how it was called. *What it's for*: feeding the SUT controlled inputs (state verification) when the real source is expensive or non-deterministic. *When & how*: use a stub for a *pure query* — e.g. `when(clock.now()).thenReturn(day16)` so the overdue check sees a fixed date; you assert on the SUT's *result*, not on the clock.
- **Mock** — *what it is*: "objects that **store method calls** ... dynamic wrappers for dependencies ... used to **record and verify the interaction** between Java classes. A mock is ... the **most powerful and flexible** test double. We use ... `mock()` ." (Software Testing p.59) — a mock both returns canned values *and* records every call so you can later assert specific calls happened (behaviour verification). *What it's for*: testing that the SUT *collaborates correctly* with a dependency when the *interaction itself* is the thing under test and there is no return value to check. *When & how*: use a mock for a *command with side effects* — e.g. verify the SUT actually called `emailService.send(receipt)` exactly once: `verify(emailService).send(receipt)` . Because it is the heaviest double, reserve it for cases that genuinely need interaction checking, or tests become over-specified and fragile.
- **Spy** — *what it is*: "**partially mock objects** ... a partial object / half dummy of the real object by stubbing or spying the real ones. The real object remains unchanged, and we just spy some specific methods of it." (Software Testing p.72). A spy wraps a *real* object: "Every call, unless specified otherwise, is delegated to the [real] object." (Software Testing p.73) — i.e. real behaviour by default, overridden or recorded only where you say. *What it's for*: situations where you want *mostly real* behaviour but need to stub or verify one or two methods. *When & how*: typically on legacy code you cannot fully mock — let the real object run, but stub the one expensive method (`doReturn(...).when(spy).slowCall()`) or verify one call.

Choosing between them (exam-critical): stub when you only need *canned input* and verify *state*; mock when you need to *verify the interaction*; spy when you need *real behaviour plus* a targeted override/verification. Picking the heaviest double when a lighter one suffices over-specifies the test and makes it brittle.

(For exam completeness, the broader test-double taxonomy [Raj13] /xUnit-patterns also includes **fakes** — working but simplified implementations with real logic, e.g. an in-memory database that actually stores and queries — and **dummies** — placeholder objects passed only to fill a parameter slot and never actually used; the deck names mock/stub/spy explicitly.)

Mockito (the mocking framework)

What it is. "Mockito is a popular mock framework ... used in conjunction with JUnit ... [it] allows you to create and configure mock objects." (Software Testing p.58) — it is the Java library that *implements* the test-double concepts above, generating stand-in objects at runtime and giving you a readable API to program their responses and check their calls. **What it's for / why it matters:** it removes the boilerplate of hand-writing fake classes — one `mock(Foo.class)` call produces a configurable double for any interface/class — which is what makes isolated unit testing practical at scale. **When & how it applies:** any time the SUT has a collaborator you must replace (database, service, clock), you stand up a Mockito double, stub the queries it should answer, run the SUT, and verify the commands it should have issued. Mechanics:

- **Creating mocks** — *what & how:* either the static `Mockito.mock(MyClass.class)` method (returns a double you assign yourself), or the `@Mock` annotation on a field (more declarative). *Critical gotcha:* with annotations you "must initialize mock objects with a `MockitoAnnotations.initMocks(this)` method call" (Software Testing p.65) — without that call the annotated fields stay `null` and you get a `NullPointerException`, a classic first-time error.
- **Stubbing return values (fluent API)** — *what it's for:* tells a mock what to *return* for a given call, i.e. it makes the mock behave like a stub for queries. `when(...).thenReturn(...)` specifies a condition and its canned return value; `doReturn(object).when(...)` works similarly (and is needed for spies/void cases) (Software Testing p.69). *When to read which:* `when(x).thenReturn(y)` reads "when this method is called, return y." Example (Software Testing p.70): `java MyClass test = Mockito.mock(MyClass.class); when(test.getUniqueId()).thenReturn(43); // stubbed behaviour`
- **Verifying interactions** — *what it's for:* the *behaviour-verification* half of mocking — `verify()` asserts that a method *was actually called* on the mock (Software Testing p.69), which is how you test commands that have no return value to check. You can match arguments and call counts (Software Testing p.71): `java Mockito.verify(test).testing(Matchers.eq(12)); // called once with 12 Mockito.verify(test, Mockito.times(2)); // called exactly twice` *Why argument matchers and counts matter:* `eq(12)` asserts not just *that* `testing` was called but *with the right value*, and `times(2)` catches "called too many/few times" bugs (e.g. a duplicate send). `verifyNoMoreInteractions()` tightens this further, asserting that *no other* method was called on the mock — useful to prove the SUT did *exactly* what it should and nothing extra (Software Testing p.74).
- **@Spy and @InjectMocks** — *what they are:* `@Spy` wraps a *real* object so it behaves normally except where you override it (Software Testing p.73); `@InjectMocks` "tries to do **constructor dependency injection** based on the type," i.e. it *automatically wires* your `@Mock` / `@Spy` fields into the object under test by matching types (Software Testing p.74). *What @InjectMocks is for:* it spares you from manually passing each double into the SUT's constructor — Mockito finds the matching fields and injects them, so the object under test is built with all its dependencies already faked. Full example (Software Testing p.75): ``java public class ArticleManagerTest { @Mock private ArticleCalculator calculator; @Mock private ArticleDatabase database; @Spy private UserProvider userProvider = new ConsumerUserProvider(); @InjectMocks private ArticleManager manager = new ArticleManager();


```
@Test public void shouldDoSomething() { MockitoAnnotations.initMocks(this);
verify(database).addListener(any(ArticleListener.class)); } }
```

`` - **Mockito limitations** - **what they are & why:** it "can not test the following constructs: **Final classes, Anonymous classes, Primitive types**." (Software Testing p.67). The reason is mechanical: Mockito creates doubles by **subclassing** the target at runtime, but a final class cannot be subclassed, an anonymous class has no name to target, and a primitive (int, boolean) is not an object to wrap. **Design implication / when it bites:** if you need to mock a collaborator, **program to interfaces** and avoid marking classes final` on the seams you test through — this is a direct reason the interface-driven design of JHotDraw/GoF code is so testable.

The deck also includes a worked **"Overdue book"** mock example (Software Testing p.68) and a problem→DataServer→Mock→How sequence (Software Testing p.61–66) — *what it teaches*: it walks you from the *problem* (a unit test would otherwise hit a real DataServer) to the *solution* (stand up a Mockito double of that server) to the *how* (stub its responses), demonstrating the general recipe of replacing an external data source so the unit test never touches a real server.

The DataServer mock walkthrough — Problem → Create DataServer → Create Mock → How? (p.61–66)

What the sequence is. The Mockito section opens with a five-slide visual narrative that motivates mocking before showing any API: **"Problem"** (Software Testing p.61), **"Create DataServer"** (Software Testing p.62), **"Create Mock"** — with the stand-in explicitly labelled *Mock* in the diagram (Software Testing p.63), **"How?"** (Software Testing p.64), and then the two **"Using Mockito"** slides, one with the bullet-point mechanics (Software Testing p.65) and one showing the code (Software Testing p.66). The five pages are predominantly diagrams (flagged in the grounding note), so the prose below expands the titled storyboard into its standard reading.

- **p.61 "Problem":** a unit test for some class must exercise logic that *depends on a data server*. Running the test against the real server breaks every property a unit test needs — it is slow, requires the server to be up and seeded, and the test verdict now depends on code (and infrastructure) outside the SUT. This is the dependency situation TestLab1 warns about: "When the execution of a method passes outside of that method, you have a dependency and should apply mocks/stubs to avoid the dependency" (TestLab1 p.1).
- **p.62 "Create DataServer":** the dependency is given an explicit shape — a DataServer component the SUT talks to through an interface/API seam. Making the dependency explicit (rather than buried inside the SUT) is the design-for-testability move from p.14: a visible seam is a mockable seam.
- **p.63 "Create Mock":** the diagram swaps the real DataServer for a box labelled **Mock** — the test now wires the SUT to a Mockito-generated stand-in instead of the real server, so the test exercises *only* the SUT's logic (Software Testing p.63).
- **p.64 "How?":** the transition question — how do you actually build that stand-in? — answered by the next two slides.
- **p.65–66 "Using Mockito":** the mechanics: "Mockito supports creating mock objects with the static `mock()` method. It also supports creating mock objects based on the `@Mock` annotation. If using annotations, you must initialize mock objects with an `MockitoAnnotations.initMocks(this)` method call." (Software Testing p.65), with p.66 showing the corresponding code (Software Testing p.66).

Why this storyboard is the right mental template. Every mocking situation you will meet in the lab reduces to the same four beats: (1) notice the test would leave the SUT (problem), (2) identify/extract the dependency behind a seam (create DataServer), (3) substitute a double at that seam (create mock), (4) configure it with the Mockito API (how — `mock()` / `@Mock`, then `when().thenReturn()` for queries and

`verify()` for commands, p.69–71). The "Overdue book" example (Software Testing p.68) is this same template applied to a library-domain dependency.

Spring + JUnit (dependency injection in tests)

What it is. For Spring applications, the deck shows how to let the Spring *dependency-injection container* build and wire the objects inside a JUnit test, instead of constructing them by hand (Software Testing p.76).

What it's for / why it matters: in a DI-based app the real wiring (which implementation gets injected where) is what you often need to test, and reproducing it manually in a test is error-prone; letting the container do it means the test exercises the *same* wiring as production, and you can swap in test configurations centrally. **When & how it applies:** annotate the test class so a Spring runner stands up an application context and `@Autowired` injects the configured bean, then assert on that injected object:

```
@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration(classes = {AppConfig.class})
public class MachineLearningTest {
    @Autowired @Qualifier("ml") DataModelService ml;    // injected by Spring
    @Test public void test_ml_always_return_true() {
        assertThat(ml, instanceOf(MachineLearningService.class));
        assertThat(ml.isValid(""), is(true));
    }
}
```

The point: the framework wires the real (or test) implementations via DI, and you assert on the injected bean. `assertThat(...).is(...)` / `instanceOf(...)` are Hamcrest matchers for readable assertions.

Acceptance tests

What it is. An acceptance test is a "test defined by/with the help of the **user**, based on the requirements" (Software Testing p.78) — it checks the system from the *user's* point of view against what they actually asked for, at the *feature* level rather than the code level. **What it's for / why it matters:** unit tests prove the code does what the *developer* intended; acceptance tests prove the feature does what the *user* needs — they are the contract that defines "done" and catch the gap where correct code solves the wrong problem. **The two modes (and when each is used):**

- **Traditionally** — *what:* **manual** tests, run **by the customer**, **after** delivery, based on use cases / user stories (Software Testing p.78). *Trade-off:* faithful to the user but late and expensive — defects surface only after the software ships.
- **Agile** — *what:* **automatic** tests (JUnit, **Fit**, ...), created **before** the user story is implemented (Software Testing p.78) — i.e. *acceptance-test-driven development*, the same red-first TDD discipline applied at the feature level. *Why preferred:* the acceptance criterion is captured and automated up front, so it both specifies the work and verifies it continuously, with no late surprise.

How it's applied — the template. The deck's acceptance-test template (Software Testing p.79) structures a scenario as: *Name* (Login Admin), *Actor* (Admin), *Precondition* (Admin not logged in), *Main scenario* (1. enters password, 2. system responds true), *Alternative scenarios* (1a. wrong password, 1b. system reports wrong and restarts), *Postcondition* (Admin logged in). This is the BDD/Verification bridge: the acceptance scenario *is* the specification of "done," its main/alternative scenarios map directly onto the test cases you write, and turning it into an automated test is how Verification confirms the feature meets the user's requirement [Raj13].

Manual vs. automated testing, and regression testing

What the distinction is. *Manual* tests are executed by a person clicking through the software; *automated* tests are code that runs itself. The deck's verdict is decisive (Software Testing p.81): "**Manual tests should be avoided**" because "They are expensive (time and personnel) to execute: Can't be run often." **Automated tests** are "cheap ... to execute: Can be run as soon [as] something is changed in the system," giving "immediate feedback if a code change introduced a bug → **Regression tests**." (Software Testing p.81). *Why the cost difference is the whole point:* an expensive test runs rarely (so bugs sit undetected for a long time), while a cheap test runs on *every* change (so bugs are caught the moment they are introduced).

Regression testing — what it is and why it matters. *Regression testing* is **re-running the existing automated suite after every change to confirm nothing previously working broke** (Software Testing p.81). A *regression* is a feature that used to work and now doesn't; the suite's job is to detect that immediately. This is the heartbeat of the **Verification** phase and of **CI** (Continuous Integration): every commit re-runs the suite, so any regression is flagged within minutes of the commit that caused it, while the change is still fresh in mind. **The remaining hard part & its fix:** the UI ("More difficult (but not impossible) when they include the UI"), whose solution is again "**Test under the UI**" — push logic below the widgets so most of the regression suite is fast, stable, non-UI tests (Software Testing p.81). **When & how it ties to the change process:** because Postfactoring and Actualization happen *continuously* in maintenance, only an automated regression suite makes them safe — the suite is the safety net that lets a maintainer keep changing code with confidence that old behaviour is preserved [Raj13] .

The tester's mindset

What it is. A *cultural / psychological* concept, not a technique: the closing slide frames that "testing + development are different" attitudes (Software Testing p.83). The **Developer** mindset is "I want this code to succeed"; the **Tester** mindset is "I want this code to fail." **Why the difference matters:** the two goals are in productive tension — a developer who only wants success unconsciously writes tests that confirm the happy path and avoid the cases that would expose bugs, so the suite gives false confidence. A tester actively *hunting* for failure exercises the nasty edges where defects actually hide. The ideal engineer is the **Hybrid** who can hold both stances and switch between them — build optimistically, then attack their own code adversarially (Software Testing p.83). **How to apply it:** the deck's practical advice is "Test creatively" and "Don't ignore the weird stuff" (Software Testing p.83) — when something looks slightly off (an odd value, a flicker, an unexpected log), *chase* it rather than dismiss it, because off-nominal observations are exactly where real bugs live. The deck signs off "Fun ... Profit" (Software Testing p.83).

JHotDraw Connection

JHotDraw is the course's running maintenance subject; Lecture 7 is the **Verification** step you perform *after* changing it. Concretely, when you implement a feature change in JHotDraw via the mini-cycle:

- After **Concept Location** (find where the change goes) and **Impact Analysis** (find what else is affected) [Raj13] , the modules touched are exactly the modules whose **unit tests** you must (re)write and run — the testing levels of p.8 map onto the impact set.
- JHotDraw is a **Swing GUI** drawing editor. The deck's GUI advice applies directly: **test under the UI** (Software Testing p.29) — verify the *domain logic* of figures/handles/commands by calling the model directly with JUnit rather than driving Swing widgets, because GUI inputs are "clicks, events, swipes" producing "App. states," which are brittle to test (Software Testing p.30). TestLab1 says exactly this for

your feature: "Create JUnit 4 tests for most important **domain logic** methods of your feature. Note: **Swing and JUnit extensions often work best with JUnit 4.**" (TestLab1 p.1).

- JHotDraw is built on **Design Patterns** (it was co-authored by Erich Gamma, also the JUnit author, p.51), so its collaborators are interface-based — ideal for **mocks/stubs** when a figure depends on a drawing context, a storage layer, or an undo manager (Software Testing p.57–75). Where a JHotDraw class is `final` or anonymous, recall Mockito's limitations (Software Testing p.67) and program to the interface.
- **Assertions** / `checkRep` translate to JHotDraw invariants — e.g. a figure's bounds, a connection's endpoints, a drawing's z-order — encoded as `assert` invariant checks (Software Testing p.18–19) per TestLab1's "test invariants" requirement (TestLab1 p.1).
- The **portfolio deliverable** ties it together: "At class level write unit tests of important business functionality of your selected Feature. **Document how you have verified your Feature.**" (TestLab1 p.1). That documented test suite is the artifact you present for the Verification phase of your JHotDraw change.

(Note: the slides use a *library/Queue* running example, not JHotDraw figures, for the code demos; the connection above is the standard mapping of those techniques onto the JHotDraw case study and the lab, consistent with [Raj13].)

Running TestLab1 against your JHotDraw feature, step by step

A concrete execution plan that satisfies every lab requirement, with the deck page that justifies each move:

1. **Build setup.** Confirm the JHotDraw fork builds under Maven and add the JUnit 4 dependency to the POM if absent — lab step 1 verbatim: "Add maven dependency to [JUnit4] if it is not already done" (TestLab1 p.1). JUnit 4 (not 5) is deliberate: "Swing and JUnit extensions often works best with JUnit 4" (TestLab1 p.1).
2. **Pick the targets.** List the *domain-logic* methods your selected feature touches — the methods that compute, decide, or mutate model state, not the Swing listeners that merely forward events. This is "Create JUnit 4 tests for most important domain logic methods of your feature" (TestLab1 p.1) implemented through the deck's test-under-the-UI strategy (Software Testing p.29): if a handler does more than delegate, refactor so it delegates, then test the delegate.
3. **Best case first.** For each target method, write the main-scenario JUnit test in Arrange-Act-Assert form, asserting the precondition before acting, exactly like `testLoginAdmin` asserts `assertFalse(libApp.adminLoggedIn())` before logging in (Software Testing p.33) — lab step 3 (TestLab1 p.1).
4. **Boundaries second.** Identify each method's equivalence classes and write tests at the class edges — empty selection, single figure, maximum/zero sizes, null arguments — following the Queue's full/empty/wrap model (Software Testing p.11) and the `book==null` defect-test template (Software Testing p.47) — lab step 4 (TestLab1 p.1).
5. **Isolate every dependency.** Apply the lab's single-code-path rule: "When the execution of a method passes outside of that method, you have a dependency and should apply mocks/stubs" (TestLab1 p.1). For JHotDraw that typically means doubling the drawing/view context, storage, or undo machinery through their interfaces with Mockito — `mock()` / `@Mock` + `when().thenReturn()` for queries, `verify()` for commands (Software Testing p.65, p.69–71) — and remembering the final/anonymous/primitive limits (Software Testing p.67).
6. **Encode invariants.** Write the feature's `checkRep`-style invariants (figure bounds sane, z-order consistent, endpoints connected) as Java `assert` statements per lab step 5, observing the never-happen rule and the stop-vs-continue contrast (TestLab1 p.1; Software Testing p.18–19), and run the suite with assertions enabled (Software Testing p.16).

7. **Document for the portfolio.** Record which test covers which scenario (main, alternatives, boundaries) and how the suite was run — "At class level write unit tests of important business functionality of your selected Feature. Document how you have verified your Feature." (TestLab1 p.1). This document is your Verification-phase evidence in the change mini-cycle [Raj13].

Worked Example / Process Walkthrough

Goal: verify a feature change end-to-end using TDD + JUnit + a mock, exactly as the deck's "Borrow Book" example does (Software Testing p.41–47), framed as the Verification phase of one change mini-cycle.

Step 0 — Initiation/Impact (recap). A change request: "a user may borrow a book, but no more than 10." Impact analysis says the change lives in `User.borrowBook(...)` and its callers [Raj13].

Step 1 — RED: write the failing main-scenario test first (Software Testing p.42–43). Define test data (CPR "1234651234", signature "Som001"), then write `testBorrowBook()` asserting precondition (book not yet borrowed), act (`user.borrowBook(book)`), postcondition (list size 1, contains book). It fails because `borrowBook` doesn't exist.

Step 2 — GREEN: minimal implementation (Software Testing p.44): `public void borrowBook(Book book){ borrowedBooks.add(book); }`. Test passes.

Step 3 — RED again for the alternative path (Software Testing p.45): the "10 books already" scenario. Test data adds "book1"..."book10"; borrow all ten, then expect `TooManyBooksException` on the eleventh. Fails.

Step 4 — GREEN (Software Testing p.46): add the guard `if (borrowedBooks.size() >= 10) throw new TooManyBooksException();`. Passes.

Step 5 — Defect/boundary test (Software Testing p.47): "What if `book==null`?" Assert the borrowed count is unchanged. Add a null-guard to make it pass — a boundary case per TestLab1's "boundary cases" requirement (TestLab1 p.1).

Step 6 — Isolate a dependency with a mock. Suppose `borrowBook` must check an external `OverdueService` (cf. the deck's "Overdue book" mock, Software Testing p.68). Replace it with a Mockito mock so the unit test never hits the network (Software Testing p.58–70):

```
OverdueService svc = Mockito.mock(OverdueService.class);
when(svc.hasOverdue(user)).thenReturn(false);           // stubbed query
// ... user.borrowBook(book) ...
verify(svc).hasOverdue(user);                           // behaviour check
```

This keeps the test a single code-path with no real dependency (TestLab1 p.1).

Step 7 — REFACTOR (Software Testing p.48): with green tests as a safety net, remove duplication (DRY) and tidy `borrowBook` — behaviour preserved because the suite still passes. This is the TDD third step and the change-process **Postfactoring** step [Fowler99] [Raj13].

Step 8 — Add assertions / checkRep (Software Testing p.18–19): encode the invariant " $0 \leq \text{borrowedBooks.size()} \leq 10$ " as an `assert`, run with `-ea`, so every test also validates the invariant (TestLab1 p.1).

Step 9 — Regression run (Software Testing p.81): run the *whole* suite (this feature + all prior features). Green everywhere = the change is **verified** and introduced no regression. Red somewhere = triage via the p.9 decision tree (SUT? test? spec? platform?).

Step 10 — Document for the portfolio (TestLab1 p.1): record which tests cover which scenarios and what coverage was achieved — the Verification evidence.

The result is the deck's whole pipeline exercised once: incompleteness-aware design (p.5–6), levels (p.8), TDD cycle (p.39), JUnit (p.51–55), mocks (p.58–71), assertions (p.18–20), and regression (p.81).

Definitions & Terminology

Each row gives **what** the term is, **what it's for / what it catches**, and **when/how** it applies.

Term	Definition	Source
Turing machine	<i>What:</i> abstract model of computation manipulating symbols on a tape that can implement any algorithm. <i>Why it's here:</i> it is the simplest model still powerful enough to express any computation, so its limits are <i>every</i> language's limits. <i>Applies when:</i> establishing that Java/C/C# (Turing-complete) inherit the halting problem and thus testing incompleteness.	p.3
Halting problem	<i>What:</i> the proven undecidability of whether an arbitrary program halts. <i>Why it matters:</i> if you cannot even decide halting, you cannot decide correctness, which makes a perfect test suite theoretically impossible. <i>Applies when:</i> justifying why exhaustive testing is unattainable in principle, not just in practice.	p.4–5
Testing incompleteness	<i>What:</i> the principle "Testing can demonstrate the presence of bugs, but not their absence" (Dijkstra). <i>Why it matters:</i> a passing suite is evidence of working features, never proof of correctness — bugs hide in untested paths. <i>Applies when:</i> setting expectations after a green run; motivates partitioning/boundary/coverage to maximise assurance on a finite budget.	p.6
SUT (Software Under Test)	<i>What:</i> the component receiving test inputs and producing test outputs. <i>Why named:</i> it pins down exactly <i>what</i> a given test exercises (a method, a module, the whole system). <i>Applies when:</i> describing any test — the SUT is the box between the inputs and the oracle.	p.7
Test oracle	<i>What:</i> the "Outputs OK?" judge that decides pass/fail by comparing actual output to expected. <i>Why it matters:</i> a test is only as trustworthy as its oracle; a wrong oracle hides or invents bugs. <i>Applies when:</i> choosing how to judge output — a hard-coded value, a reference implementation, or an invariant (<code>checkRep</code>).	p.7
Unit testing	<i>What:</i> testing the smallest testable unit (a method or class/interface) in isolation. <i>Catches:</i> logic faults inside one component, pinpointed precisely. <i>Applies when:</i> the unit under test calls out to a dependency — replace it with a mock/stub so the test stays a single code-path.	p.8; TestLab1 p.1
Integration testing	<i>What:</i> testing that already-unit-tested components work together across their interfaces. <i>Catches:</i> interaction faults — mismatched assumptions, wrong call order, format disagreements — that unit tests cannot see. <i>Applies when:</i> run after units are green, on the seams that matter (e.g. persistence ↔ domain); the Mars-Orbiter mismatch is the archetype.	p.8
System testing	<i>What:</i> end-to-end testing of the whole assembled system vs. its specification. <i>Catches:</i> faults that emerge only from the whole — config, end-to-end workflows, unmet requirements. <i>Applies when:</i> run last, against acceptance criteria (e.g. log in → search → borrow → loan appears).	p.8

Term	Definition	Source
Differential testing	<i>What:</i> compare SUT output against a reference implementation (=?) used as the oracle. <i>Solves:</i> the oracle problem when you cannot state the right answer — you only need agreement. <i>Applies when:</i> in maintenance the previous release is a free oracle; run old and new on the same workload and review any mismatch.	p.8
Stress testing	<i>What:</i> test behaviour past expected load/volume/rate. <i>Catches:</i> non-functional failures — leaks, deadlocks, throughput collapse — invisible on a single input. <i>Applies when:</i> at system level before release, e.g. 10× concurrent users or 100× file size.	p.8
Random testing	<i>What:</i> generate random inputs to explore the input space cheaply. <i>Catches:</i> inputs the author never imagined (malformed data, extreme values) that crash or break invariants. <i>Applies when:</i> fuzzing a parser/decoder, asserting only "never crashes / invariant holds" since the right answer is unknown.	p.8
White-box (glass-box) testing	<i>What:</i> tests derived from source-code structure (statements, branches, paths). <i>Catches:</i> implementation-specific faults (unreachable branch, special path) and lets you <i>measure</i> coverage. <i>Applies when:</i> you can read the code and target internal routes (e.g. the queue's wrap-around path).	p.8, p.13
Black-box testing	<i>What:</i> tests derived only from the spec/interface (domain→range), code unseen. <i>Catches:</i> deviations from the contract and <i>missing</i> functionality; survives internal rewrites. <i>Applies when:</i> designing from a spec via equivalence partitioning and boundary analysis.	p.8, p.23
Equivalence partitioning	<i>What:</i> split the input/output space into classes the program treats the same; test one representative per class. <i>Why:</i> turns an infinite domain into a finite, justified test set and avoids redundant same-class tests. <i>Applies when:</i> e.g. <code>grade(score)</code> → classes {neg},{0–59},{60–100},{> 100}, one test each.	p.12
Boundary-value analysis	<i>What:</i> test the <i>edges</i> of equivalence classes (B-1, B, B+1; empty/full/wrap). <i>Catches:</i> off-by-one, overflow, empty/full handling — the most common bug family, which cluster at edges. <i>Applies when:</i> required by the lab; e.g. at grade boundary 60 test 59/60/61; queue <code>true, true, false, 6, 7, null</code> is all boundaries.	p.11; TestLab p.1
Statement coverage	<i>What:</i> every executable statement run by at least one test (weakest criterion). <i>Catches:</i> completely untested lines (e.g. an unreached error handler). <i>Limit:</i> reaching a line ≠ testing its decision; weakest of the three.	p.49 (+ [Raj 13])
Branch (decision) coverage	<i>What:</i> every branch of every decision taken — both true and false. <i>Catches:</i> faults on the <i>untaken</i> side (missing <code>else</code> , wrong default) that statement coverage misses; stronger than statement. <i>Applies when:</i> forces both a queue-full (<code>false</code>) and not-full (<code>true</code>) enqueue test.	p.11 (+ [Raj 13])
Path coverage	<i>What:</i> every independent execution path (combination of branches) exercised — strongest. <i>Catches:</i> faults from a <i>specific sequence</i> of decisions. <i>Limit:</i> path count explodes for loops, so usually infeasible — approximate with representative paths. <i>Hierarchy:</i> statement ⊂ branch ⊂ path; 100% still ≠ proof.	p.13 (+ [Raj 13])
Specification (domain→range)	<i>What:</i> the contract mapping each input domain to its expected output range, e.g. <code>sqrt: R+ -> R+</code> . <i>Why it matters:</i> defines the equivalence classes and the oracle, so it tells you which tests must exist. <i>Applies when:</i> writing black-box tests straight off the table; note reals (<code>R</code>) vs floats (<code>F</code>) differ (NaN, overflow, exception).	p.23
Fault-location triage	<i>What:</i> a decision tree to locate a failure's true source — SUT / acceptance test / spec / platform — <i>before</i> fixing. <i>Why:</i> prevents "fixing" correct code when the test or spec was actually wrong. <i>Applies when:</i> any red test; walk the four locations cheap-and-likely first.	p.9
Mars Climate Orbiter	<i>What:</i> the canonical spec/interface unit mismatch (m/s vs ft/s) where each side passed its own tests. <i>Why cited:</i> shows a fault that lives in the <i>seam/spec</i> , not any one module, invisible to unit tests. <i>Applies when:</i> illustrating triage location 3 (spec) and integration sensitivity.	p.9

Term	Definition	Source
Design for testability	<i>What:</i> shaping code to be easy to test — clean code, no threads/global swaps/pointer soup, module tests, fault injection, assertions. <i>Why:</i> you cannot test code built to resist testing (hidden state, non-determinism). <i>Applies when:</i> while writing/refactoring, e.g. inject a <code>Clock</code> instead of reading the wall clock.	p.14
Assertion	<i>What:</i> an executable check, embedded in production code, for an invariant that must always be true. <i>Catches:</i> programmer mistakes ("can't happen") at the exact point they occur. <i>Applies when:</i> documenting/enforcing pre/post-conditions and invariants; run with <code>-ea</code> .	p.18
Assertion Rule 1	<i>What:</i> assertions are <i>not</i> for error handling — use exceptions for expected errors. <i>Why:</i> bad input is anticipated and must be <i>handled and continued</i> (exception); a broken invariant is a bug and unsafe to continue (assertion stops). <i>Applies when:</i> deciding exception vs assertion — could a valid caller trigger it? → exception.	p.18; TestLab1 p.1
Assertion Rule 2	<i>What:</i> assertions must have <i>no side effects</i> . <i>Why:</i> since assertions can be disabled, a side-effecting assert would change behaviour when toggled — a heisenbug. <i>Applies when:</i> never put state-changing calls inside <code>assert</code> .	p.18
Assertion Rule 3	<i>What:</i> no silly/trivial assertions (<code>assert 1+1==2</code>). <i>Why:</i> checks that can never fail add noise and dilute the real assertions' signal. <i>Applies when:</i> assert only things that <i>could</i> plausibly be violated by a bug.	p.18
<code>checkRep()</code>	<i>What:</i> a method bundling the assertions that define a class's representation invariant. <i>Why:</i> turns a unit test into a <i>property check</i> — verifies internal consistency after every operation, catching corruption a value-only assert misses. <i>Applies when:</i> called from <code>setUp</code> /after mutators (e.g. the queue's ring-buffer invariant).	p.19
Enable assertions	<i>What:</i> Java assertions are off by default and enabled at runtime (<code>-ea</code>). <i>Why it matters:</i> if never enabled, your <code>checkRep</code> /invariants run nowhere. <i>Applies when:</i> keep on in recoverable software (fail early); disable only in mission-critical stages where continuing beats recovering.	p.16–17
Fragile test	<i>What:</i> a test that breaks for reasons unrelated to a real defect. <i>Why it's harmful:</i> it cries wolf, eroding trust until the team ignores the suite. <i>Applies when:</i> diagnose which of the four sensitivities is the cause and decouple; cure is "test under the UI" + own your data.	p.24
Interface sensitivity	<i>What:</i> a test over-coupled to API shape/signatures. <i>Breaks when:</i> a method is renamed or parameters reordered though behaviour is unchanged — constant false alarms in refactor-heavy work. <i>Mitigation:</i> test through stable interfaces.	p.25
Behavior sensitivity	<i>What:</i> a test over-coupled to behaviour that legitimately changed. <i>Nuance:</i> a correct break (behaviour really changed) — but over-specifying incidental behaviour (log text, internal order) is still noise. <i>Mitigation:</i> assert only spec-promised behaviour.	p.26
Data sensitivity	<i>What:</i> a test over-coupled to specific external data/state (a live DB row, a fixed file). <i>Breaks when:</i> the fixture changes for unrelated reasons. <i>Mitigation:</i> own and control test data — create it in the test (e.g. CPR "1234651234") and tear it down.	p.27
Context sensitivity	<i>What:</i> a test over-coupled to environment — time, locale, FS, network, order. <i>Breaks when:</i> run at a different time/locale/CI machine. <i>Mitigation:</i> inject the environment (fake clock, fixed locale) so the test controls it.	p.28
Test under the UI	<i>What:</i> test the domain/application logic directly, bypassing the GUI widgets. <i>Why:</i> the business rules become fast, deterministic, unit-testable code, removing UI-layer fragility at its source. <i>Applies when:</i> a button handler just calls <code>controller.borrowBook(...)</code> you can test in JUnit.	p.29
Record and play	<i>What:</i> capture and replay GUI sessions (e.g. Sikuli Script, image-based). <i>Why:</i> verifies the thin presentation layer end-to-end (wiring, layout). <i>Caveat/when:</i> fragile and slow (context/data sensitive) — use sparingly, above the under-the-UI layer.	p.30, p.56

Term	Definition	Source
Fault injection	<i>What:</i> deliberately force a dependency to fail on demand (e.g. fail 1% of calls). <i>Catches:</i> untested unhappy paths — disk-full, network-drop, timeout — that rarely run naturally. <i>Applies when:</i> wrap the dependency (e.g. <code>my-open</code>) or stub a throw; a stub that throws on the Nth call is fault injection.	p.31
TDD	<i>What:</i> write the test before the implementation. <i>Why:</i> guarantees testable code, designs the API from the caller's view, yields high coverage by construction. <i>Applies when:</i> driving any feature/bugfix via red/green/refactor.	p.35
Red/Green/Refactor	<i>What:</i> the TDD cycle — failing test → minimal pass → clean up. <i>Why each step:</i> red captures the requirement, green satisfies exactly it, refactor pays down the mess safely. <i>Applies when:</i> repeat per requirement until no more test ideas.	p.39
Acceptance test	<i>What:</i> a test defined with the user, based on the requirements, at feature level. <i>Catches:</i> the gap where correct code solves the wrong problem; defines "done". <i>Applies when:</i> traditionally manual/after delivery, or agile/automated/before implementation (ATDD) using the Login-Admin-style template.	p.78–79
Defect test	<i>What:</i> a test pinning down a possible defect / preventing over-written code. <i>Why:</i> whenever you'd write more code than the current test demands, a defect test justifies it (null input, overflow). <i>Applies when:</i> e.g. "what if <code>book==null</code> ?" in <code>borrowBook</code> .	p.40
Refactoring (in TDD)	<i>What:</i> restructure the code without changing behaviour; the mandatory third TDD step. <i>Why:</i> improves design / removes duplication (DRY); needs a good test suite as the proof behaviour is preserved. <i>Applies when:</i> after green, and as Prefactoring/Postfactoring in the change cycle.	p.48
DRY principle	<i>What:</i> Don't Repeat Yourself — remove duplication during refactoring. <i>Why:</i> duplicated logic must be fixed in many places, breeding inconsistency bugs. <i>Applies when:</i> the refactor step, consolidating repeated code behind one method.	p.48
JUnit	<i>What:</i> the Java xUnit framework for unit/component/acceptance tests (Gamma & Beck). <i>Why:</i> provides runner, lifecycle hooks, and assertions so a whole suite runs with one verdict — the basis for regression/CI. <i>Applies when:</i> writing automated Java tests.	p.51
@BeforeClass / @AfterClass	<i>What:</i> static setup/teardown run once per test class. <i>Why:</i> expensive shared setup you don't want to repeat (open a DB, start a server). <i>Applies when:</i> the resource is costly and safe to reuse across all tests.	p.52
@Before / @After	<i>What:</i> setup/teardown run before/after each test method. <i>Why:</i> gives every test a fresh, isolated fixture so tests can't contaminate each other. <i>Applies when:</i> building a clean SUT per test (e.g. <code>q = new CheckRepWrapper(new Queue(2))</code>).	p.52
@Test	<i>What:</i> marks a method as a JUnit test for the runner to execute and report. <i>Why:</i> tells the framework what to run. <i>Applies when:</i> on every test method.	p.52
JUnit assertions	<i>What:</i> the oracle library — <code>assertTrue</code> , <code>assertFalse</code> , <code>assertEquals</code> , <code>assertNull</code> , <code>assertNotNull</code> , <code>fail()</code> . <i>Why specialised forms:</i> clearer failure messages ("expected 3 but was 4"). <i>Applies when:</i> pick the most specific assertion so a red test is self-explanatory.	p.53
Test suite	<i>What:</i> <code>@RunWith(Suite.class) + @Suite.SuiteClasses({...})</code> to group test classes. <i>Why:</i> run a subsystem's (or the whole) tests with one command and one combined verdict. <i>Applies when:</i> assembling the regression suite.	p.54
@Test(expected=...)	<i>What:</i> assert a method throws a given exception type (concise idiom). <i>Why:</i> verifies an error path is taken when only the <i>type</i> matters. <i>Applies when:</i> no need to inspect the exception's contents (else use <code>try/fail/catch</code>).	p.55
Test double	<i>What:</i> a stand-in for a real dependency in a test. <i>Why:</i> keeps the test isolated, fast, and deterministic when the SUT would otherwise call out (DB, clock, network). <i>Applies when:</i> execution leaves the method under test; pick stub/mock/spy by need.	p.57–60

Term	Definition	Source
Stub	<i>What:</i> a double returning predefined data; lightest/most static, no call verification. <i>Why:</i> feeds the SUT controlled input for <i>state</i> verification. <i>Applies when:</i> a pure query — e.g. <code>when(clock.now()).thenReturn(day16)</code> .	p.60
Mock	<i>What:</i> a double that records and <i>verifies</i> interactions; most powerful. <i>Why:</i> tests that the SUT <i>collaborates correctly</i> when the interaction is what matters (no return value to check). <i>Applies when:</i> a command with side effects — <code>verify(email).send(receipt)</code> .	p.59
Spy	<i>What:</i> a partial mock wrapping a <i>real</i> object; delegates to it by default. <i>Why:</i> mostly-real behaviour with a targeted override/verification. <i>Applies when:</i> legacy code you can't fully mock — let it run, stub one slow method.	p.72–73
Mockito	<i>What:</i> the Java mock framework used with JUnit; <code>mock()</code> , <code>@Mock</code> . <i>Why:</i> generates configurable doubles at runtime, removing fake-class boilerplate so isolated testing scales. <i>Applies when:</i> replacing any collaborator (DB/service/clock); init <code>@Mock</code> fields via <code>initMocks(this)</code> .	p.58, p.65
<code>when().thenReturn()</code>	<i>What:</i> stub a return value for a given call (<code>when(x).thenReturn(y)</code> = "when called, return y"). <i>Why:</i> makes a mock answer queries with canned data. <i>Applies when:</i> feeding the SUT a controlled response, e.g. <code>when(test.getUniqueId()).thenReturn(43)</code> .	p.69–70
<code>verify()</code>	<i>What:</i> assert a mock method <i>was called</i> (with <code>times(n)</code> counts and argument matchers). <i>Why:</i> the behaviour-verification half of mocking — <code>eq(12)</code> checks the right argument, <code>times(2)</code> the right count. <i>Applies when:</i> confirming the SUT issued the expected command(s).	p.69, p.71
<code>@InjectMocks</code>	<i>What:</i> constructor DI of <code>@Mock</code> / <code>@Spy</code> fields into the object under test, matched by type. <i>Why:</i> auto-wires the doubles so you don't hand-build the SUT's constructor. <i>Applies when:</i> the SUT has several injected dependencies you've mocked.	p.74–75
Mockito limitations	<i>What:</i> cannot mock final classes, anonymous classes, or primitive types. <i>Why:</i> Mockito doubles by subclassing at runtime — final can't be subclassed, anonymous has no name, primitives aren't objects. <i>Implication:</i> program to interfaces, avoid <code>final</code> on tested seams.	p.67
Regression testing	<i>What:</i> re-run the automated suite after each change to catch newly broken (previously working) behaviour. <i>Why:</i> it is the safety net of Verification/CI — flags regressions within minutes of the commit. <i>Applies when:</i> every change; only automation makes it cheap enough to run that often.	p.81
Tester mindset	<i>What:</i> developer wants success, tester wants failure; the hybrid holds both. <i>Why:</i> a success-only mindset writes confirming tests that miss bugs; adversarial testing finds edge-case faults. <i>Applies when:</i> switch to "make it fail" mode — test creatively, chase the weird stuff.	p.83

Common Pitfalls / Gotchas

- **Treating a green suite as proof of correctness.** It is not — "presence, not absence" (Software Testing p.6). Residual bugs hide in untested paths (Software Testing p.6). Always reason about *which* equivalence classes/boundaries you actually covered (Software Testing p.12).
- **Skipping fault triage.** A red test can mean a bug in the *test*, the *spec*, or the *platform*, not the SUT (Software Testing p.9). "Fixing" the code when the test is wrong injects real bugs. Mars Orbiter: the units were a *spec* defect (Software Testing p.9).
- **Forgetting Java assertions are off by default.** If you never enable them (`-ea`), your `checkRep` /invariant checks run nowhere (Software Testing p.16–17). And never put **side effects** in assertions — disabling them then changes behaviour (Rule 2, Software Testing p.18).
- **Using assertions for error handling.** Assertions are for "should never happen"; expected errors (bad input) get **exceptions**. "An assertion should stop the program; an exception should let it continue."

(TestLab1 p.1; Software Testing p.18).

- **Fragile tests.** Over-coupling to interface shape, incidental behaviour, fixed data, or environment (time/locale/order) makes the suite break on non-defects and erodes trust (Software Testing p.25–28). Cure: test under the UI and own your test data (Software Testing p.29; p.42).
- **Driving the GUI directly.** GUI inputs/outputs (clicks→states) are brittle and slow to test; move logic under the UI and unit-test it (Software Testing p.29–30). For JHotDraw/Swing, use JUnit 4 on domain logic (TestLab1 p.1).
- **Writing more production code than a test demands (TDD violation).** TDD says implement *only enough to pass*; if the method "looks incomplete," add another failing test rather than speculatively coding (Software Testing p.39–40).
- **Skipping the refactor step.** Refactoring is the *mandatory third* TDD step; skipping it lets duplication/debt accumulate. It is only safe *because* you have green tests (Software Testing p.48).
- **Mocking the wrong things / hitting Mockito limits.** You cannot mock **final** or **anonymous** classes or **primitive types** (Software Testing p.67). Program to interfaces. Also don't over-mock: a stub suffices for a pure query; reserve mocks/ `verify()` for when the *interaction* is the thing under test (Software Testing p.59–60).
- **Confusing the doubles.** Stub = canned data, no verification (Software Testing p.60); Mock = records + verifies (Software Testing p.59); Spy = real object, override/verify a few methods (Software Testing p.72–73). Picking the heaviest double (mock) when a stub would do makes tests over-specified and fragile.
- **Forgetting MockitoAnnotations.initMocks(this)** when using `@Mock` / `@Spy` / `@InjectMocks` — the annotated fields stay `null` otherwise (Software Testing p.65, p.75).
- **Leaning on manual tests.** They're expensive and can't run often, so they don't give you regression coverage (Software Testing p.81). Automate so every change is regression-checked.
- **Boundary blindness.** The Queue's `true, true, false, 6, 7, null` outcome is all boundaries (Software Testing p.11). TestLab1 *requires* boundary tests (TestLab1 p.1); best-case-only suites miss exactly where defects cluster.

Slide-code traps: typos on the slides you should not reproduce

The deck's code slides contain several literal typos. Knowing them serves two purposes: you won't be confused when re-reading the slides, and you won't copy a non-compiling form into lab code or an exam answer.

- **test.when(...)** (p.70–71). The slides write `test.when(test.getUniqueId()).thenReturn(43);` — but `when` is **not** a method on the mock object; it is a static method of the `Mockito` class. The compiling form is `Mockito.when(test.getUniqueId()).thenReturn(43);` (or static-import `when`). The deck's own bullet slide states the correct fluent API: "`when(...).thenReturn(...)` can be used to specify a condition and a return value for this condition" (Software Testing p.69–71).
- **Mokito (p.71) and MokitoAnnotations (p.75).** Both misspell **Mockito** (missing the `c`): the slides print `Mokito.verify(test, Mokito.times(2));` and `MokitoAnnotations.initMocks(this);` — the real classes are `Mockito` and `MockitoAnnotations` (Software Testing p.71, p.75; the correct spelling appears on p.65).
- **Mockito.verify(test, Mockito.times(2)); is incomplete as written (p.71).** A `times(n)` verification must name *which method* is expected `n` times — the complete form is `Mockito.verify(test, Mockito.times(2)).someMethod(...);`. The slide's comment "Was the method called twice?" tells you the intent (Software Testing p.71).
- **testEqality (p.13).** The hand-rolled black-box test's name is misspelled on the slide (should be "testEquality") — quote it as the slide spells it when citing, but don't read meaning into it (Software Testing p.13).

- **Missing { on p.46.** The alternative-scenario implementation slide reads `public void borrowBook(Book book) throws TooManyBooksException` with no opening brace before the `if` — a slide-formatting artifact, not a Java idiom (Software Testing p.46).
 - **enqueue(8) (p.11).** The FIFO illustration line `enqueue(7); enqueue(8); dequeue(); -> 7 then 8` drops the second `ue` — the method is `enqueue` throughout the actual class (Software Testing p.11).
-

Exam Focus

High-probability exam targets, with the angle examiners use:

- **State and explain Dijkstra's principle** and tie it to the halting problem / impossibility of a perfect test suite (Software Testing p.4–6). Expect "why can't testing prove correctness?"
 - **Classify testing kinds** on the two axes and *define each*: unit/integration/system levels; white-box vs black-box; differential/stress/random (Software Testing p.8). Be ready to place a given scenario in the map.
 - **Black-box design techniques:** define and *apply* equivalence partitioning and boundary-value analysis to a small spec (the Queue or `sqr t`) (Software Testing p.11–12, p.23).
 - **White-box coverage criteria:** define statement vs branch vs path coverage, order them by strength, and explain why 100% coverage still isn't proof (Software Testing p.13, p.49; p.6).
 - **Fault-location triage:** given a failing test, walk SUT → acceptance test → spec → platform (Software Testing p.9). Mars Orbiter is the stock example.
 - **Assertions:** define an assertion as an invariant check; recite the three rules; explain enable/disable trade-offs and *when* to use them; write/read a `checkRep` (Software Testing p.18–21). Contrast assertion vs exception (TestLab1 p.1).
 - **Design for testability checklist** — be able to list it (clean code, no threads, no global swaps, no pointer soup, module unit tests, fault injection, assertions) (Software Testing p.14).
 - **TDD:** define red/green/refactor, the discipline rules ("one test at a time," "only enough code"), where test ideas come from, and *why refactoring is the mandatory third step needing a test suite* (Software Testing p.39–40, p.48–49). Be ready to walk the Borrow-Book example (Software Testing p.41–47).
 - **JUnit specifics:** the lifecycle annotations and their *frequency* (`@BeforeClass` once vs `@Before` per test), the assertion family, suites, and the two exception-test idioms (Software Testing p.52–55). Know Gamma & Beck wrote it (p.51).
 - **Test doubles:** crisp definitions of mock vs stub vs spy and *when to use which* (Software Testing p.59–60, p.72–73); Mockito API (`mock`, `when().thenReturn()`, `verify(...).times(n)`, `@Mock/@Spy/@InjectMocks`) and its limitations (Software Testing p.65–75, p.67).
 - **Acceptance vs regression vs automated/manual:** define each; explain why automation enables regression and why regression is the core of Verification/CI (Software Testing p.78–81).
 - **Fragile-test problem:** name the four sensitivities and the "test under the UI" cure (Software Testing p.25–29).
 - **The tester's mindset** as a one-liner: developer wants success, tester wants failure, hybrid is best (Software Testing p.83).
 - **Process linkage (likely synthesis question):** explain that testing is the **Verification** phase of the change mini-cycle, that a green regression suite is what makes **Prefactoring/Postfactoring/Actualization** safe, and how this applies to a JHotDraw feature change (Software Testing p.48, p.81; TestLab1 p.1; [Raj13]).
-

TestLab1 — The Lab Handout in Full

The lab sheet for this lecture is a single page titled "[TestLab1] Testing" (TestLab1 p.1). Because it is the *graded, actionable* face of the lecture — its steps are exactly what the portfolio expects — this section reproduces and explains every element of it.

The handout's definition of unit testing (Introduction)

The handout opens with a precise, quotable definition: "In computer programming, **unit testing** is a software testing method by which **individual units of source code** — sets of one or more computer program modules together with associated control data, usage procedures, and operating procedures — **are tested to determine whether they are fit for use**" (TestLab1 p.1). Three follow-on sentences refine it, each worth knowing verbatim:

- **Who writes and runs them:** "Unit tests are typically **automated tests written and run by software developers** to ensure that a section of an application (known as the 'unit') meets its design and behaves as intended." (TestLab1 p.1) — developers, not a separate QA team; automated, not manual (this anticipates the deck's manual-vs-automated verdict on p.81).
- **What a "unit" is, per paradigm:** "In **procedural programming**, a unit could be an entire module, but it is more commonly an **individual function or procedure**. In **object-oriented programming**, a unit is often an **entire interface, such as a class**, but could be an **individual method**." (TestLab1 p.1). *Exam trap:* the OO unit is *often a class/interface*, not always a method — quoting "a unit is always one method" misstates the handout.
- **The bottom-up build-up strategy:** "By writing tests **first for the smallest testable units, then the compound behaviors between those**, one can build up **comprehensive tests for complex applications**." (TestLab1 p.1). This single sentence encodes the testing-level ladder of the deck's p.8 (units → integrations of units → the whole) *and* the test-first ordering of TDD (p.35): start smallest, then compose upward.

Objectives

The lab states exactly two objectives (TestLab1 p.1):

1. **Understand the importance of testing.** — the conceptual half: the deck's incompleteness results (p.5–6), the cost argument for automation (p.81), and the Verification role in the change process [Raj13].
2. **Implement unit tests.** — the practical half: the five classwork steps below, performed on *your* feature in the course's JHotDraw case study.

Classwork — the five steps, expanded

1. **"Add maven dependency to [JUnit4] if it is not already done."** (TestLab1 p.1). The course project is built with **Maven**, so JUnit 4 enters the build as a declared dependency in the POM rather than a manually-managed JAR. The bracketed [JUnit4] is the handout's reference link to the JUnit 4 distribution. *Why JUnit 4 and not 5:* see step 2's note.
2. **"Create JUnit 4 tests for most important domain logic methods of your feature. Note: Swing and JUnit extensions often works best with JUnit 4."** (TestLab1 p.1). Two instructions in one: (a) target the **domain logic** of your selected feature — not the GUI — which is the deck's "test under the UI" strategy applied to JHotDraw (Software Testing p.29); and (b) the explicit version pin: because JHotDraw is a **Swing** application and the Swing-related JUnit extensions interoperate best with JUnit 4, the lab

standardises on JUnit 4 (hence the `@Before` / `@BeforeClass` -style annotations throughout the deck, Software Testing p.52).

3. **"Write JUnit tests for best case scenario"** (TestLab1 p.1) — the happy path first: the feature's main scenario behaving as intended, mirroring the deck's main-scenario-first ordering in the Borrow Book example (Software Testing p.42–44).
4. **"Write JUnit tests for identified boundary cases"** (TestLab1 p.1) — then the edges, mirroring the deck's boundary discipline (the Queue's full/empty/wrap cases, p.11; the `book==null` defect test, p.47). The step carries the lab's single most quotable sub-rule, **4(a):** "A unit test should test a **single code-path through a single method**. When the execution of a method **passes outside of that method**, you **have a dependency** and should **apply mocks/stubs to avoid the dependency**, see [mockito.org]." (TestLab1 p.1). This is the operational definition of unit-test *isolation* and the bridge to the deck's whole Mockito section (Software Testing p.57–75); [mockito.org] is the handout's reference to the Mockito framework site, matching the deck's p.56 links slide.
5. **"Use JAVA Assertions to test invariants, see [JAVA Asserts]."** (TestLab1 p.1) — encode your feature's invariants as Java `assert` statements (`checkRep` -style, Software Testing p.19), with two sub-rules: **5(a)** "Assertions should be used to check **something that should never happen**" and **5(b)** "an assertion should **stop the program** from running, but an exception should **let the program continue** running." (TestLab1 p.1). 5(a) is the deck's Rule 1 (assertions ≠ error handling, Software Testing p.18) and 5(b) is the crispest assertion-vs-exception contrast in the course — quote it verbatim when asked to distinguish the two.

Portfolio Work

"At **class level** write unit tests of important **business functionality** of your selected Feature. **Document how you have verified your Feature.**" (TestLab1 p.1). Two deliverables: the test classes themselves (scoped to business functionality — again domain logic, not widgets), and a written verification account — which tests cover which scenarios — that becomes the Verification-phase evidence in your portfolio [Raj13].

Mapping the lab steps onto the deck

Lab element (TestLab1 p.1)	Deck grounding
Definition of unit testing; unit = class/interface or method	Kinds of testing — unit level (Software Testing p.8)
Maven dependency on JUnit 4	JUnit intro & ecosystem (Software Testing p.51, p.56)
Tests for domain logic, not Swing GUI	Testing under the UI; Testing a GUI (Software Testing p.29–30)
Best-case scenario tests	Main-scenario test in Borrow Book (Software Testing p.42–44)
Boundary-case tests	Queue boundary run <code>true, true, false, 6, 7, null</code> (Software Testing p.11); <code>book==null</code> defect test (p.47)
Single code-path / mocks & stubs for dependencies	Mock Objects section (Software Testing p.57–75); fragile-test isolation (p.24–28)
Java assertions for invariants; never-happen rule; stop-vs-continue	Assertions and <code>checkRep</code> (Software Testing p.18–21)
Document verification (portfolio)	Acceptance/regression as Verification evidence (Software Testing p.78–81) [Raj13]

Tooling named by the handout

The handout's complete tool list is short and exact: **Maven** (dependency management), **JUnit 4** (the test framework, with the Swing-compatibility note), **Mockito** via the [mockito.org] reference (mocks/stubs), and **Java Assertions** via the [JAVA Asserts] reference (invariants) (TestLab1 p.1). The feature under test is the one you selected in the course's running **JHotDraw** project (the "your feature" / "your selected Feature" wording, TestLab1 p.1). No other test libraries are referenced by this lab sheet.

Slide-by-Slide Walkthrough (Software Testing deck, p.1–83)

This section walks the deck in page order so every slide is individually represented, with verbatim wording where the slide carries text and an explicit flag where it is a title/diagram-only page (per the grounding note at the top of this guide). Use it to reconstruct the lecture's narrative arc and to locate any quote precisely.

p.1–2 — Title and Introduction

p.1 is the title slide: "Software Testing — **How to make software fail!**" (Software Testing p.1). The subtitle is the thesis of the whole deck: testing's job is to *provoke* failure under controlled conditions, which is restated at the very end as the tester's mindset ("I want this code to fail," p.83). **p.2** is the section divider "Introduction" with no further text (Software Testing p.2).

p.3–6 — Turing, the halting problem, and the incompleteness of testing

p.3 defines the Turing machine verbatim: "A Turing machine is a simple mathematical model of computation. It manipulates symbols on a tape based on rules, yet can implement any computer algorithm." (Software Testing p.3). **p.4** is titled "Turing's halting problem" and carries the language list "JAVA, C#, C++, C" (Software Testing p.4) — visually tying the four mainstream languages to the Turing machine's limits: each is Turing-complete, so each inherits the undecidability of halting. **p.5** draws the testing consequence in three bullets: there is a "**Theoretical reason for testing incompleteness**"; "It is **theoretically impossible to create a perfect test suite**"; and "Programmers have been trying to do the best under the circumstances" — with two sub-points: "techniques of the testing **cannot guarantee a complete correctness** of software" and "**well designed tests come close to be adequate**" (Software Testing p.5). **p.6**, titled "Incompleteness of Testing," states Dijkstra's principle verbatim — "**Testing can demonstrate the presence of bugs, but not their absence.**" — and the residual-bug corollary: "Residual bugs can still hide in the code, undetected by tests, as no test suite guarantees an error-free program." (Software Testing p.6).

p.7–9 — The test model, the kinds-of-testing map, and failure triage

p.7 is the basic test-model diagram: **Test inputs** → **Software Under Test** → **Test outputs** → "**Outputs OK?**" (Software Testing p.7) — the four boxes every later technique plugs into (the oracle being the "Outputs OK?" judge). **p.8**, "Kinds of Testing," is the one-slide taxonomy: the SUT in the centre; the level ladder **unit testing / integration testing / system testing**; the knowledge axis **white box / black box**; **differential testing** marked with "**= ?**" (compare two implementations' outputs); **stress testing**; and **random testing** marked with "**?? / ?**" symbols (unknown, generated inputs) (Software Testing p.8). **p.9**, "What is going on?", is the triage decision tree: from "test output — OK?" through the chain of *no* branches — "**Bug in SUT?**" → "**Bug in acceptability test?**" → "**Bug in specification?**" → "**Bug in OS, compilers, libs, hardware?**" — each with a *yes* exit (Software Testing p.9). The slide pins the specification branch with the case study: "**Mars climate orbiter — Metric: m/s; English: ft/s**" (Software Testing p.9).

p.10–13 — Testing Example: the Fixed Size Queue

p.10 is the divider "Testing Example" (Software Testing p.10). **p.11** carries the complete `Queue` class (constructor with `assert sizeMax > 0`, `empty`, `full`, `enqueue` with wrap-around, `dequeue` returning boxed `Integer`), the FIFO contract line `enqueue(7); enqueue(8); dequeue(); -> 7 then 8`, the two state diagrams (`size=0` and `size=1` with `head / tail` markers), the six-call run `q=Queue(2); r1..r6`, and the expected tuple `true, true, false, 6, 7, null` (Software Testing p.11) — dissected line-by-line in the Key Concepts section above. **p.12**, "Equivalent Tests," shows three test snippets over an **Input space / Output space** diagram: the plain `q.enqueue(7); x=q.dequeue(); if(x==7) success`; the same with `sleep(100)` inserted (equivalent — time cannot matter to the contract); and the generalised `q.enqueue(?); x=q.dequeue(); if(x==?) success` with a question mark inviting you to ask *which* values are interchangeable; plus the output-space prompt "**Large Integers?**" (Software Testing p.12). **p.13**, "Testing the Queue," shows the two hand-rolled static test drivers `testEquality` (black-box FIFO contract via `IQueue`) and `testEnqueueTail` (white-box wrap-around via `getTail()`), both with the manual `out.println("test1/test2 NOT OK")` -and-return-false protocol (Software Testing p.13) — reproduced in full in Key Concepts above.

p.14 — Creating Testable Software

A single checklist slide around the SUT box, with eight bullets verbatim: "**Clean Code**", "**Refactor**", "**Describe what it does and how it interacts**", "**No extra Threads**", "**No swap of global variables**", "**No pointer soup**", "**Module unit tests**", "**Support fault injection**", and the triple-emphasised "**Assertions, Assertions Assertions !!!**" (Software Testing p.14). This is an enumeration the exam can ask for *in full* — all eight items, not a sample.

p.15–21 — Assertions

p.15 is the divider "Assertions" (Software Testing p.15). **p.16**, "Enable Assertions," is a visual slide on switching assertions on (diagram-only; in practice the JVM flag `-ea`) (Software Testing p.16). **p.17**, "Disable Assertions?", weighs the toggle: *Advantages* — "Code runs faster", "Code keeps running"; *Disadvantages* — "Code can rely on a side-effect assertion?" and "Even in production code, may be better to fail early" (Software Testing p.17). **p.18** defines the concept — "Assertion: **Executable check for a property that must be true (invariant)**" — shows the `sqr` example (`assert result > 0; before return`), and states the three rules with their counter-examples: "Rule1: Assertions are **not for error handling**"; "Rule2: **NO SIDE EFFECTS**" with `assert foo()==0; // where foo() changes a global variable`; "Rule3: **No silly assertions**" with `assert 1+1==2`; (Software Testing p.18). **p.19**, "Queue Assertions," is the full `checkRep()` ring-buffer invariant (`size within [0,max]; tail>head ⇒ tail-head==size; tail<head ⇒ head-tail==max-size; tail==head ⇒ size is 0 or max`) (Software Testing p.19). **p.20**, "Why Assertions?", lists the three benefits — "Make code **self-checking**, leading to effective testing"; "Make code **fail early, closer to the bug**"; "**Document** assumptions, preconditions, postconditions and invariants" — and the production statistics under "In production?": "GCC: ~9000 assertions; LLVM: ~13,000 assertions; **One assertion per 110 L.O.C**" (Software Testing p.20). **p.21**, "When to use Assertions?", gives the two-sided rule: use in "Running Software that can be **recovered by failing early**"; "**Disable in mission critical stage** when it is better to continue then recover" (Software Testing p.21).

p.22–23 — Software Under Test and Specifications

p.22 is the divider "Software Under Test" (Software Testing p.22). **p.23**, "Specifications," diagrams the SUT behind its **API** with a **domain** → **range** mapping and the `sqr` signature family: `sqr: R+ -> R+`, `sqr: R -> C`, `sqr: F+ -> F+`, `sqr: F -> F+ U exception`; the concrete examples on the slide are input `9` → `-3` or `3` (both square roots — note the slide includes the *negative* root) and input `-1` → `i, NaN, exception`

(the result depends on whether the domain is mathematical reals, complex numbers, or machine floats) (Software Testing p.23).

p.24–32 — The Fragile Test Problem and its escapes

p.24 is the section divider "The Fragile Test Problem" (Software Testing p.24). **p.25–28** are four title slides naming the sensitivities in order: "**Interface Sensitivity**" (p.25), "**Behavior Sensitivity**" (p.26), "**Data Sensitivity**" (p.27), "**Context Sensitivity**" (p.28) (Software Testing p.25–28; the bodies are diagrams, expanded into prose in Key Concepts above). **p.29**, "Testing under the UI," contrasts a **Manual Test** (driving the GUI) with an **Automatic Test** (calling beneath it) (Software Testing p.29). **p.30**, "Testing a GUI," diagrams why GUIs resist testing: the domain side is "GUI: **clicks, events, swipes...**" and the range side is "GUI -> **App. states**" — gestures in, application states out, neither being plain values — and names the GUI-level technique "**Record and Play using scripts**" (Software Testing p.30). **p.31**, "Fault Injection," shows the wrapper trick: replace `file=open("/tmp/foo", 'w')` with `file=my-open("/tmp/foo", 'w')` where `my-open` is programmed to "**succeed 100 times then fail 1% of calls**" (Software Testing p.31). **p.32**, "Time Dependent Problems," is the companion diagram (SUT behind an API with a time-dependent dependency; diagram-only) — the class of code whose behaviour depends on the clock, cured by injecting a controllable time source (Software Testing p.32).

p.33–40 — Automatic tests and the TDD discipline

p.33, "Automatic tests," shows the two `LibraryApp` login tests side by side, labelled "Successful login" and "Failed login": `testLoginAdmin()` (precondition `assertFalse(libApp.adminLoggedIn())`, act `adminLogin("adminadmin")`, postconditions `assertTrue(login)` and `assertTrue(libApp.adminLoggedIn())`) and `testWrongPassword()` (same precondition, act with the wrong password `"admin"`, postconditions `assertFalse(login)` and `assertFalse(libApp.adminLoggedIn())`) (Software Testing p.33). **p.34** is the divider "Test-Driven Development" (Software Testing p.34). **p.35** defines TDD in three bullets: "**Test before the implementation**"; "**Tests = expectations on software**"; "**All kind of tests: unit, component, and system tests**" (Software Testing p.35). **p.36–38** are the three timeline diagrams "Traditional Testing" / "Moving to TDD" / "Real TDD" (Software Testing p.36–38; expanded in Key Concepts above). **p.39**, "TDD cycle," gives the loop verbatim: "Repeat for functionality, bug, . . ." — "**red: Create a failing test; green: Make the test pass; refactor: clean up your code**" — "Until: **no more ideas for tests**"; and the *Important* rules: "**One test at a time**"; "**Implement only as much code so that the test does not fail.**"; "If the method looks incomplete, **add more failing tests that force you to implement more code**" (Software Testing p.39). **p.40**, "Ideas for tests," is the five-source list: "**Use case scenarios (missing functions): Acceptance tests**"; "**Possibility for defects (missing code): Defect tests**"; "**You want to write more code than is necessary to pass the test**"; "**Complex behaviour of classes: Unit tests**"; "**Code experiments: 'How does the system behave, if . . .'**"; plus "**Make a list of new test ideas**" (Software Testing p.40).

p.41–47 — TDD example: Borrow Book

p.41 states the use case: *Name*: borrow book; *Description*: the user borrows a book; *Actor*: user; *Main scenario*: "1. the user borrows a book"; *Alternative scenario*: "1. the user wants to borrow a book, but has already 10 books borrowed; 2. the system presents an error message" (Software Testing p.41). **p.42**, "Create a test for the main scenario," defines *Testdata* ("A user with CPR `"1234651234"` and book with signature `"Som001"` ") and the four-step *Testcase* (retrieve the user by CPR; retrieve the book by signature; the user borrows the book; the book is in the user's borrowed list) (Software Testing p.42). **p.43** is the full `testBorrowBook()` JUnit method implementing those steps with `assertEquals` / `assertFalse` / `assertTrue` (Software Testing p.43). **p.44**, "Implement the main scenario," is the one-line green implementation `public`

`void borrowBook(Book book) { borrowedBooks.add(book); } (Software Testing p.44). p.45, "Create a test for the alternative scenario," extends the test data with "10 books with signatures "book1", ..., "book10"" and the test case: retrieve the user; retrieve and borrow book1 ... book10; retrieve and borrow "Som001"; "Check that a TooManyBooksException is thrown" (Software Testing p.45). p.46 implements it: if (borrowedBooks.size() >= 10) { throw new TooManyBooksException(); } borrowedBooks.add(book); with the method now declared throws TooManyBooksException (Software Testing p.46). p.47, "More test cases," is the defect test: "What happens if book==null in borrowBook?" — retrieve the user, "Call the borrowBook operation with the null value," and "Check that the number of borrowed books has not changed" (Software Testing p.47).`

p.48–49 — Refactoring in TDD and the advantages

p.48, "Refactoring and TDD": refactoring is the "**Third step in TDD**"; "Restructure the system **without changing its functionality**"; "Goal: **improve the design** of the system, e.g. **remove code duplication (DRY principle)**"; it is a "**Necessary step**"; and it "**Requires good testsuite**" — with the forward pointer "later in the course more about refactoring mechanics" (Software Testing p.48). **p.49**, "TDD: Advantages," groups the payoff: *Test benefits* — "**Good code coverage: Only write production code to make a failing test pass**"; *Design benefits* — "**Helps design the system: defines usage of the system before the system is implemented**"; and "**Testable system**" (Software Testing p.49).

p.50–56 — JUnit

p.50 is the divider "JUnit" (Software Testing p.50). **p.51**, "JUnit Intro": "JUnit is a **framework for writing tests**"; "Written by **Erich Gamma (Design Patterns)** and **Kent Beck (eXtreme Programming)**"; "Unit-, component-, and acceptance tests"; the URL <http://www.junit.org>; and the family name "**xUnit**" (Software Testing p.51) [GHJV94]. **p.52**, "JUnit Test Case," is the `EnqueueTest` class: field `private IQueue q`; , the four lifecycle methods (`@BeforeClass setUpClass()`, `@AfterClass tearDownClass()`, `@Before setUp()` building `q = new CheckRepWrapper(new Queue(2))`; , `@After tearDown()` calling `q.empty()`;), and the `@Test testEnqueue()` method (`empty → enqueue 1 → enqueue 2 → enqueue 3 → assertTrue(q.getTail() == 0)`;), documented with the Javadoc "Test of enqueue method, of Queue." (Software Testing p.52). Note that the `@Before` fixture wraps the queue in a `CheckRepWrapper` — the invariant checker from p.19 made into a decorating wrapper, so *every* test action is invariant-checked. **p.53**, "JUnit assertions," lists the API under `import static org.junit.Assert.*`; — *General Assertions*: `assertTrue(bexp)`, `assertTrue(msg,bexp)`; *Specialised assertions for readability*: `assertFalse(bexp)`, `fail()`, `assertEquals(exp,act)`, `assertNull(obj)`, `assertNotNull(obj)`, "..." (Software Testing p.53). **p.54**, "TestSuite," shows `QueueTestSuite` with `@RunWith(Suite.class)` and `@Suite.SuiteClasses({dk.sdu.mmmi.fixedsizequeue.EnqueueTest.class, dk.sdu.mmmi.fixedsizequeue.DequeueTest.class})` — note the fully-qualified `dk.sdu.mmmi.fixedsizequeue` package (the course's own namespace), and that the suite class itself carries (empty) `@BeforeClass/@AfterClass/@Before/@After` methods, showing a suite can host shared fixtures (Software Testing p.54). It also implies a `DequeueTest` class exists as the companion to `EnqueueTest`. **p.55**, "JUnit: exceptions," gives the task "Test that method `m()` throws an exception `MyException`" and both idioms: the `try/ fail()/catch` form with the comment "If we reach here, then the test fails because no exception was thrown" and the catch-block note "Do something to test that `e` has the correct values"; and the "Alternative" `@Test(expected=MyException.class)` (Software Testing p.55). **p.56**, "Links," lists the ecosystem exhaustively: **xUnit**, **CppUnit**, **JUnit**, **Mockito**, **Sikuli Script**, **DbUnit** (Software Testing p.56).

p.57–60 — Mock Objects: Mockito, Mock, Stub

p.57 is the divider "Mock Objects" (Software Testing p.57). **p.58**, "Mockito for mocking objects": "Mockito is a **popular mock framework** which can be used **in conjunction with JUnit**. Mockito allows you to **create and configure mock objects**." (Software Testing p.58). **p.59**, "Mock," defines the heaviest double verbatim: "Mocks are the objects that **store method calls**. It referred to as the **dynamic wrappers for dependencies** used in the tests. It is used to **record and verify the interaction** between the Java classes. A mock is known as the **most powerful and flexible** version of the test doubles. We use a method for mocking that is called **mock()**." (Software Testing p.59). **p.60**, "Stub," defines the lightest: "Stubs are objects holding **predefined data** to provide responses during tests. They resemble real objects with minimal methods, used when real data responses are undesirable. Stubs are considered the **lightest and most static** test doubles." (Software Testing p.60).

p.61–67 — The DataServer sequence and Mockito's limitations

p.61–64 are the four storyboard slides "Problem" → "Create DataServer" → "Create Mock" (stand-in labelled *Mock*) → "How?" (Software Testing p.61–64; expanded in Key Concepts above). **p.65**, "Using Mockito," gives the creation mechanics: the static `mock()` method; the `@Mock` annotation; and the requirement "If using annotations, you must initialize mock objects with an `MockitoAnnotations.initMocks(this)` method call." (Software Testing p.65). **p.66** is the companion "Using Mockito" code slide (diagram/screenshot; Software Testing p.66). **p.67**, "Limitations," is the exhaustive three-item list: "Mockito has certain limitations. It can not test the following constructs: **Final classes, Anonymous classes, Primitive types**" (Software Testing p.67).

p.68–71 — Overdue book; configuring and verifying mocks

p.68 is the example slide "Mock Example 1: **Overdue book**" (Software Testing p.68) — a library-domain mock scenario (visual), the natural fit being a time/overdue dependency replaced by a double (cf. p.32). **p.69**, "Configuring the mock objects," states the fluent API: "Mockito has a **fluent API**. You can use the `verify()` method to ensure that a method was called. `when(...).thenReturn(...)` can be used to specify a condition and a return value for this condition. `doReturn(object).when(...)` method call works similar." (Software Testing p.69). **p.70** shows the stubbing test: `MyClass test = Mockito.mock(MyClass.class);` then (as printed on the slide) `test.when(test.getUniqueId()).thenReturn(43);` under the comment "Define return value for method getUniqueId()" and a `// TODO use mock in test...` placeholder (Software Testing p.70 — see the slide-code traps subsection for the `test.when` typo). **p.71**, "Verify the calls on the mocks," extends the same test with `Mockito.verify(test).testing(Matchers.eq(12));` under "Now check if method testing was called with the parameter 12" and `Mokito.verify(test, Mokito.times(2));` under "Was the method called twice?" (Software Testing p.71 — `Mokito` as printed; see traps).

p.72–75 — Spy, verifyNoMoreInteractions, @InjectMocks

p.72, "Spy," defines the partial double verbatim: "Spies are known as **partially mock objects**. It means spy creates a **partial object or a half dummy of the real object** by stubbing or spying the real ones. In spying, the **real object remains unchanged**, and we just **spy some specific methods** of it. In other words, we take the existing (real) object and replace or spy only some of its methods." (Software Testing p.72). **p.73** adds the mechanism: "**@Spy or the spy() method** can be used to wrap a real object. **Every call, unless specified otherwise, is delegated to the object.**" (Software Testing p.73). **p.74** introduces two more tools: "The `verifyNoMoreInteractions()` allows you to check that **no other method was called**. You also have the `@InjectMocks` annotation which tries to do **constructor dependency injection based on the type**." (Software Testing p.74). **p.75** is the full `ArticleManagerTest: @Mock private ArticleCalculator`


```
calculator;, @Mock private ArticleDatabase database;, @Spy private UserProvider userProvider =
new ConsumerUserProvider();, and @InjectMocks private ArticleManager manager = new
ArticleManager(); with the comment "Creates instance of ArticleManager and performs constructor
injection on it"; the test method shouldDoSomething() calls MockitoAnnotations.initMocks(this); (sic) and
verify(database).addListener(any(ArticleListener.class)); (Software Testing p.75).
```

p.76 — Spring and JUnit

A single slide showing container-driven testing: `@RunWith(SpringJUnit4ClassRunner.class)`, `@ContextConfiguration(classes = {AppConfig.class})`, the injected field `@Autowired @Qualifier("ml") DataModelService ml;` (commented `//DI`), and the test `test_ml_always_return_true()` asserting `assertThat(ml, instanceOf(MachineLearningService.class));` (commented "assert correct type/impl") and `assertThat(ml.isValid(""), is(true));` (commented "assert true") (Software Testing p.76). The `@Qualifier("ml")` selects *which* configured bean to inject when several share the type — the test then asserts the wiring delivered the expected implementation class before asserting behaviour.

p.77–81 — Acceptance tests; manual vs. automated; regression

p.77 is the divider "Acceptance Tests" (Software Testing p.77). **p.78** defines them: "Tests defined **by/with the help of the user** based on the **requirements**"; *Traditionally*: "manual tests / by the customer / after the software is delivered / based on use cases / user stories"; *Agile software development*: "automatic tests: **JUnit, Fit, . . .** / created **before** the user story is implemented" (Software Testing p.78). **p.79**, "Example of acceptance tests," is the Login Admin template in full: *Name*: Login Admin; *Actor*: Admin; *Precondition*: "Admin is not logged in"; *Main scenario*: "1. Admin enters password; 2. System responds true"; *Alternative scenarios*: "1a. Admin enters wrong password; 1b. The system reports that the password is wrong and the use case starts from the beginning"; *Postcondition*: "Admin is logged in" (Software Testing p.79) — note this is exactly the scenario automated by the two p.33 login tests. **p.80** is the divider "Manual tests" (Software Testing p.80). **p.81**, "Manual vs. automated tests," is the verdict slide: "**Manual tests should be avoided** — They are expensive (time and personal) to execute: **Can't be run often**"; "**Automated tests** — Are cheap (time and personal) to execute: Can be run **as soon something is changed** in the system — **immediate feedback if a code change introduced a bug** → **Regression tests**"; "More difficult (but not impossible) when they include the UI — **Solution: Test under the UI**" (Software Testing p.81).

p.82–83 — Summary: Being Great at Testing

p.82 is the divider "Summary" (Software Testing p.82). **p.83**, "Being Great at Testing," closes the deck: "**testing + development are different** — Developer: 'I want this code to succeed.' / Tester: 'I want this code to fail.' / **Hybrid**"; "**Test creatively**"; "**Don't ignore the weird stuff**"; signed off with "**Fun ... Profit**" (Software Testing p.83).

Divider and diagram-only pages at a glance

For completeness, the deck's pages that carry *no body text beyond a title* (section dividers) or are *primarily visual* are inventoried here, so you know exactly which quotes can and cannot be sourced to them:

- **Pure section dividers (title only)**: p.2 "Introduction", p.10 "Testing Example", p.15 "Assertions", p.22 "Software Under Test", p.24 "The Fragile Test Problem", p.34 "Test-Driven Development", p.50 "JUnit", p.57 "Mock Objects", p.77 "Acceptance Tests", p.80 "Manual tests", p.82 "Summary" (Software Testing p.2, p.10, p.15, p.22, p.24, p.34, p.50, p.57, p.77, p.80, p.82).
- **Titled visual pages (title + diagram, minimal or no prose)**: p.4 "Turing's halting problem" (with the JAVA/C#/C++/C labels), p.16 "Enable Assertions", p.25–28 the four sensitivity titles, p.32 "Time

Dependent Problems", p.36–38 the Traditional/Moving-to-TDD/Real-TDD timelines, p.61–64 the Problem/DataServer/Mock/How? storyboard, p.66 the "Using Mockito" code visual, p.68 "Mock Example 1: Overdue book" (Software Testing p.4, p.16, p.25–28, p.32, p.36–38, p.61–64, p.66, p.68).

- **Everything else carries quotable text or code** — definitions (p.3, p.5–6, p.18, p.20–21, p.35, p.39–40, p.48–49, p.51, p.58–60, p.65, p.67, p.69, p.72–74, p.78, p.81, p.83), full code (p.11, p.13, p.19, p.33, p.43–46, p.52–55, p.70–71, p.75–76), or structured templates and data (p.7–9, p.12, p.14, p.17, p.23, p.29–31, p.41–42, p.45, p.47, p.56, p.79).

Why this matters: if an exam answer attributes a verbatim sentence to a divider page (say, quoting a "definition" from p.57), the citation is wrong — the mock/stub/spy definitions live on p.59, p.60, and p.72 respectively. Conversely, for the visual pages this guide's prose is expansion (flagged in the grounding note), so paraphrase rather than "quote" them.

Compare & Contrast Tables

Quick-revision tables for the distinctions the exam most often probes. Every cell is grounded in the cited pages.

Testing levels at a glance

	Unit testing	Integration testing	System testing
Scope	Smallest testable unit — in OO often a class/interface, possibly a method (TestLab1 p.1)	Several already-tested units across their interfaces (Software Testing p.8)	The whole assembled system end-to-end (Software Testing p.8)
Fault class caught	Logic faults inside one component (wrong condition, off-by-one)	Interaction faults at the seams (mismatched assumptions, formats)	Whole-system faults: config, workflows, unmet requirements
Isolation	Total — dependencies replaced by mocks/stubs (TestLab1 p.1)	Real seams between the integrated units	None — real system
Who/when	Developers, continuously (TestLab1 p.1)	After units are green	Last, against the spec/acceptance criteria
Canonical example	Queue <code>enqueue</code> boundary tests (Software Testing p.11)	Mars-Orbiter-style seam mismatch (Software Testing p.9)	Login → borrow → loan visible (Software Testing p.79)

White box vs. black box

	White-box (glass-box)	Black-box
Knowledge used	Source code structure (Software Testing p.8)	Specification / interface only (Software Testing p.8, p.23)
Derives tests from	Statements, branches, paths	Domain → range contract (<code>sqrt</code> family, p.23)
Catches	Implementation-specific faults (unreached branch, wrap-around path)	Contract violations and <i>missing</i> functionality
Survives refactoring?	Often breaks (coupled to internals)	Yes, if behaviour is preserved

	White-box (glass-box)	Black-box
Deck example	<code>testEnqueueTail</code> asserting <code>getTail()==0</code> (Software Testing p.13)	<code>testEquality</code> via the <code>IQueue</code> contract (Software Testing p.13)
Enables	Coverage measurement (statement/branch/path)	Equivalence partitioning, boundary analysis (p.11–12)

Differential vs. stress vs. random testing

	Differential	Stress	Random
Slide marker	" = ? " (Software Testing p.8)	"Stress testing" (Software Testing p.8)	" ?? ? " (Software Testing p.8)
Core idea	Compare SUT output to a reference implementation	Push past expected load/volume	Generate inputs the author never imagined
Solves	The oracle problem (the other implementation is the oracle)	Non-functional limits (leaks, deadlocks, throughput)	Input-selection bias
Maintenance use	Old release as free oracle for the new one	Pre-release capacity check	Fuzzing parsers/decoders with invariant oracles

Stub vs. mock vs. spy

	Stub	Mock	Spy
Slide definition	"Objects holding predefined data... lightest and most static test doubles" (Software Testing p.60)	"Store method calls... record and verify the interaction... most powerful and flexible " (Software Testing p.59)	" Partially mock objects ... real object remains unchanged, we just spy some specific methods" (Software Testing p.72)
Underlying object	None real — canned answers only	None real — dynamic wrapper	A real object, wrapped (Software Testing p.73)
Default behaviour	Returns predefined data	Records calls; returns stubbed values	"Every call, unless specified otherwise, is delegated to the object" (Software Testing p.73)
Verification style	State (assert on the SUT's result)	Behaviour (<code>verify()</code> the interaction, p.69)	Mixed — real behaviour plus targeted <code>verify</code> /override
Mockito construct	<code>when().thenReturn()</code> on a mock (p.69–70)	<code>mock()</code> / <code>@Mock</code> (p.59, p.65)	<code>spy()</code> / <code>@Spy</code> (p.73)
Use when	Pure query needs controlled input	The interaction <i>is</i> the thing under test	Mostly-real behaviour with one override

Assertion vs. exception

	Java assertion	Exception
Checks for	"Something that should never happen" — programmer error (TestLab1 p.1; Software Testing p.18 Rule 1)	Expected runtime error (bad input, missing file)

	Java assertion	Exception
On trigger	"Should stop the program from running" (TestLab1 p.1)	"Should let the program continue running" (TestLab1 p.1)
Can be disabled?	Yes — off by default, enable at runtime (Software Testing p.16–17)	No — always active
Side effects allowed?	Never (Rule 2 — behaviour must not change when disabled, p.18)	Normal code rules apply
Deck example	<code>assert result > 0;</code> in <code>sqrt</code> ; <code>assert sizeMax > 0;</code> in the Queue constructor (p.18, p.11)	<code>throw new TooManyBooksException();</code> for the legitimate 10-book limit (p.46)

The sharpest exam cue: the 10-book limit in Borrow Book is an **exception**, not an assertion — a perfectly valid user can hit it (Software Testing p.45–46); a zero-size queue can only come from buggy code, so it is an **assertion** (p.11).

JUnit 4 lifecycle annotations

Annotation	Static ?	Runs	Typical use (deck example)
@BeforeClasses	Yes	Once , before all tests in the class	Expensive shared setup (<code>setUpClass</code> , Software Testing p.52)
@Before	No	Before each test method	Fresh fixture — <code>q = new CheckRepWrapper(new Queue(2));</code> (p.52)
@Test	No	The test itself	<code>testEnqueue()</code> (p.52)
@After	No	After each test method	Cleanup — <code>q.empty();</code> (p.52)
@AfterClasses	Yes	Once , after all tests in the class	Release shared resources (<code>tearDownClass</code> , p.52)

The two exception-testing idioms

	try / fail() / catch	@Test(expected=...)
Form	<code>try { m(); fail(); } catch (MyException e) { ... }</code> (Software Testing p.55)	<code>@Test(expected=MyException.class)</code> (Software Testing p.55)
Fails when no exception because	The explicit <code>fail()</code> after <code>m()</code> runs (slide comment: "If we reach here, then the test fails because no exception was thrown", p.55)	The framework notices no exception of the declared type
Can inspect the exception?	Yes — "Do something to test that e has the correct values" (p.55)	No — only the type is checked
Choose when	Message/contents/cause matter	Type alone suffices; conciseness wins

Manual vs. automated testing

	Manual	Automated
Slide verdict	"Should be avoided" (Software Testing p.81)	Preferred

	Manual	Automated
Cost	"Expensive (time and personal) to execute" (p.81)	"Cheap (time and personal) to execute" (p.81)
Frequency	"Can't be run often" (p.81)	"As soon something is changed in the system" (p.81)
Consequence	Bugs sit undetected between rare runs	"Immediate feedback if a code change introduced a bug → Regression tests " (p.81)
Hard case	—	The UI ("more difficult but not impossible") → test under the UI (p.81)

Traditional acceptance testing vs. agile acceptance testing

	Traditional	Agile
Mode	Manual tests (Software Testing p.78)	Automatic tests: "JUnit, Fit, . . ." (p.78)
Run by	The customer (p.78)	The build/the team
When	"After the software is delivered" (p.78)	"Created before the user story is implemented" (p.78)
Based on	Use cases / user stories (p.78)	The same — but encoded as executable tests up front

Equivalence partitioning vs. boundary-value analysis

	Equivalence partitioning	Boundary-value analysis
Picks	One representative <i>inside</i> each class (Software Testing p.12)	The values <i>at the edges</i> between classes
Rationale	Same-class inputs exercise the same logic — more is redundant (p.12)	Defects cluster at edges (off-by-one, overflow, empty/full)
Deck cue	<code>sleep(100)</code> variant is equivalent — time can't matter (p.12)	The Queue run's <code>true, true, false, 6, 7, null</code> — all edge outcomes (p.11)
Lab status	Implied by "best case scenario" tests (TestLab1 p.1)	Explicitly required: "identified boundary cases" (TestLab1 p.1)
Relationship	Defines the classes	Tests each class's borders (B-1, B, B+1) — apply <i>both</i>

Easily Confused Distinctions (rapid-fire exam review)

Each entry is a pair (or trio) that exam questions like to blur. Learn the *discriminating sentence* for each.

Java `assert` vs. JUnit assertions

The **Java `assert` keyword** lives *inside production code*, checks an invariant on every (assertion-enabled) run, can be disabled (`-ea` off by default), and throws `AssertionError` to *stop* the program (Software Testing p.16–18; TestLab1 p.1). **JUnit assertions** (`assertTrue`, `assertEquals`, ... from `org.junit.Assert`) live *inside test code*, run only when the test runs, cannot be "disabled" by a JVM flag, and fail *the test*, not the application (Software Testing p.53). The deck uses both around the same Queue: `checkRep()` is Java asserts in/near the SUT (p.19); `testEnqueue` uses JUnit's `assertTrue` in the test class (p.52). *Discriminator*: who fails — the program (Java assert) or the test (JUnit assert).

checkRep() vs. setUp()

`checkRep()` is the *invariant bundle* — a method of (or wrapper around) the SUT containing the representation-invariant assertions (Software Testing p.19). `setUp()` is the *JUnit fixture method* annotated `@Before` that builds a fresh SUT for each test (Software Testing p.52). They meet on p.52: `setUp()` constructs `new CheckRepWrapper(new Queue(2))`, wiring the invariant checker around the fixture so every test action is invariant-validated. *Discriminator*: `checkRep` checks state correctness; `setUp` creates state.

@Before vs. @BeforeClass

`@Before` runs before **each** test method on a fresh instance; `@BeforeClass` is **static** and runs **once** for the whole class (Software Testing p.52). Wrong choice symptoms: heavy per-test setup in `@Before` → slow suite; shared mutable state in `@BeforeClass` → order-dependent tests (the global-state fragility of p.14).

Discriminator: frequency — per-test vs. per-class.

mock() vs. @Mock (and the initMocks trap)

Both create a Mockito mock: `Mockito.mock(MyClass.class)` is the *programmatic* form (Software Testing p.65, p.70); `@Mock` is the *annotation* form, which stays `null` unless you call

`MockitoAnnotations.initMocks(this)` (Software Testing p.65, p.75). *Discriminator*: annotations need the init call; the static method does not.

when().thenReturn() vs. verify()

`when(x.method()).thenReturn(value)` is **stubbing** — programming the double's *future answers* (state side) (Software Testing p.69–70). `verify(x).method(args)` is **verification** — checking the double's *past calls* (behaviour side) (Software Testing p.69, p.71). One configures before the act; the other asserts after it.

Discriminator: before-the-act programming vs. after-the-act checking.

verify(...) vs. verifyNoMoreInteractions()

`verify(test).testing(Matchers.eq(12))` asserts a *specific* call happened (Software Testing p.71); `verifyNoMoreInteractions()` asserts *nothing else* happened — "check that no other method was called" (Software Testing p.74). The first is positive evidence, the second negative evidence; together they pin the SUT's exact interaction protocol.

Spy vs. mock

A **mock** is a full stand-in — no real object exists behind it; every unstubbed call returns defaults (Software Testing p.59). A **spy** wraps a *real* object — "every call, unless specified otherwise, is delegated to the object" (Software Testing p.73), and "the real object remains unchanged, and we just spy some specific methods" (Software Testing p.72). *Discriminator*: default behaviour — empty (mock) vs. real (spy).

Stub (the double) vs. fault injection (the technique)

A **stub** is an *object* holding predefined data (Software Testing p.60). **Fault injection** is a *technique* — deliberately making a dependency fail on demand, e.g. `my-open` that "succeed[s] 100 times then fail[s] 1% of calls" (Software Testing p.31). They intersect: a stub configured to throw *implements* fault injection at the unit level. *Discriminator*: noun (a double) vs. verb (a strategy for reaching unhappy paths).

Acceptance test vs. system test

Both exercise the whole system, but the **acceptance test** is defined "by/with the help of the **user** based on the **requirements**" (Software Testing p.78) — its authority is the user's intent, structured by the Name/Actor/Precondition/Scenarios/Postcondition template (p.79). A **system test** is the developer-side end-to-end level from the p.8 ladder, judged against the specification. *Discriminator*: who defines pass — the user (acceptance) vs. the spec (system).

Regression testing vs. re-running a failed test

Regression testing re-runs the *entire existing suite* after a change to catch *previously working* behaviour that broke — "immediate feedback if a code change introduced a bug" (Software Testing p.81). Re-running one failed test only confirms a *known* failure. *Discriminator*: breadth and target — everything that used to pass (regression) vs. the one thing that didn't.

Fragile test vs. failing test

A **failing** test may be doing its job (a real defect, or a legitimate behaviour change worth reviewing — behavior sensitivity, Software Testing p.26). A **fragile** test fails *without* a real defect — over-coupled to interface shape, incidental behaviour, shared data, or environment (Software Testing p.25–28). *Discriminator*: is there a defect behind the red? If no, the test (not the code) needs fixing.

Test under the UI vs. record-and-play

Both respond to the GUI-testing problem (clicks/events/swipes in, app states out, Software Testing p.30).

Test under the UI *bypasses* the GUI and tests the logic directly — the preferred, stable, automatic route (Software Testing p.29, p.81). **Record and play using scripts** *drives* the GUI by replaying captured sessions (p.30) — e.g. Sikuli Script from the links slide (p.56) — and inherits the GUI's fragility.

Discriminator: below the UI (avoid it) vs. through the UI (replay it).

Defect test vs. unit test (in the p.40 idea list)

In TDD's idea sources both are unit-level JUnit tests, but they answer different prompts: a **defect test** comes from "possibility for defects (missing code)" — including the rule that wanting to write more code than the current test demands means you owe a new failing test first (Software Testing p.40); a **unit test** in the p.40 sense comes from "complex behaviour of classes." The `book==null` case is the deck's worked defect test (Software Testing p.47). *Discriminator*: the trigger — a suspected edge/missing code (defect) vs. intrinsic complexity (unit).

R vs. F in the sqrt specifications

`sqrt: R -> C` says mathematical reals can map negatives to *complex* results; `sqrt: F -> F+ U exception` says machine floats cannot represent `i`, so negative input must yield an exception (or `NaN`, per the slide's `-1 -> i, NaN, exception` line) (Software Testing p.23). Same operation, different domain, different contract — and therefore *different test classes*. *Discriminator*: mathematical idealisation (`R`) vs. machine representation (`F`).

"Testing incompleteness" vs. "tests are useless"

The halting-problem result says a *perfect* suite is impossible (Software Testing p.5) — it does **not** say testing is futile: "well designed tests come close to be adequate" (p.5), and the whole deck is the toolbox for designing them. *Discriminator*: impossibility of perfection vs. possibility of adequacy — quote both halves of p.5.

Developer mindset vs. tester mindset (and the hybrid)

Developer: "I want this code to succeed." Tester: "I want this code to fail." (Software Testing p.83). The deck's ideal is the **Hybrid** who switches stances — building optimistically, then attacking adversarially, testing creatively and not ignoring the weird stuff (p.83). *Discriminator*: the desired outcome of a run — green (developer) vs. red (tester).

Self-Test Questions and Answers

Flashcard-style review, grouped by deck section. Cover the answer column and quiz yourself; every answer cites its slide.

Foundations (p.1–9)

- **Q: What is the deck's subtitle, and why does it matter?** A: "How to make software fail!" (Software Testing p.1) — testing's purpose is to provoke failures in the lab before users meet them; it foreshadows the tester's "I want this code to fail" mindset (p.83).
- **Q: Define a Turing machine as the slide does.** A: "A simple mathematical model of computation. It manipulates symbols on a tape based on rules, yet can implement any computer algorithm." (Software Testing p.3).
- **Q: Which languages does the halting-problem slide name, and what is the implication?** A: JAVA, C#, C++, C (Software Testing p.4) — all Turing-complete, so all inherit the halting problem and therefore testing incompleteness (p.5).
- **Q: Why is a perfect test suite impossible?** A: The halting problem makes general program behaviour undecidable — "theoretical reason for testing incompleteness... theoretically impossible to create a perfect test suite"; the best achievable is "well designed tests come close to be adequate" (Software Testing p.5).
- **Q: Quote Dijkstra's principle and its corollary.** A: "Testing can demonstrate the presence of bugs, but not their absence." Corollary: "Residual bugs can still hide in the code, undetected by tests, as no test suite guarantees an error-free program." (Software Testing p.6).
- **Q: Name the four boxes of the basic test model.** A: Test inputs → Software Under Test → Test outputs → "Outputs OK?" (the oracle) (Software Testing p.7).
- **Q: List everything on the Kinds-of-Testing map.** A: Levels: unit, integration, system; knowledge axis: white box / black box; plus differential testing ("= ?"), stress testing, and random testing ("??"/"??") around the SUT (Software Testing p.8).
- **Q: A test goes red. List the four fault locations in triage order.** A: Bug in SUT? → bug in acceptability test? → bug in specification? → bug in OS/compilers/libs/hardware? (Software Testing p.9).
- **Q: What exactly went wrong on the Mars Climate Orbiter, and which triage branch is it?** A: One side used metric (m/s), the other English units (ft/s) — a specification/interface mismatch (branch 3), invisible to each side's own tests (Software Testing p.9).

The Queue, partitions, boundaries, specifications (p.10–13, p.23)

- **Q: What does the run `q=Queue(2); enqueue(6); enqueue(7); enqueue(8); dequeue()×3` return?** A: `true, true, false, 6, 7, null` (Software Testing p.11).
- **Q: Why does the third `enqueue` return `false` and the third `dequeue` return `null`?** A: `enqueue` rejects when `size == max` (full) by returning `false`; `dequeue` signals empty (`size == 0`) by returning `null` — possible because its return type is boxed `Integer` (Software Testing p.11).

- **Q: What is the wrap-around rule in enqueue, and which test targets it?** A: `if (tail == max) tail = 0;` — targeted by the white-box `testEnqueueTail()` asserting `q.getTail() != 0` fails, i.e. tail wrapped to 0 (Software Testing p.11, p.13), and by JUnit's `testEnqueue (assertTrue(q.getTail() == 0))`, p.52).
- **Q: What assertion guards the Queue constructor and why is it an assertion, not an exception?** A: `assert sizeMax > 0;` — only a buggy caller requests a non-positive capacity, so it is a "should never happen" check per Rule 1 (Software Testing p.11, p.18).
- **Q: Why is the `sleep(100)` test equivalent to the plain enqueue/dequeue test?** A: The queue's contract is time-independent, so both inputs sit in the same equivalence class — testing both is redundant (Software Testing p.12).
- **Q: What does the "Large Integers?" label on p.12 warn about?** A: The *output space* has its own partitions — values near the representable limit form a distinct class (potential overflow) needing its own test (Software Testing p.12).
- **Q: Write the four sqrt specifications from p.23.** A: `sqrt: R+ -> R+;` `sqrt: R -> C;` `sqrt: F+ -> F+;` `sqrt: F -> F+ U exception` (Software Testing p.23).
- **Q: Per the slide, what can `sqrt(9)` and `sqrt(-1)` yield?** A: `9 -> -3 or 3` (both roots); `-1 -> i`, `NaN`, `exception` depending on the domain/type system (Software Testing p.23).

Testability and assertions (p.14–21)

- **Q: Recite the full Creating-Testable-Software checklist.** A: Clean Code; Refactor; Describe what it does and how it interacts; No extra Threads; No swap of global variables; No pointer soup; Module unit tests; Support fault injection; Assertions, Assertions Assertions!!! (Software Testing p.14).
- **Q: Define an assertion per the deck.** A: "Executable check for a property that must be true (invariant)" (Software Testing p.18).
- **Q: State the three assertion rules with their slide counter-examples.** A: Rule 1 — not for error handling; Rule 2 — NO SIDE EFFECTS (`assert foo()==0;` where `foo()` changes a global variable); Rule 3 — no silly assertions (`assert 1+1==2;`) (Software Testing p.18).
- **Q: Give the three "Why assertions?" benefits.** A: Self-checking code (effective testing); fail early, closer to the bug; document assumptions, preconditions, postconditions, invariants (Software Testing p.20).
- **Q: Quote the production-assertion statistics.** A: GCC ~9,000 assertions; LLVM ~13,000; about one assertion per 110 lines of code (Software Testing p.20).
- **Q: Advantages and disadvantages of disabling assertions?** A: Advantages: code runs faster, code keeps running. Disadvantages: code may rely on a side-effect assertion; "even in production code, may be better to fail early" (Software Testing p.17).
- **Q: When should assertions be enabled vs. disabled, per p.21?** A: Enabled in running software "that can be recovered by failing early"; disabled "in mission critical stage when it is better to continue then recover" (Software Testing p.21).
- **Q: What four conditions does the Queue's `checkRep()` assert?** A: `0 ≤ size ≤ max;` if `tail > head` then `tail-head == size;` if `tail < head` then `head-tail == max-size;` if `tail == head` then `size == 0 || size == max` (Software Testing p.19).

Fragile tests, GUI, fault injection, time (p.24–32)

- **Q: Name the four fragile-test sensitivities in slide order.** A: Interface sensitivity, behavior sensitivity, data sensitivity, context sensitivity (Software Testing p.25–28).

- **Q: What conversion does the "Testing under the UI" slide depict?** A: Manual Test → Automatic Test — moving the checks beneath the GUI turns a brittle manual procedure into a stable automatic one (Software Testing p.29).
- **Q: Why are GUIs intrinsically hard to test, in the slide's terms?** A: The domain is "clicks, events, swipes..." and the range is "GUI → App. states" — neither plain parameters nor return values; the GUI-level fallback is "Record and Play using scripts" (Software Testing p.30).
- **Q: Describe the p.31 fault-injection example precisely.** A: Replace `open("/tmp/foo", 'w')` with `my-open("/tmp/foo", 'w')`, a wrapper programmed to "succeed 100 times then fail 1% of calls" — making rare I/O failures reproducible (Software Testing p.31).
- **Q: What problem class does p.32 name and what is the standard cure?** A: Time-dependent problems — code whose behaviour depends on the clock; cure: put time behind an injectable dependency so tests control "now" (Software Testing p.32; cf. the Overdue-book mock, p.68).

TDD (p.33–49)

- **Q: What three bullets define TDD on p.35?** A: Test before the implementation; tests = expectations on software; all kinds of tests — unit, component, and system (Software Testing p.35).
- **Q: Name the TDD cycle's steps and stop condition.** A: red (create a failing test) → green (make it pass) → refactor (clean up); repeat for functionality, bug, ..., until "no more ideas for tests" (Software Testing p.39).
- **Q: State the three "Important" TDD discipline rules.** A: One test at a time; implement only as much code so that the test does not fail; if the method looks incomplete, add more failing tests that force more code (Software Testing p.39).
- **Q: List all five idea-sources for tests (plus the meta-rule).** A: Use-case scenarios (missing functions) → acceptance tests; possibility for defects (missing code) → defect tests; wanting to write more code than necessary to pass; complex behaviour of classes → unit tests; code experiments ("How does the system behave, if..."); and: make a list of new test ideas (Software Testing p.40).
- **Q: In Borrow Book, what are the test data for the main and alternative scenarios?** A: Main: user CPR "1234651234", book signature "Som001" (p.42). Alternative: the same plus 10 books "book1" ... "book10" (Software Testing p.42, p.45).
- **Q: What exception does the alternative scenario expect and what guard implements it?** A: `TooManyBooksException`, thrown when `borrowedBooks.size() >= 10` (Software Testing p.45–46).
- **Q: What is the deck's worked defect test for `borrowBook`?** A: `book == null` — call `borrowBook(null)` and check the number of borrowed books has not changed (Software Testing p.47).
- **Q: Why does refactoring require a good test suite?** A: Refactoring restructures "without changing functionality"; only passing tests prove functionality was preserved — which is also why it is a necessary, not optional, third step (Software Testing p.48).
- **Q: TDD's advantage families?** A: Test benefits — good code coverage (only write production code to make a failing test pass); design benefits — defines usage of the system before implementation; and a testable system (Software Testing p.49).

JUnit (p.50–56)

- **Q: Who wrote JUnit, and what else are they known for?** A: Erich Gamma (Design Patterns [GHJV94]) and Kent Beck (eXtreme Programming) (Software Testing p.51).
- **Q: In `EnqueueTest`, what exactly do `setUp()` and `tearDown()` do?** A: `setUp()` (`@Before`) builds `q = new CheckRepWrapper(new Queue(2));`; `tearDown()` (`@After`) calls `q.empty()`; (Software Testing p.52).

- **Q: Which JUnit assertions does p.53 list, split by category?** A: General: `assertTrue(bexp)`, `assertTrue(msg, bexp)`. Specialised for readability: `assertFalse(bexp)`, `fail()`, `assertEquals(exp, act)`, `assertNull(obj)`, `assertNotNull(obj)`, ... (Software Testing p.53).
- **Q: How is a test suite declared, and which classes does the deck's suite contain?** A: `@RunWith(Suite.class) + @Suite.SuiteClasses({...})`; the suite `QueueTestSuite` contains `dk.sdu.mmmi.fixedsizequeue.EnqueueTest` and `dk.sdu.mmmi.fixedsizequeue.DequeueTest` (Software Testing p.54).
- **Q: Two ways to test that `m()` throws `MyException`?** A: `try/ fail() /catch` (inspectable exception) and `@Test(expected=MyException.class)` (type only) (Software Testing p.55).
- **Q: Reproduce the p.56 tool list and one-line roles.** A: `xUnit` (framework family), `CppUnit` (C++ port), `JUnit` (Java), `Mockito` (mocking), `Sikuli Script` (image-based GUI scripting), `DbUnit` (database fixtures) (Software Testing p.56).

Mockito and test doubles (p.57–76)

- **Q: Define mock, stub, spy in the slides' own key phrases.** A: `Mock` — stores method calls, records and verifies interaction, most powerful and flexible, created with `mock()` (p.59). `Stub` — predefined data, lightest and most static (p.60). `Spy` — partially mock object; real object remains unchanged, only some methods spied (p.72) (Software Testing p.59–60, p.72).
- **Q: What must you do when creating mocks via annotations?** A: Call `MockitoAnnotations.initMocks(this)` — otherwise `@Mock` fields are never initialised (Software Testing p.65).
- **Q: Mockito's three stated limitations?** A: It cannot test final classes, anonymous classes, primitive types (Software Testing p.67).
- **Q: What do `when(...).thenReturn(...)`, `doReturn(...).when(...)`, and `verify()` each do?** A: `when().thenReturn()` specifies a condition and its return value; `doReturn().when()` "works similar"; `verify()` ensures a method was called (Software Testing p.69).
- **Q: How do you check an argument and a call count in verification?** A: `Mockito.verify(test).testing(Matchers.eq(12))`; for the argument; `times(2)` for "was the method called twice?" (Software Testing p.71).
- **Q: What does `verifyNoMoreInteractions()` add, and what does `@InjectMocks` do?** A: It checks "no other method was called"; `@InjectMocks` "tries to do constructor dependency injection based on the type" — auto-wiring your `@Mock` / `@Spy` fields into the SUT (Software Testing p.74).
- **Q: In `ArticleManagerTest`, which fields carry which annotations?** A: `@Mock ArticleCalculator calculator`; `@Mock ArticleDatabase database`; `@Spy UserProvider userProvider = new ConsumerUserProvider()`; `@InjectMocks ArticleManager manager = new ArticleManager()`; the test verifies `database.addListener(any(ArticleListener.class))` (Software Testing p.75).
- **Q: In the Spring+JUnit slide, what do the two `assertThat` lines establish?** A: First that the injected `ml` bean is an instance of `MachineLearningService` (correct implementation wired), then that `ml.isValid("")` is `true` (correct behaviour) (Software Testing p.76).

Acceptance, regression, mindset (p.77–83)

- **Q: Define an acceptance test and its two delivery modes.** A: A test "defined by/with the help of the user based on the requirements"; traditionally manual, by the customer, after delivery, from use cases/user stories; in agile, automatic (JUnit, Fit, ...) and created before the user story is implemented (Software Testing p.78).

- **Q: Reproduce the Login Admin acceptance template's six fields.** A: Name: Login Admin; Actor: Admin; Precondition: Admin is not logged in; Main scenario: 1. Admin enters password, 2. System responds true; Alternative: 1a. wrong password, 1b. system reports wrong and the use case restarts; Postcondition: Admin is logged in (Software Testing p.79).
- **Q: Why exactly should manual tests be avoided, in slide terms?** A: "They are expensive (time and personal) to execute: Can't be run often" (Software Testing p.81).
- **Q: How does the deck define the path from automation to regression testing?** A: Automated tests are cheap, runnable "as soon something is changed in the system," giving "immediate feedback if a code change introduced a bug → Regression tests" (Software Testing p.81).
- **Q: The closing slide's three pieces of advice?** A: Recognise developer vs. tester stances (succeed vs. fail) and be the hybrid; test creatively; don't ignore the weird stuff (Software Testing p.83).

TestLab1

- **Q: What is a "unit" in OO programming per the lab sheet?** A: "Often an entire interface, such as a class, but could be an individual method" (TestLab1 p.1).
- **Q: Recite the lab's five classwork steps.** A: (1) Add the Maven dependency to JUnit 4; (2) create JUnit 4 tests for the most important domain-logic methods of your feature (Swing extensions work best with JUnit 4); (3) write best-case-scenario tests; (4) write boundary-case tests — a unit test should test a single code-path through a single method, using mocks/stubs when execution passes outside the method; (5) use Java assertions to test invariants (TestLab1 p.1).
- **Q: The lab's one-sentence assertion-vs-exception rule?** A: "An assertion should stop the program from running, but an exception should let the program continue running." (TestLab1 p.1).
- **Q: What does the portfolio require?** A: Class-level unit tests of important business functionality of your selected feature, plus documentation of how you verified the feature (TestLab1 p.1).
- **Q: What is the lab's bottom-up comprehensive-testing sentence?** A: "By writing tests first for the smallest testable units, then the compound behaviors between those, one can build up comprehensive tests for complex applications." (TestLab1 p.1).

Mixed synthesis drill (cross-topic)

- **Q: Connect the halting problem to the existence of equivalence partitioning.** A: Because a perfect (exhaustive) suite is theoretically impossible (Software Testing p.5), testers need a principled way to choose a *finite* subset of inputs; partitioning supplies it — one representative per behaviour class is the rational way to spend the finite budget (Software Testing p.12).
- **Q: The Queue example appears on p.11, p.13, p.19, p.52 and p.54. Name what each appearance teaches.** A: p.11 — the SUT itself and boundary behaviour (`true`, `true`, `false`, `6`, `7`, `null`); p.13 — hand-rolled black-box vs white-box tests (`testEquality`, `testEnqueueTail`); p.19 — the `checkRep()` representation invariant; p.52 — the same tests rewritten with JUnit lifecycle and assertions via `CheckRepWrapper`; p.54 — `EnqueueTest` + `DequeueTest` grouped into `QueueTestSuite` (Software Testing p.11–54).
- **Q: Which two deck ideas does the lab's "single code-path through a single method" rule fuse?** A: The unit level of the p.8 ladder (smallest scope) and the isolation cure for fragility/dependencies — mocks/stubs at every seam where execution would leave the method (Software Testing p.8, p.57–60; TestLab1 p.1).
- **Q: Why is the 10-book limit an exception but the Queue's `sizeMax > 0` an assertion, although both guard a size?** A: A legitimate user can reach 10 borrowed books — an *expected* condition the program must handle and continue (exception, `TooManyBooksException`, Software Testing p.45–46); no

legitimate caller constructs a zero-capacity queue — only a bug can (assertion, p.11), per Rule 1 and the lab's stop-vs-continue note (Software Testing p.18; TestLab1 p.1).

- **Q: How do the fragile-test sensitivities explain why "test under the UI" works?** A: The GUI layer concentrates the couplings that break tests — widget interfaces (interface sensitivity), pixel/layout behaviour (behavior sensitivity), screen content (data sensitivity), timing/environment (context sensitivity); moving assertions below the UI removes all four couplings at once, converting a manual test into a stable automatic one (Software Testing p.25–30).
- **Q: Relate "Ideas for tests" (p.40) to the acceptance-test slide (p.78).** A: The first idea source — "use case scenarios (missing functions): acceptance tests" — is exactly the agile acceptance mode of p.78: encode the user's use case as an automatic test *before* implementing the story; the Login Admin template (p.79) and the Borrow Book use case (p.41) are the same artefact at two grains (Software Testing p.40, p.78–79, p.41).
- **Q: A team has 100% branch coverage and all tests green. Give three deck-grounded reasons a bug may still ship.** A: (1) Incompleteness in principle — coverage measures executed structure, not correctness on all inputs ("presence, not absence", Software Testing p.6); (2) the bug may live in the *specification* (Mars Orbiter — both sides covered and green, the seam wrong, p.9); (3) output-space classes may be untested despite full branch coverage — e.g. the "Large Integers?" overflow region (p.12).
- **Q: Why does TDD make the p.14 testability checklist partly self-enforcing?** A: Code written to satisfy a pre-existing test is *by construction* callable in isolation — its dependencies had to be injectable for the test to exist; the p.49 "testable system" advantage is the checklist's "module unit tests" item falling out of the process for free (Software Testing p.14, p.49).
- **Q: Which Mockito limitation interacts with JHotDraw's design heritage, and how?** A: Mockito cannot mock final/anonymous classes or primitives (Software Testing p.67); JHotDraw, co-authored by Erich Gamma in the Design Patterns tradition [GHJV94], is interface-driven (p.51), so its seams are mockable — the limitation is a reason to *keep* programming to interfaces in your feature work.
- **Q: Trace one requirement from user wording to regression net using only deck artefacts.** A: User story → acceptance template (Name/Actor/Pre/Main/Alt/Post — Login Admin, p.79) → automated acceptance/unit tests (p.33 login pair) → red/green/refactor growth of the implementation (p.39–48) → suite membership (@RunWith(Suite.class) , p.54) → re-run on every change for immediate regression feedback (p.81) (Software Testing p.79, p.33, p.39–48, p.54, p.81).
- **Q: What single sentence from each source best captures the lecture, and why do they pair?** A: Deck: "Testing can demonstrate the presence of bugs, but not their absence" (Software Testing p.6) — the limit. Lab: "A unit test should test a single code-path through a single method" (TestLab1 p.1) — the practice. The pair says: certainty is unreachable, so maximise evidence per test by keeping each test small, isolated, and repeatable.

Annotated Code Index (every code artifact in the deck)

Every piece of code shown anywhere in the Software Testing deck, indexed by page, with its role and the line(s) that carry the teaching point. Useful when a question quotes a fragment and asks you to identify or complete it.

Production code

- **Queue (Software Testing p.11)** — the bounded FIFO ring buffer; five members: constructor (`assert sizeMax > 0;`, zeroes `head/tail/size`, allocates `data = new int[sizeMax]`), `empty()` (`size == 0`), `full()` (`size == max`), `enqueue(int x)`: `boolean` (rejects with `false` when full; writes `data[tail]`);

wrap if (tail == max) tail = 0;), dequeue(): Integer (returns null when empty; reads data[head]; wrap if (head == max) { head = 0; }). Teaching points: precondition assertion, boundary behaviour, the wrap-around path, and the boxed- Integer null convention.

- **sqrt with assertion (Software Testing p.18)** — `double sqrt(arg){ //...compute result... assert result > 0; return result; }` — the minimal postcondition-assertion example: the invariant "a (real, positive-domain) square root is positive" checked before returning.
- **checkRep() (Software Testing p.19)** — the Queue's representation invariant as four assert clauses (size bounds; tail>head, tail<head, tail==head cases). Teaching point: an invariant is *conditional on pointer order* in a ring buffer — three mutually exclusive geometric cases.
- **borrowBook v1 (Software Testing p.44)** — `public void borrowBook(Book book) { borrowedBooks.add(book); }` — the deliberately minimal "green" implementation: exactly enough to pass `testBorrowBook`, nothing more (the p.39 rule made flesh).
- **borrowBook v2 (Software Testing p.46)** — adds `throws TooManyBooksException` and the guard `if (borrowedBooks.size() >= 10) { throw new TooManyBooksException(); }` before `borrowedBooks.add(book);`. Teaching point: the next failing test (p.45) *forced* this code into existence; the limit is an exception (legitimate user can hit it), not an assertion.
- **my-open wrapper (Software Testing p.31)** — pseudo-code substitution `file=open("/tmp/foo", 'w')` → `file=my-open("/tmp/foo", 'w')` with the policy "succeed 100 times then fail 1% of calls" — the fault-injection seam.

Hand-rolled test code (pre-JUnit)

- **testEquality() (Software Testing p.13)** — `static boolean`, black-box, against `IQueue`: empty → enqueue 10 → enqueue 11 → dequeue==10 → dequeue==11 → empty; reports via `out.println("test1 OK"/"test1 NOT OK")` and boolean return.
- **testEnqueueTail() (Software Testing p.13)** — `static boolean`, white-box, against concrete `Queue`: empty → enqueue 1 → enqueue 2 → enqueue 3 (rejected, full) → `q.getTail() != 0` check; reports `"test2 OK"/"test2 NOT OK"`. The internal-cursor assertion is the white-box signature.

JUnit test code

- **testLoginAdmin() / testWrongPassword() (Software Testing p.33)** — the Arrange-Act-Assert pair on `LibraryApp`: precondition `assertFalse(libApp.adminLoggedIn());` act `adminLogin("adminadmin")` (correct) vs `adminLogin("admin")` (wrong); postconditions `assertTrue / assertFalse` on both the returned boolean and `adminLoggedIn()`. Teaching point: assert state *before and after*, and test the failure path as its own test.
- **testBorrowBook() (Software Testing p.43)** — the main-scenario TDD test: lookups `libApp.userByCprNumber("1234651234")` and `libApp.bookBySignature("Som001")` each immediately re-asserted (`assertEquals(cprNumber, user.getCprNumber())`, `assertEquals(signature, book.getSignature())` — verifying the *fixture* before trusting it); precondition `assertFalse(borrowedBooks.contains(book));` act `user.borrowBook(book);` postconditions `assertEquals(1, borrowedBooks.size())` and `assertTrue(borrowedBooks.contains(book))`. Declared `throws Exception`.
- **EnqueueTest (Software Testing p.52)** — the lifecycle showcase: `private IQueue q;`, empty static `@BeforeClass setUpClass() / @AfterClass tearDownClass(), @Before setUp() = q = new CheckRepWrapper(new Queue(2));`, `@After tearDown() = q.empty();`, and `@Test testEnqueue()` re-running the wrap-around scenario with JUnit assertions, Javadoc'd "Test of enqueue method, of Queue."

- **QueueTestSuite (Software Testing p.54)** — `@RunWith(Suite.class)`, `@Suite.SuiteClasses({dk.sdu.mmmi.fixedsizequeue.EnqueueTest.class, dk.sdu.mmmi.fixedsizequeue.DequeueTest.class})`, plus its own (empty) lifecycle methods each declared `throws Exception`.
- **Exception idioms (Software Testing p.55)** — `try { m(); fail(); } catch(MyException e) { ... }` and `@Test(expected=MyException.class)`; the embedded comments are the explanation ("If we reach here, then the test fails because no exception was thrown"; "Do something to test that e has the correct values").

Mockito code

- **Stubbing test test1 (Software Testing p.70)** — `MyClass test = Mockito.mock(MyClass.class);` then (slide typo) `test.when(test.getUniqueId()).thenReturn(43);` and `// TODO use mock in test...`. Correct form: `Mockito.when(test.getUniqueId()).thenReturn(43);`.
- **Verification test test1 extended (Software Testing p.71)** — adds `Mockito.verify(test).testing(Matchers.eq(12));` ("check if method testing was called with the parameter 12") and `Mokito.verify(test, Mockito.times(2));` ("Was the method called twice?" — slide misspells Mockito and omits the trailing method call).
- **ArticleManagerTest (Software Testing p.75)** — the annotation showcase: two `@Mock` fields (`ArticleCalculator calculator`, `ArticleDatabase database`), one `@Spy` field initialised to a real object (`UserProvider userProvider = new ConsumerUserProvider();`), one `@InjectMocks` SUT (`ArticleManager manager = new ArticleManager();` — "Creates instance of ArticleManager and performs constructor injection on it"), and `shouldDoSomething()` calling `MokitoAnnotations.initMocks(this);` (sic — MockitoAnnotations) then `verify(database).addListener(any(ArticleListener.class));`. Teaching point: `any(Type.class)` is an argument matcher accepting any instance of the type.

Spring code

- **MachineLearningTest (Software Testing p.76)** — `@RunWith(SpringJUnit4ClassRunner.class)`, `@ContextConfiguration(classes = {AppConfig.class})`, `@Autowired @Qualifier("ml") DataModelService ml;`, and `test_ml_always_return_true()` with Hamcrest assertions `assertThat(ml, instanceOf(MachineLearningService.class));` and `assertThat(ml.isValid(""), is(true));`. Teaching point: the container performs the wiring (DI), the test asserts both *which implementation* arrived and *what it does*.

Named Entities Index (people, systems, classes, values)

Everything the sources name, with where and why — designed so a lookup on any proper noun from the lecture lands here.

People and history

- **Alan Turing** — namesake of the Turing machine (p.3) and the halting problem (p.4–5); the theoretical root of testing incompleteness (Software Testing p.3–5).
- **Edsger Dijkstra** — author of the quoted principle "Testing can demonstrate the presence of bugs, but not their absence." (Software Testing p.6; attribution per the standard canon flagged in the grounding note).
- **Erich Gamma** — co-author of JUnit and of *Design Patterns* [GHJV94] (Software Testing p.51).

- **Kent Beck** — co-author of JUnit; eXtreme Programming (and TDD) figure (Software Testing p.51).
- **Mars Climate Orbiter** — the NASA probe lost to a metric (m/s) vs English (ft/s) units mismatch; the deck's exhibit for specification-level faults in the triage tree (Software Testing p.9).

Frameworks, tools, libraries

- **JUnit** — Java's xUnit test framework; unit-, component-, and acceptance tests; junit.org (Software Testing p.51). Lab-pinned to **JUnit 4** for Swing compatibility (TestLab1 p.1).
- **xUnit** — the cross-language family name (Software Testing p.51, p.56).
- **CppUnit** — the C++ family member (Software Testing p.56).
- **Mockito** — the mock framework used with JUnit (Software Testing p.56, p.58); referenced by the lab as [mockito.org] (TestLab1 p.1).
- **Sikuli Script** — image-recognition GUI scripting; the record-and-play tool on the links slide (Software Testing p.30, p.56).
- **DbUnit** — database-fixture tool for known-state DB testing (Software Testing p.56).
- **Fit** — named alongside JUnit as an agile acceptance-test framework (Software Testing p.78).
- **Spring** — DI container demonstrated with `SpringJUnit4ClassRunner` (Software Testing p.76).
- **Maven** — the build/dependency tool through which JUnit 4 is added in the lab (TestLab1 p.1).
- **GCC / LLVM** — production compilers cited for assertion density: ~9,000 and ~13,000 assertions respectively, ~1 per 110 LOC (Software Testing p.20).
- **JAVA, C#, C++, C** — the Turing-complete languages named on the halting-problem slide (Software Testing p.4).

Classes, interfaces, methods (deck examples)

- **Queue / IQueue** — the ring-buffer SUT and its interface; `IQueue` appears in the black-box `testEquality` and as the field type in `EnqueueTest` (Software Testing p.11, p.13, p.52).
- **CheckRepWrapper** — the decorating wrapper that runs `checkRep()` invariant checks around a wrapped `Queue`; built in `EnqueueTest.setUp()` (Software Testing p.52, p.19).
- **EnqueueTest / DequeueTest / QueueTestSuite** — the JUnit classes in package `dk.sdu.mmmi.fixedsizequeue` (Software Testing p.52, p.54).
- **LibraryApp** — the library system with `adminLogin(String)`, `adminLoggedIn()`, `userByCprNumber(String)`, `bookBySignature(String)` (Software Testing p.33, p.43).
- **User / Book** — domain classes: `User.getCprNumber()`, `User.getBorrowedBooks()`, `User.borrowBook(Book)`; `Book.getSignature()` (Software Testing p.43–46).
- **TooManyBooksException** — thrown by `borrowBook` at the 10-book limit (Software Testing p.45–46).
- **MyClass.getUniqueId() / testing(int)** — the Mockito demo methods: `getUniqueId` stubbed to return 43; `testing` verified with argument 12 (Software Testing p.70–71).
- **ArticleManager / ArticleCalculator / ArticleDatabase / UserProvider / ConsumerUserProvider / ArticleListener** — the `@InjectMocks` example's cast (Software Testing p.75).
- **DataModelService / MachineLearningService / AppConfig** — the Spring example's service interface, expected implementation, and configuration class; bean qualifier `"ml"`; method `isValid(String)` (Software Testing p.76).
- **MyException / m()** — the generic exception-testing placeholders (Software Testing p.55).
- **DataServer** — the dependency in the mock storyboard (Software Testing p.61–64).

Literal test data values (recognise them on sight)

- `"1234651234"` — the user's CPR number in Borrow Book (Software Testing p.42–43, p.45, p.47).
 - `"Som001"` — the borrowed book's signature (Software Testing p.42–43, p.45).
 - `"book1"` ... `"book10"` — the ten pre-borrowed books in the alternative scenario (Software Testing p.45).
 - `"adminadmin"` / `"admin"` — correct vs wrong admin password in the login tests (Software Testing p.33).
 - `43` — the stubbed `getUniqueId()` return (Software Testing p.70).
 - `12` / `times(2)` — the verified argument and call count (Software Testing p.71).
 - `6, 7, 8` and `true, true, false, 6, 7, null` — the Queue run's inputs and expected outputs (Software Testing p.11).
 - `10` / `11` and `1, 2, 3` — the values in `testEquality` and `testEnqueueTail` respectively (Software Testing p.13).
 - `"/tmp/foo"` — the file path in the fault-injection example (Software Testing p.31).
 - `9 → -3 or 3, -1 → i, NaN, exception` — the `sqrt` specification examples (Software Testing p.23).
 - `~9,000` / `~13,000` / `1 per 110 LOC` — the GCC/LLVM assertion statistics (Software Testing p.20).
 - `m/s vs ft/s` — the Mars Climate Orbiter units (Software Testing p.9).
-

Synthesis — How Lecture 7's Pieces Interlock

A connected re-telling of the deck for essay-style questions ("discuss the role of testing in maintenance"), showing the dependency chain between its ideas.

From impossibility to discipline

The deck's argument starts at the bottom: Turing-completeness (p.3–4) $\Rightarrow$ halting undecidable $\Rightarrow$ perfect test suites impossible (p.5) $\Rightarrow$ Dijkstra's presence-not-absence principle (p.6). That negative result does not end the lecture — it *organises* it. If exhaustive testing is impossible, the engineering question becomes *how to spend a finite test budget for maximum assurance*, and every subsequent technique is an answer: equivalence partitioning spends one test per behaviour class (p.12); boundary analysis spends extra tests where defects cluster (p.11); white-box knowledge aims tests at internal paths a spec reader would miss (p.13); differential/stress/random testing cover oracle-less, load, and unimagined-input territory respectively (p.8) (Software Testing p.3–13).

From design to testability to doubles

The middle of the deck argues that what you can test is determined by *how the code is built*. The p.14 checklist defines testable construction (clean code, visible interactions, no threads/global swaps/pointer soup, module tests, fault-injection seams, assertions). Two of its items grow into whole sections: **assertions** become the self-checking machinery of p.15–21 (rules, `checkRep`, enable/disable policy, GCC/LLVM scale), and **seams for substitution** become the fragile-test analysis (p.24–28), the under-the-UI strategy (p.29–30), fault injection (p.31), the time-dependency problem (p.32) — and ultimately the entire Mockito apparatus (p.57–75), because a mock can only replace a dependency the design exposes. Mockito's limitations slide (final/anonymous/primitive, p.67) closes the loop back to design: code that cannot be subclassed cannot be doubled, so *design for testability* and *program to interfaces* are the precondition for everything in the doubles section (Software Testing p.14–32, p.57–75).

From automation to TDD to regression

The deck's third movement is temporal: *when* do tests run, and *when* are they written? Automation (p.33) makes tests cheap to run; TDD (p.34–49) moves their *creation* before the code, which yields coverage-by-construction and caller-first design (p.49) and installs refactoring as a perpetual, test-protected third step (p.48). Cheap-to-run plus always-being-added equals a living **regression suite** (p.81), which is what Verification in the maintenance change cycle actually consists of: after every Actualization/Postfactoring, re-run everything; any red is triaged through the p.9 tree [Raj13]. Acceptance tests (p.78–79) bound this machinery from above — they encode what the *user* meant — and the closing mindset slide (p.83) supplies the psychology that keeps the whole system honest: someone must *want* the code to fail for the suite to be adversarial enough to matter (Software Testing p.33–49, p.78–83).

The lab as the deck in miniature

TestLab1 compresses the deck into five actionable steps that trace the same arc: tooling (Maven + JUnit 4 — automation), domain-logic targets (test under the UI), best-case then boundary tests (partitioning + boundary analysis), mocks/stubs at every dependency (single code-path isolation), and assertions on invariants (self-checking code) — finishing with documented verification for the portfolio (the Verification evidence) (TestLab1 p.1). If you can explain *why each lab step exists* by pointing at its deck pages, you have synthesised the lecture.

Exam-Style Long-Form Questions with Model Answers

Q1. "Why can't a passing test suite prove a program correct? What should an engineer do about it?" (10 marks)

Model answer. Correctness-proof by testing is theoretically impossible: mainstream languages (Java, C#, C++, C) are Turing-complete (Software Testing p.4), so they inherit the halting problem — no algorithm can decide arbitrary program behaviour, hence "it is theoretically impossible to create a perfect test suite" (Software Testing p.5). Dijkstra's formulation: "Testing can demonstrate the presence of bugs, but not their absence" — a failing test proves a bug exists, a passing test only vouches for the tested inputs; "residual bugs can still hide in the code" (Software Testing p.6). What to do: aim for *adequacy*, not proof — "well designed tests come close to be adequate" (p.5). Concretely: partition the input space into equivalence classes and test one representative each (p.12); add boundary-value tests at every class edge, where defects cluster (p.11); use white-box knowledge to reach internal paths like the Queue's tail wrap (p.13); cover oracle-less cases with differential testing, load with stress testing, unimagined inputs with random testing (p.8); embed assertions so the code self-checks on every run (p.18–20); and automate everything so the whole suite re-runs on every change as a regression net (p.81).

Q2. "Your test fails. Describe a disciplined diagnosis before changing any code." (8 marks)

Model answer. Walk the p.9 decision tree in order. (1) **Bug in the SUT?** — the common case; fix locally. (2) **Bug in the acceptability test?** — the oracle/setup may encode a wrong expectation; "fixing" correct production code against a wrong test injects a real defect. (3) **Bug in the specification?** — both code and test may faithfully implement a flawed contract; the canonical case is the Mars Climate Orbiter, where one team computed in m/s and the other in ft/s — each side internally consistent, the defect living in the agreement between them (Software Testing p.9). (4) **Bug in the OS, compilers, libs, hardware?** — rare but real platform faults. Order matters: cheap-and-likely first, exotic last. Also classify the failure against the fragile-test sensitivities — if the break traces to a renamed method (interface sensitivity), changed shared data

(data sensitivity), or the clock/locale (context sensitivity) rather than a defect, the *test* needs decoupling, not the code (Software Testing p.25–28).

Q3. "Define TDD, give its cycle and rules, and trace the deck's Borrow Book example through one full iteration." (12 marks)

Model answer. TDD = "test before the implementation"; tests are "expectations on software"; it applies to "unit, component, and system tests" (Software Testing p.35). Cycle: **red** (create a failing test) → **green** (make it pass) → **refactor** (clean up), repeated "for functionality, bug, ..." until "no more ideas for tests"; rules: one test at a time; implement only as much code so the test does not fail; if the method looks incomplete, add more failing tests that force more code (Software Testing p.39). Trace: the use case "borrow book" (actor: user; main scenario: borrow; alternative: already 10 books → error, p.41) becomes test data (CPR "1234651234", signature "Som001") and a test case (p.42), coded as `testBorrowBook()` with precondition `assertFalse(borrowedBooks.contains(book))` and postconditions `size==1` and `contains==true` (p.43) — *red*, since `borrowBook` doesn't exist. *Green* is the minimal `borrowedBooks.add(book)`; (p.44). The next red is the alternative scenario: borrow "book1" ... "book10", then expect `TooManyBooksException` on "Som001" (p.45); green adds the `size() >= 10` guard (p.46). A defect test follows: `borrowBook(null)` must leave the count unchanged (p.47). *Refactor* then restructures without changing functionality (DRY), legitimised by the green suite (p.48). Advantages earned: coverage by construction and caller-first design (p.49).

Q4. "Compare stub, mock, and spy, and show the Mockito mechanics for each." (10 marks)

Model answer. Stub: "objects holding predefined data to provide responses during tests... the lightest and most static test doubles" (Software Testing p.60) — used for state verification; mechanics: `when(test.getId()).thenReturn(43)` (p.69–70). **Mock:** "objects that store method calls... record and verify the interaction... the most powerful and flexible" (p.59) — used for behaviour verification; mechanics: created with `mock()` or `@Mock` (+ mandatory `MockitoAnnotations.initMocks(this)` for annotations, p.65), checked with `verify(test).testing(Matchers.eq(12))` and `times(2)` counts (p.71), tightened with `verifyNoMoreInteractions()` (p.74). **Spy:** "partially mock objects... the real object remains unchanged, and we just spy some specific methods" (p.72); created with `@Spy` / `spy()`; "every call, unless specified otherwise, is delegated to the object" (p.73). `@InjectMocks` wires all of the above into the SUT by constructor injection on type (p.74–75). Choose the lightest double that suffices; remember Mockito cannot double final classes, anonymous classes, or primitives (p.67), so design to interfaces.

Q5. "Why does the course insist on automated tests, and what is regression testing's role in software maintenance?" (8 marks)

Model answer. "Manual tests should be avoided — they are expensive (time and personal) to execute: can't be run often" (Software Testing p.81). Automated tests are cheap and run "as soon something is changed in the system," giving "immediate feedback if a code change introduced a bug → regression tests" (p.81). In maintenance terms [Raj13]: the change mini-cycle ends in Verification, and its Prefactoring/Postfactoring steps are *only safe* if a suite can instantly confirm behaviour preservation (the deck's own point that refactoring "requires good testsuite," p.48). The residual difficulty is the UI — "more difficult (but not impossible) when they include the UI" — solved by testing under the UI so the bulk of the suite stays fast and stable (p.81, p.29). Acceptance tests automated in the agile mode ("created before the user story is implemented," JUnit/Fit, p.78) extend the same regression net up to the requirements level.

Q6. "When is an assertion the right tool, and when is it the wrong one? Include the enable/disable question." (8 marks)

Model answer. Right: checking invariants — "executable check for a property that must be true" (Software Testing p.18) — i.e. conditions only a *bug* could violate: `assert result > 0` after `sqrt` (p.18), `assert sizeMax > 0` in the Queue constructor (p.11), the four-clause ring-buffer `checkRep()` (p.19). Benefits: self-checking code, failing early close to the bug, executable documentation of pre/postconditions and invariants (p.20) — at industrial scale (GCC ~9,000, LLVM ~13,000, ~1 per 110 LOC, p.20). Wrong: error handling — Rule 1 — expected conditions like wrong passwords or the 10-book limit take exceptions (`TooManyBooksException`, p.46), because "an assertion should stop the program from running, but an exception should let the program continue running" (TestLab1 p.1). Also wrong: side-effecting checks (Rule 2 — behaviour would change when assertions are disabled) and trivial checks (Rule 3, `assert 1+1==2`) (p.18). Enable/disable: Java assertions are off by default (p.16); disabling buys speed and keeps-running behaviour but loses the early-fail net — "even in production code, may be better to fail early" (p.17); the deck's policy: keep on where failing early is recoverable, disable only in mission-critical stages where continuing beats recovering (p.21).

Key Quotations Bank (verbatim lines worth memorising)

The lines below are quoted exactly as the sources print them (including the slides' occasional informal grammar). Memorise the bolded core of each; exam answers that reproduce these phrasings signal precise knowledge of the course material.

Foundations

- "A Turing machine is a simple mathematical model of computation. It manipulates symbols on a tape based on rules, yet can implement any computer algorithm." (Software Testing p.3)
- "Theoretical reason for testing incompleteness" / "It is **theoretically impossible to create a perfect test suite**" (Software Testing p.5)
- "techniques of the testing **cannot guarantee a complete correctness** of software" / "**well designed tests come close to be adequate**" (Software Testing p.5)
- "**Testing can demonstrate the presence of bugs, but not their absence.**" (Software Testing p.6)
- "Residual bugs can still hide in the code, undetected by tests, as **no test suite guarantees an error-free program.**" (Software Testing p.6)
- "Mars climate orbiter — Metric: m/s — English: ft/s" (Software Testing p.9)

Testability and assertions

- "Assertions, Assertions Assertions !!!" (Software Testing p.14)
- "Assertion: **Executable check for a property that must be true (invariant)**" (Software Testing p.18)
- "Rule1: Assertions are **not for error handling**" / "Rule2: **NO SIDE EFFECTS**" / "Rule3: **No silly assertions**" (Software Testing p.18)
- "Make code **self-checking**, leading to effective testing" / "Make code **fail early, closer to the bug**" / "**Document** assumptions, preconditions, postconditions and invariants" (Software Testing p.20)
- "GCC: ~9000 assertions — LLVM: ~13,000 assertions — One assertion per 110 L.O.C" (Software Testing p.20)

- "Running Software that can be **recovered by failing early**" / "**Disable in mission critical stage** when it is better to continue then recover" (Software Testing p.21)
- "Even in production code, **may be better to fail early**" (Software Testing p.17)

TDD

- "**Test before the implementation**" / "**Tests = expectations on software**" / "All kind of tests: **unit, component, and system tests**" (Software Testing p.35)
- "**red: Create a failing test — green: Make the test pass — refactor: clean up your code**" / "Until: **no more ideas for tests**" (Software Testing p.39)
- "**One test at a time**" / "**Implement only as much code so that the test does not fail.**" / "add more failing tests that **force you to implement more code**" (Software Testing p.39)
- "Code experiments: '**How does the system behave, if . . .**'" / "**Make a list of new test ideas**" (Software Testing p.40)
- "Restructure the system **without changing its functionality**" / "remove code duplication (**DRY principle**)" / "**Requires good testsuite**" (Software Testing p.48)
- "**Good code coverage: Only write production code to make a failing test pass**" / "**Helps design the system: defines usage of the system before the system is implemented**" (Software Testing p.49)

JUnit and Mockito

- "JUnit is a **framework for writing tests** — Written by **Erich Gamma (Design Patterns)** and **Kent Beck (eXtreme Programming)**" (Software Testing p.51)
- "Mockito is a **popular mock framework** which can be used **in conjunction with JUnit.**" (Software Testing p.58)
- "Mocks are the objects that **store method calls... record and verify the interaction... the most powerful and flexible** version of the test doubles." (Software Testing p.59)
- "Stubs are objects holding **predefined data... the lightest and most static** test doubles." (Software Testing p.60)
- "you must initialize mock objects with an **MockitoAnnotations.initMocks(this)** method call" (Software Testing p.65)
- "It can not test the following constructs: **Final classes, Anonymous classes, Primitive types**" (Software Testing p.67)
- "**when(...).thenReturn(...)** can be used to specify a condition and a return value for this condition." (Software Testing p.69)
- "In spying, the **real object remains unchanged**, and we just **spy some specific methods** of it." (Software Testing p.72)
- "**Every call, unless specified otherwise, is delegated to the object.**" (Software Testing p.73)
- "The **verifyNoMoreInteractions()** allows you to check that **no other method was called.**" / "**@InjectMocks** ... tries to do **constructor dependency injection based on the type**" (Software Testing p.74)

Acceptance, regression, mindset

- "Tests defined **by/with the help of the user** based on the **requirements**" (Software Testing p.78)
- "automatic tests: JUnit, Fit, . . . — created **before** the user story is implemented" (Software Testing p.78)

- **"Manual tests should be avoided** — They are expensive (time and personal) to execute: **Can't be run often**" (Software Testing p.81)
- **"immediate feedback if a code change introduced a bug → Regression tests"** (Software Testing p.81)
- "More difficult (but not impossible) when they include the UI — Solution: **Test under the UI**" (Software Testing p.81)
- "Developer: **'I want this code to succeed.'** — Tester: **'I want this code to fail.'** — **Hybrid**" (Software Testing p.83)
- **"Test creatively"** / **"Don't ignore the weird stuff"** (Software Testing p.83)

TestLab1

- "unit testing is a software testing method by which **individual units of source code** ... are tested to determine whether they are **fit for use**" (TestLab1 p.1)
- "In object-oriented programming, a unit is often an **entire interface, such as a class**, but could be an **individual method**." (TestLab1 p.1)
- "By writing tests **first for the smallest testable units, then the compound behaviors between those**, one can build up comprehensive tests for complex applications." (TestLab1 p.1)
- "A unit test should test a **single code-path through a single method**." (TestLab1 p.1)
- "When the execution of a method **passes outside of that method, you have a dependency** and should apply **mocks/stubs** to avoid the dependency" (TestLab1 p.1)
- "Assertions should be used to check **something that should never happen**" (TestLab1 p.1)
- "an assertion should **stop the program from running**, but an exception should **let the program continue running**." (TestLab1 p.1)
- **"Document how you have verified your Feature."** (TestLab1 p.1)

Source Map

Pages	Topic
p.1–2	Title — "Software Testing: How to make software fail!"; Introduction
p.3–6	Turing machine; halting problem; testing incompleteness; Dijkstra's principle
p.7	Basic test model: inputs → SUT → outputs → oracle ("Outputs OK?")
p.8	Kinds of testing map: unit/integration/system, differential/stress/random, white vs black box
p.9	Fault-location decision tree; Mars Climate Orbiter (units mismatch)
p.10–11	Testing example; Fixed Size Queue code + boundary run (<code>true,true,false,6,7,null</code>)
p.12	Equivalent tests; input/output space (equivalence partitioning)
p.13	White-box "Testing the Queue" (<code>testEquality</code> , <code>testEnqueueTail</code>)
p.14	Creating testable software (design-for-testability checklist)
p.15–21	Assertions: enable/disable, three rules, <code>checkRep</code> , why/when, production stats (GCC/LLVM)
p.22–23	Software Under Test; specifications (domain→range, <code>sqrt</code> family)
p.24–28	Fragile test problem: interface / behavior / data / context sensitivity
p.29–30	Testing under the UI; testing a GUI; record-and-play

Pages	Topic
p.31–32	Fault injection; time-dependent problems
p.33	Automatic tests as code (login success/failure)
p.34–40	TDD: traditional vs TDD, red/green/refactor cycle, ideas for tests
p.41–47	TDD worked example: Borrow Book (use case, tests, implementations, defect test)
p.48–49	Refactoring as TDD step (DRY); TDD advantages
p.50–56	JUnit: intro (Gamma & Beck), lifecycle annotations, assertions, suites, exception tests, related tools
p.57–60	Mock objects: Mockito intro; mock vs stub definitions
p.61–71	Mockito mechanics: problem/DataServer/mock, <code>mock()</code> / <code>@Mock</code> , <code>when().thenReturn()</code> , <code>verify()</code> , <code>times()</code>
p.72–75	Spy / partial mocks; <code>@Spy</code> , <code>@InjectMocks</code> , <code>verifyNoMoreInteractions</code> , full example
p.76	Spring + JUnit (DI in tests, <code>@Autowired</code> , Hamcrest <code>assertThat</code>)
p.77–79	Acceptance tests: definition, traditional vs agile, template (Login Admin)
p.80–81	Manual vs automated tests; regression testing; test under the UI
p.82–83	Summary; "Being Great at Testing" — developer/tester/hybrid mindset
TestLab1 p.1	Lab: unit-test definition, objectives, classwork (JUnit 4, boundary cases, mocks/stubs, assertions/invariants), portfolio task

Coverage note

Every page of both source documents is represented in this guide: all 83 pages of the Software Testing deck (each treated individually in the Slide-by-Slide Walkthrough, and thematically in Key Concepts, the tables, the indexes, and the question banks) and the single page of TestLab1 (treated in full in "TestLab1 — The Lab Handout in Full" and threaded through the JHotDraw, pitfalls, and self-test sections). Divider and diagram-only pages are explicitly inventoried in "Divider and diagram-only pages at a glance" so that quotation-bearing pages and visual-only pages are never confused. For any fragment of slide text, code, or test data, the fastest lookups are: verbatim sentences → Key Quotations Bank; code → Annotated Code Index; names and literal values → Named Entities Index; page number → Slide-by-Slide Walkthrough (Software Testing p.1–83; TestLab1 p.1).